

Formation Spark & Databricks



COURSE OBJECTIVES

By the end of this course, you will be able to effectively use Databricks to get business value out of your data.

- Understand the basics of Spark architecture
- Read and write data in Databricks in a variety of different formats
- Know to navigate the different interfaces of Databricks
- Query data using Spark SQL
- Transform data using the basics of Pyspark
- Connect Databricks to BI Tools and Azure Data Factory
- Understand versioning with Databricks
- Understand how to read the Spark UI
- Debug and Optimize Spark Jobs
- Create tables using Delta format
- Automate Tasks using Workflows
- Understand the limitations of Databricks

PRE-REQUISITES

Here is the list of pre-requisites :

- Knowledge of basic SQL (select, from, group by, order by, having)
- Basic knowledge of cloud platform
- Good to have: Knowledge of working with Jupyter notebooks

Let's start with a tour of the table....

- Experience of Databricks/ Big Data/Spark/Cloud ?
- Data Engineer / Data Scientist difference ?
- CGI and Polytech

Agenda

- Introduction to Spark
- Why use Databricks?
- Databricks Interface
- Reading and Writing Files
- Query data using Spark SQL
- Transform data with Pyspark
- Connectors and Azure Ecosystem (5 min)
- DBFS and Delta tables (45 min)
- Workflow (10 min)
- Problems with Spark (10 min)
- Pandas on Spark (10 min)
- Conclusions et Quizz (30 min)
- Databricks Academy (2h)

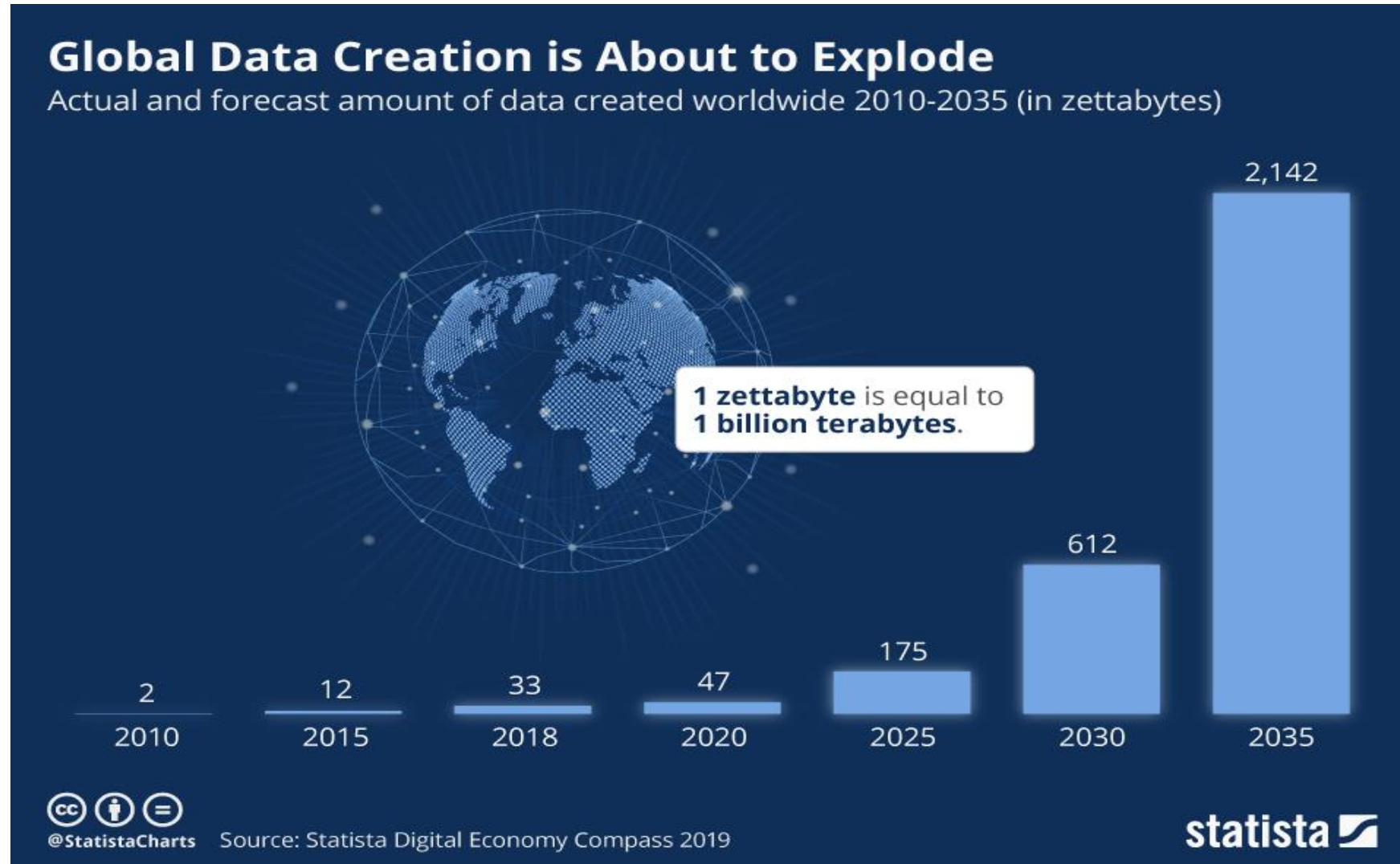
Introduction to Spark



OBJECTIVES

- Understand what is Spark
- Understand why do we use Spark
- Understand key terms in Spark

EXPLOSION OF VOLUME OF DATA



3V OF DATA

Volume

The amount of data being generated



Velocity

The speed at which data is generated



Variety

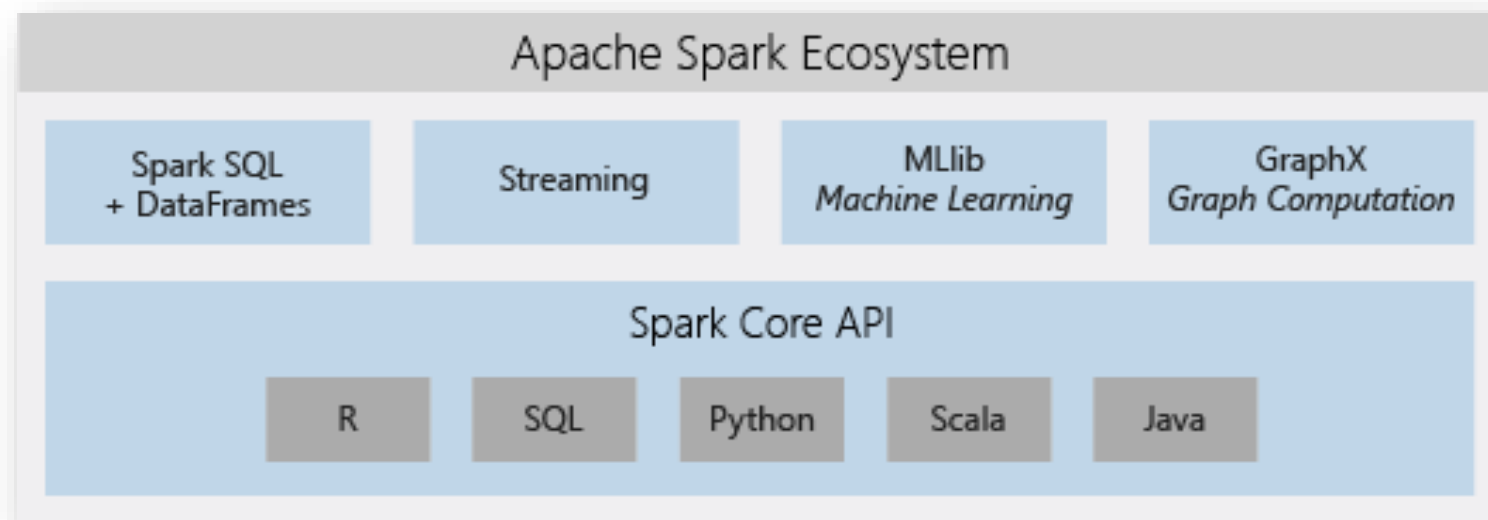
The different types of data generated



WHAT IS SPARK ?

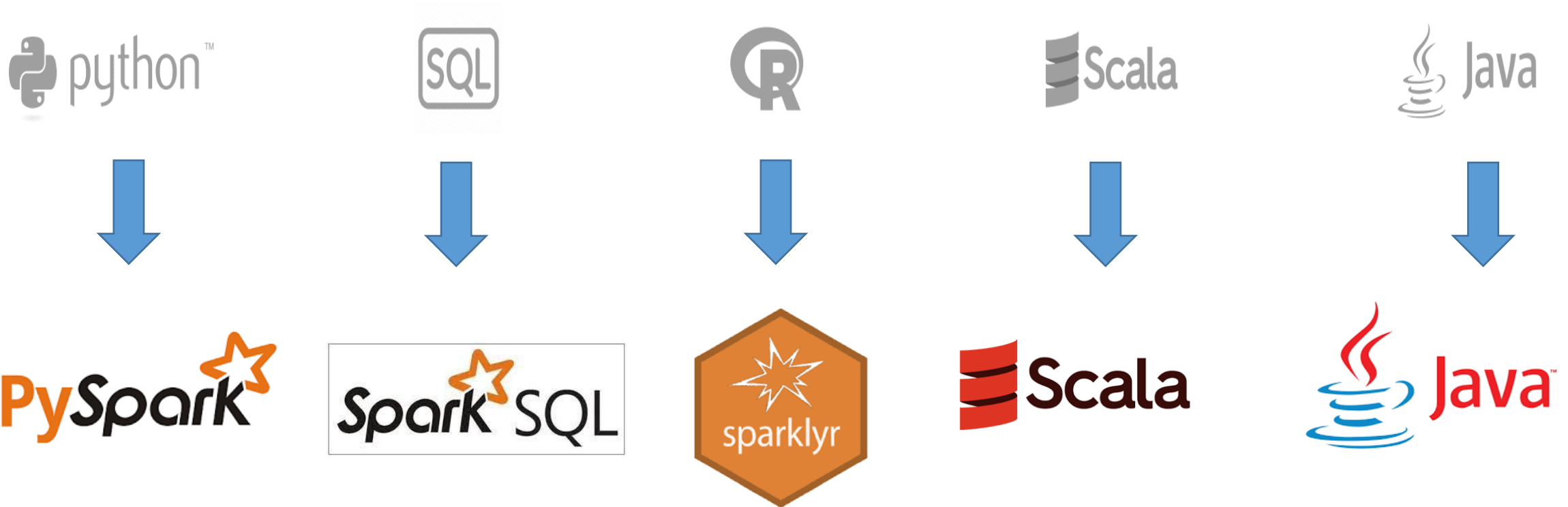
Apache Spark is the most active open-source data processing engine/ framework built for speed, ease of use, and advanced analytics.

Spark is accessible via unified APIs in popular programming languages, including Scala, Java, Python, and R.

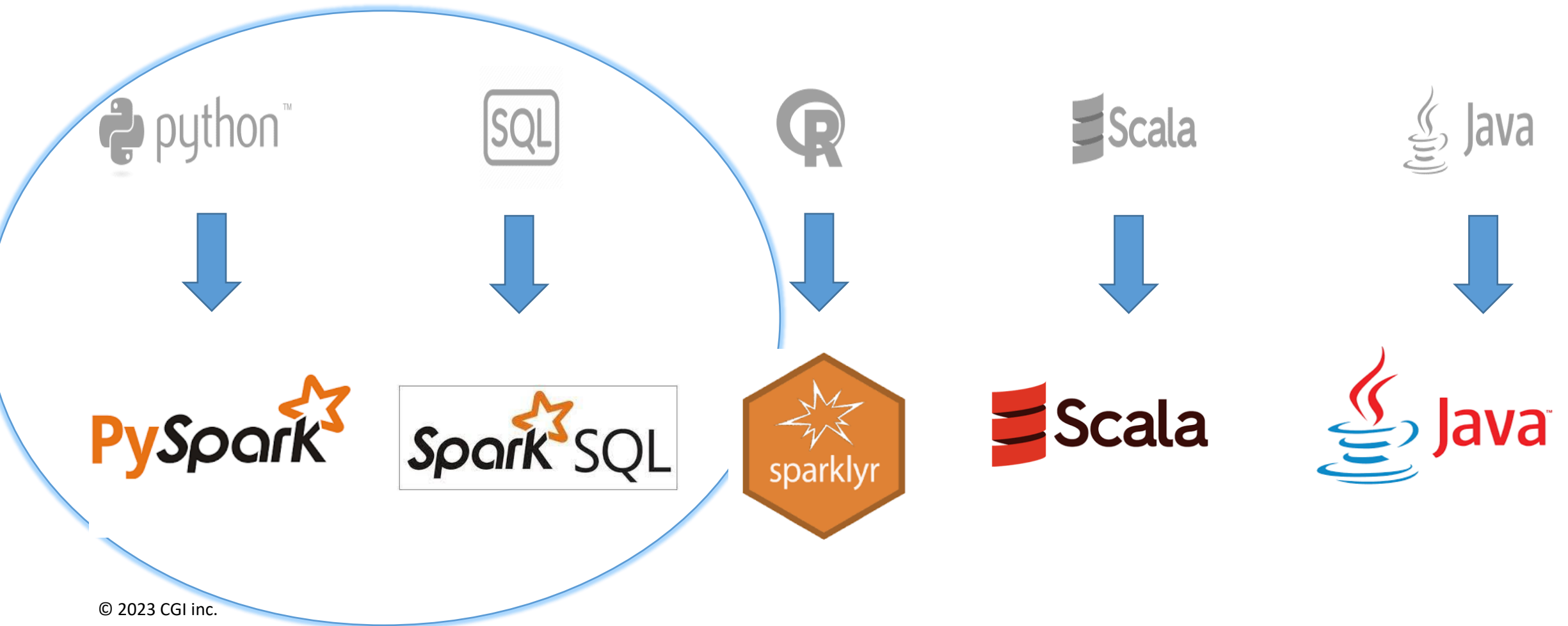


CODE IN THE LANGUAGE OF YOUR CHOICE

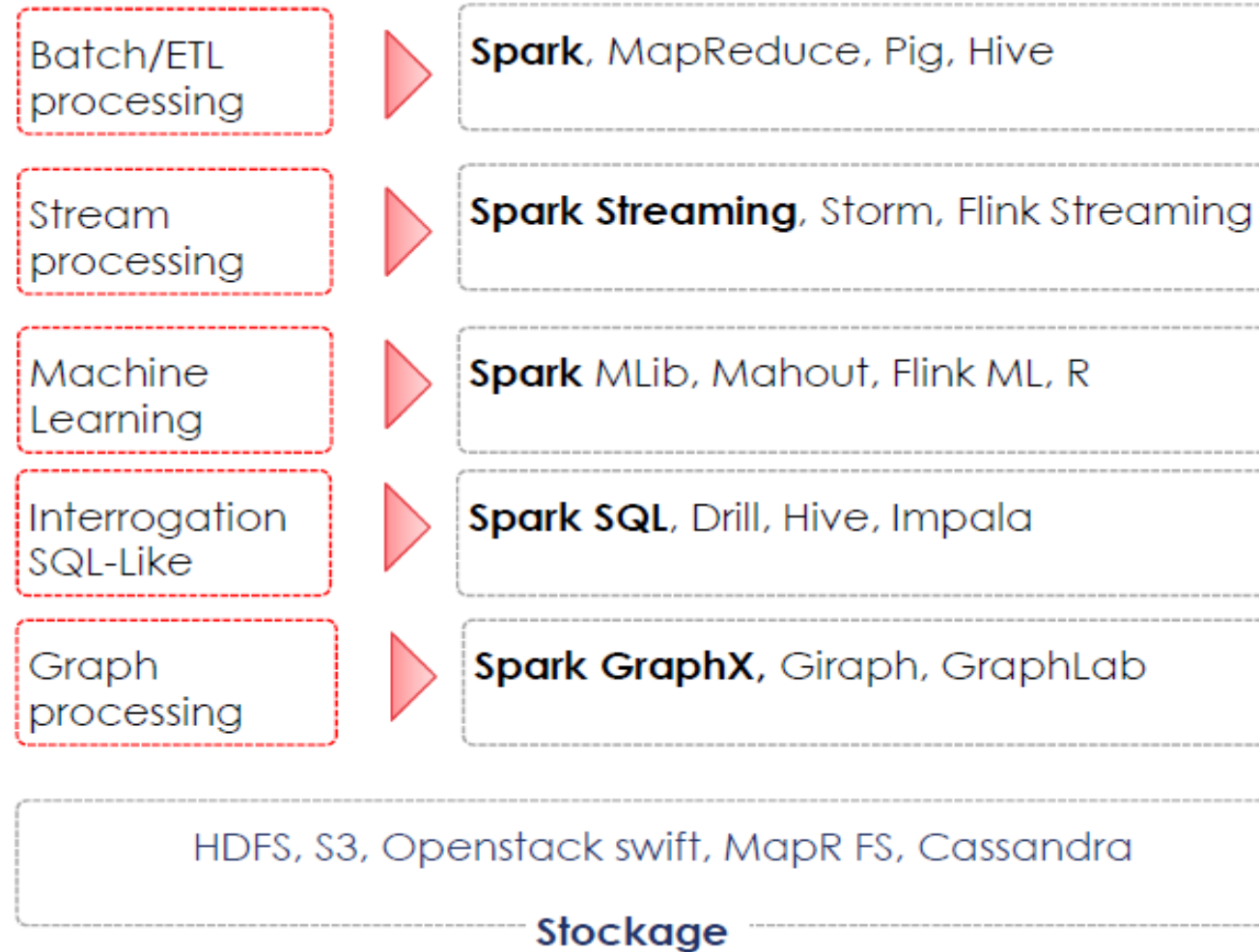
Spark allows to distribute calculation on these languages to make them more efficient at treating data



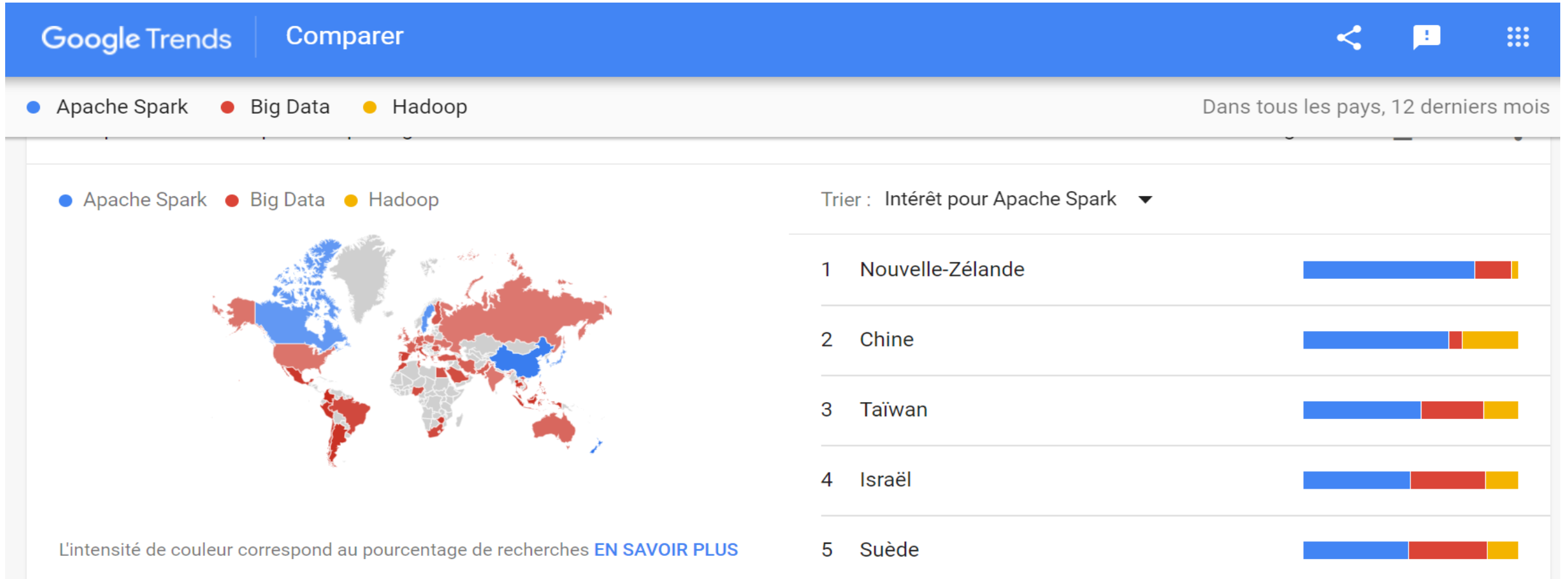
IN THIS COURSE



SPARK POSITION IN TODAY'S WORLD

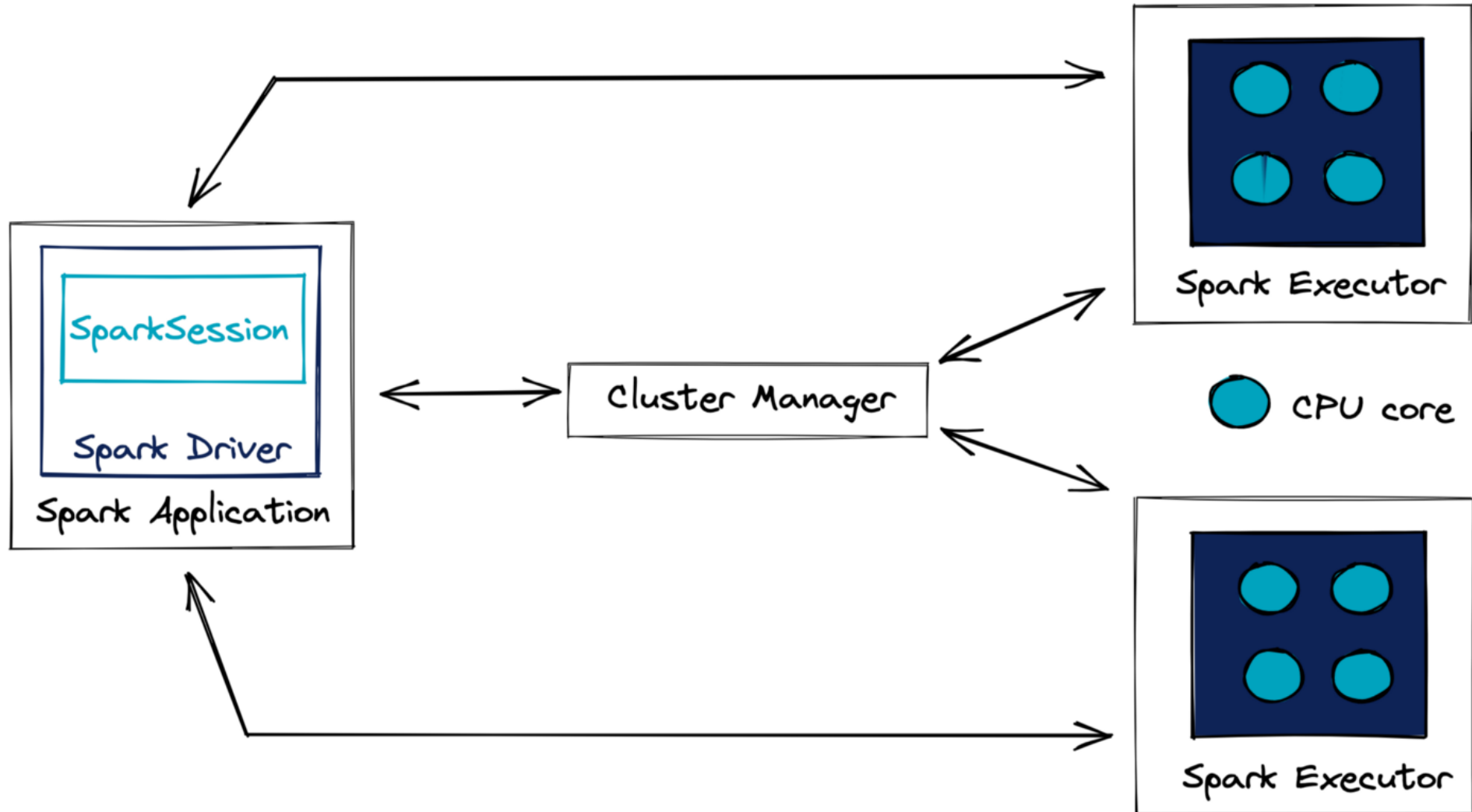


THE WORLD WANTS SPARK AND BIG DATA SOLUTIONS



HOW DOES SPARK FUNCTION ?

SPARK PROCESSES DATA IN A DISTRIBUTED COMPUTING CLUSTER



SPARK PROCESSING IN A DISTRIBUTED COMPUTING CLUSTER

The *Spark driver*, also called the master node, **orchestrates the execution of the processing and its distribution** among the *Spark executors* (also called *slave nodes*).

The *driver* is not necessarily hosted by the computing cluster, it can be an external client. The *cluster manager* manages the available resources of the cluster in real time. With a better overview than the Spark application, it allocates the requested resources to the Spark driver if they are available.

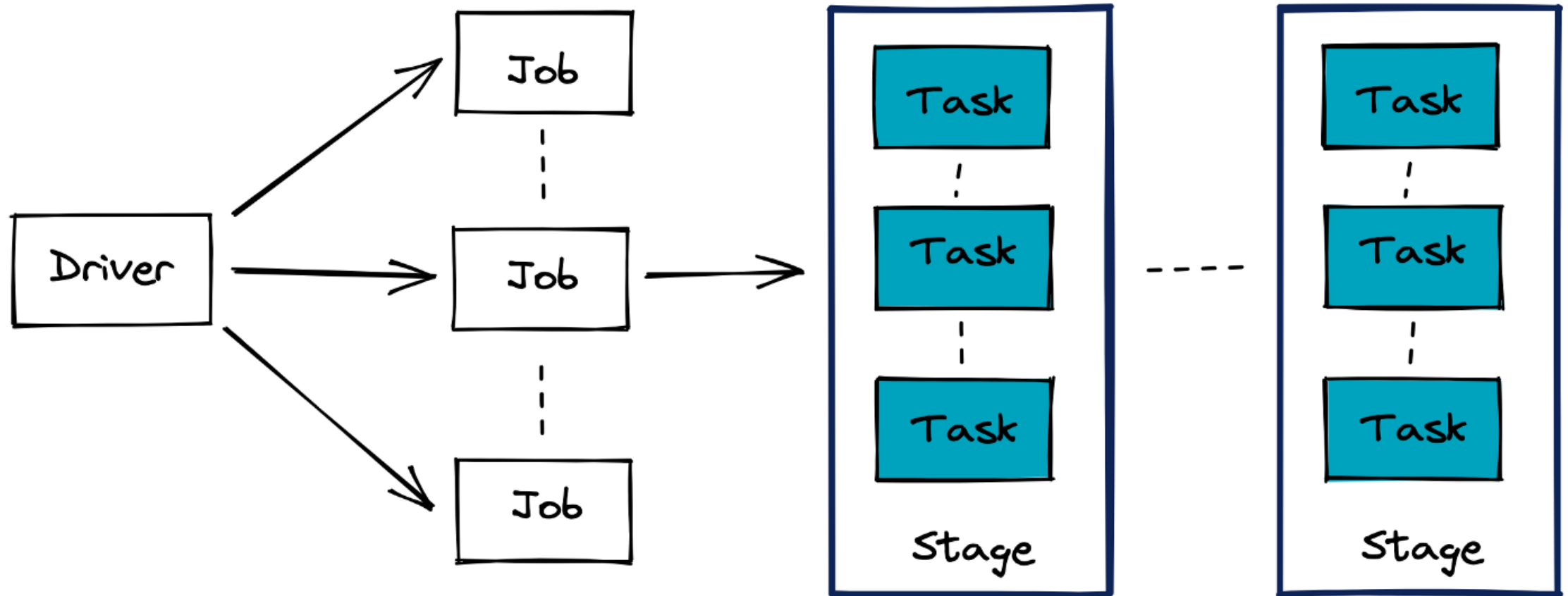
DECOMPOSITION OF A SPARK JOB

A Spark job is a sequence of **stages** that are composed of **tasks**.

An example of a *Spark job* is an Extract Transform Load (ETL) data processing pipeline.

Finally, a **task is a unit of execution** in Spark that is assigned to a partition of data.

DECOMPOSITION OF A SPARK JOB



BUT WHY USE SPARK ?

Spark is Resilient:

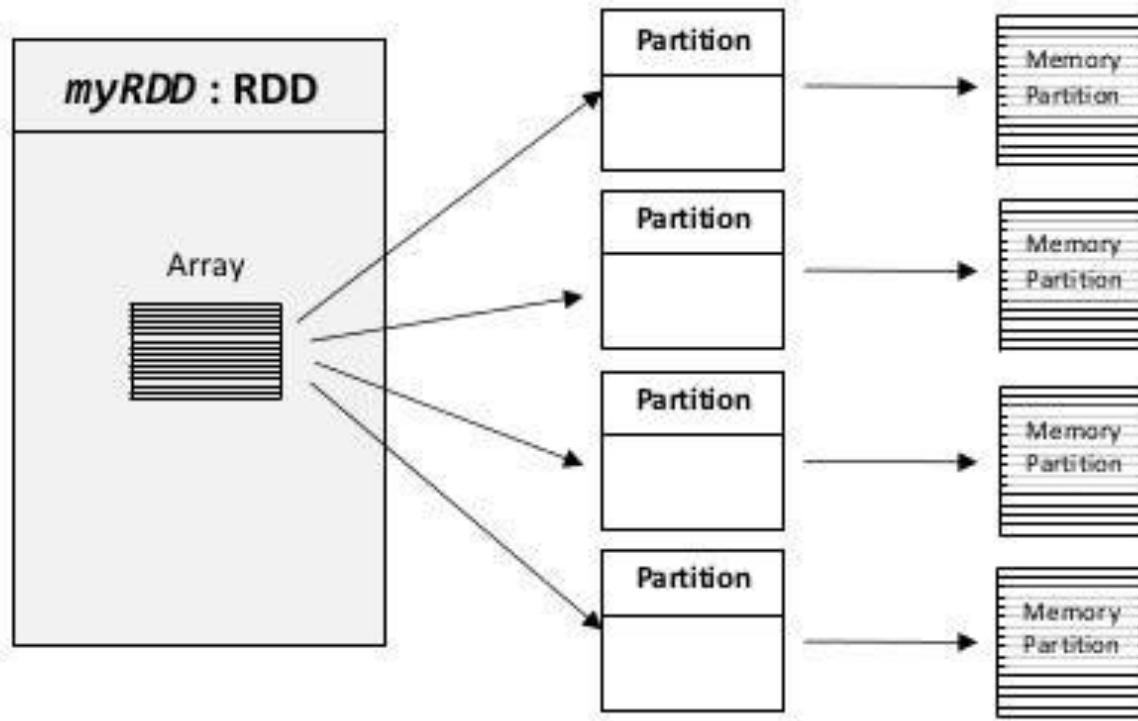
If a machine on the cluster falls, the calculation continues on other machines.

Spark is Distributed:

The data is distributed on the cluster. Spark knows how to place the calculation on different nodes of the cluster in an autonomous and optimised manner.

SPARK HAS RESILIENT DISTRIBUTED DATASETS

What is an RDD?



Some RDD Characteristics

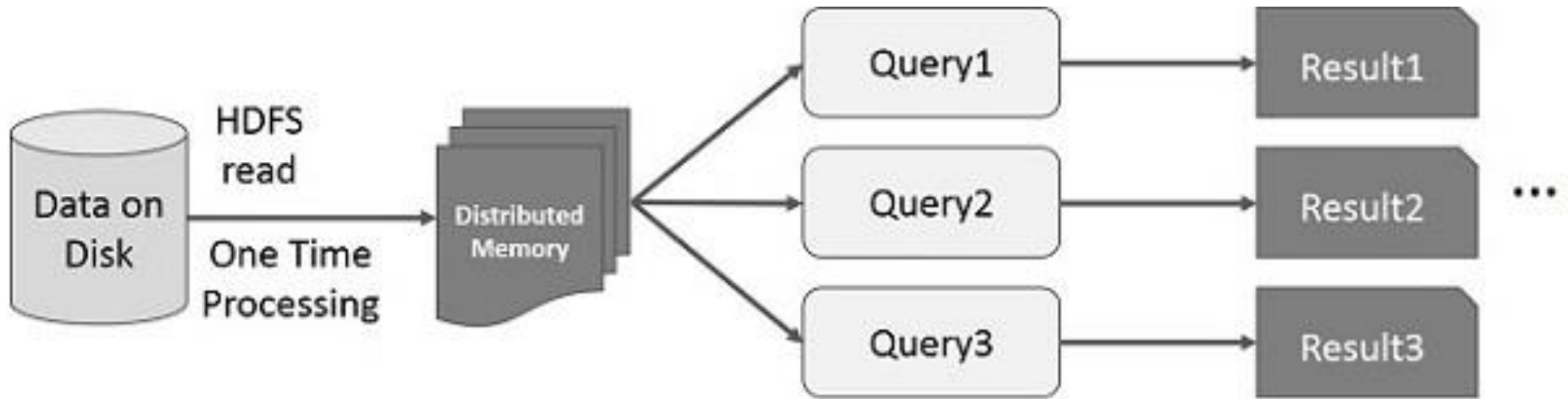
- Hold references to Partition objects
- Each Partition object references a subset of your data
- Partitions are assigned to nodes on your cluster
- Each partition/split will be in RAM (by default)

Source:

<https://www.slideshare.net/sparkInstructor/apache-spark-rdd-101>

SPARK IS DISTRIBUTED.....

When a task is **distributed** in Spark, it means that the data being operated on is split across different nodes in the cluster, and that the tasks are being performed concurrently.



If different queries are run on the same set of data repeatedly, this particular data can be kept in memory for better execution times.

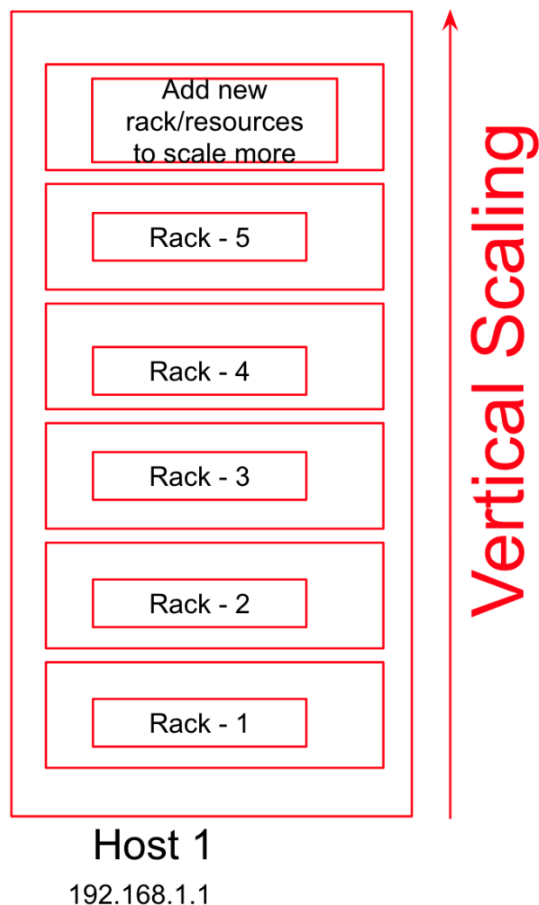
MAKING IT IDEAL FOR CHARGING BIG DATASETS

One of the use-cases of Spark was to avoid Out Of Memory limitations caused by Pandas library when trying to upload big data onto a single node.

Now a single 10GB dataset can be distributed onto many nodes in Spark.

```
pandas\parser.pyx in pandas.parser.textreader._conver
pandas\parser.pyx in pandas.parser.TextReader._conver
pandas\parser.pyx in pandas.parser.TextReader._conver
pandas\parser.pyx in pandas.parser._try_int64 (pandas
MemoryError: Memory ERROR.
```

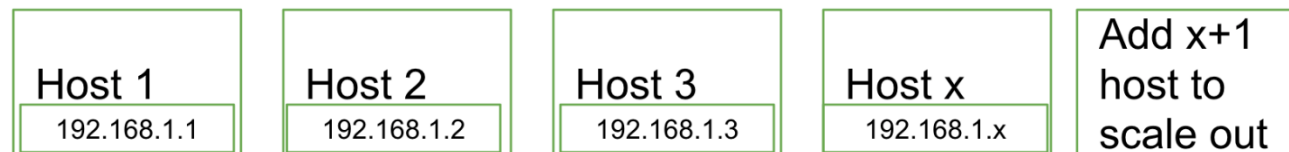
SPARK IS SCALABLE.....



To scale more, Add more RAM, CPU, Memory to the **one existing machine**

Horizontal Scaling

To scale more: Add more machines to existing **group of distributed system**



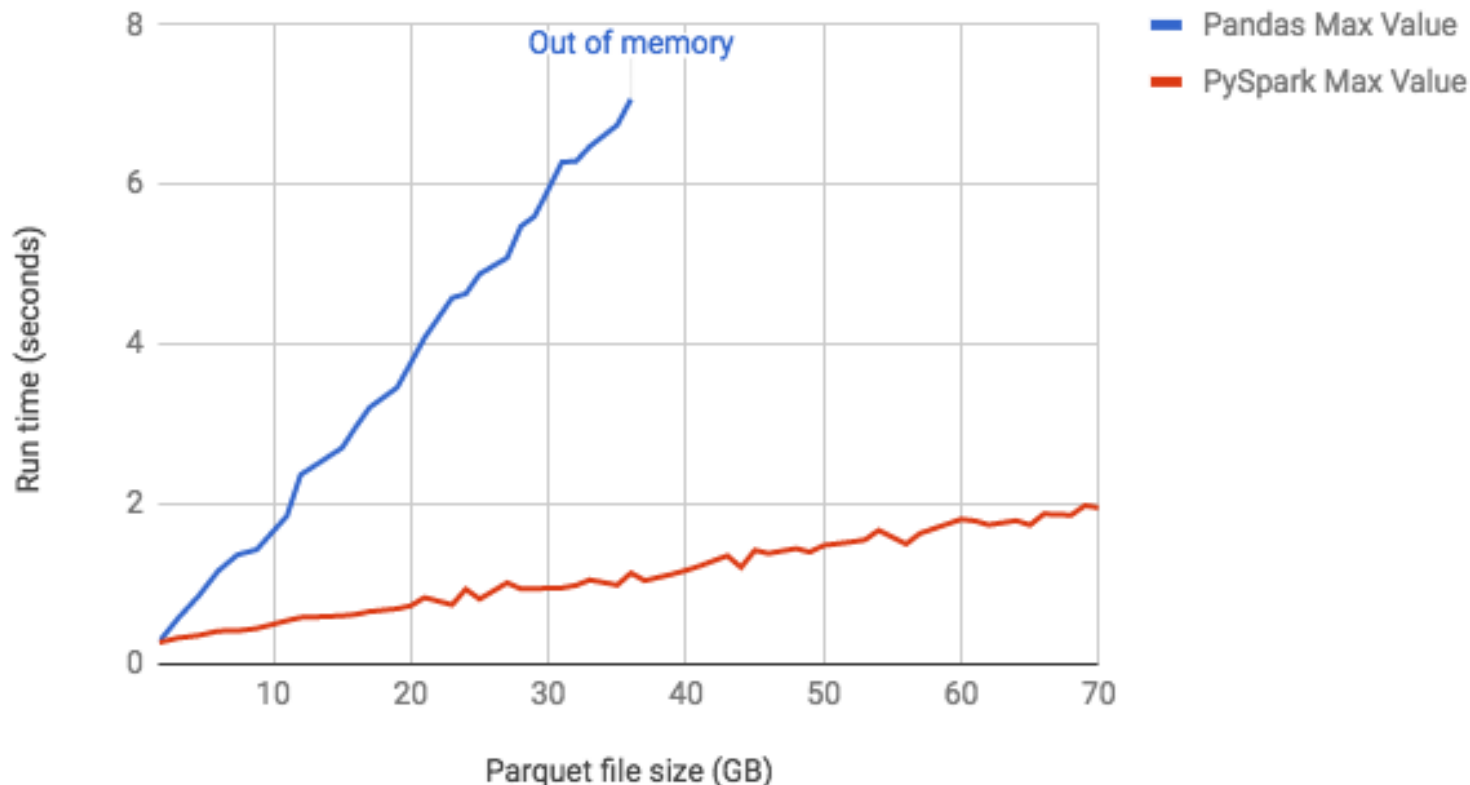
This means Spark can scale up to treat massively big datasets through distributed calculation.

LEADING TO CHEAPER AND HIGHER PERFORMANCE

PySpark ran in local cluster mode with **10GB** memory and 16 threads.

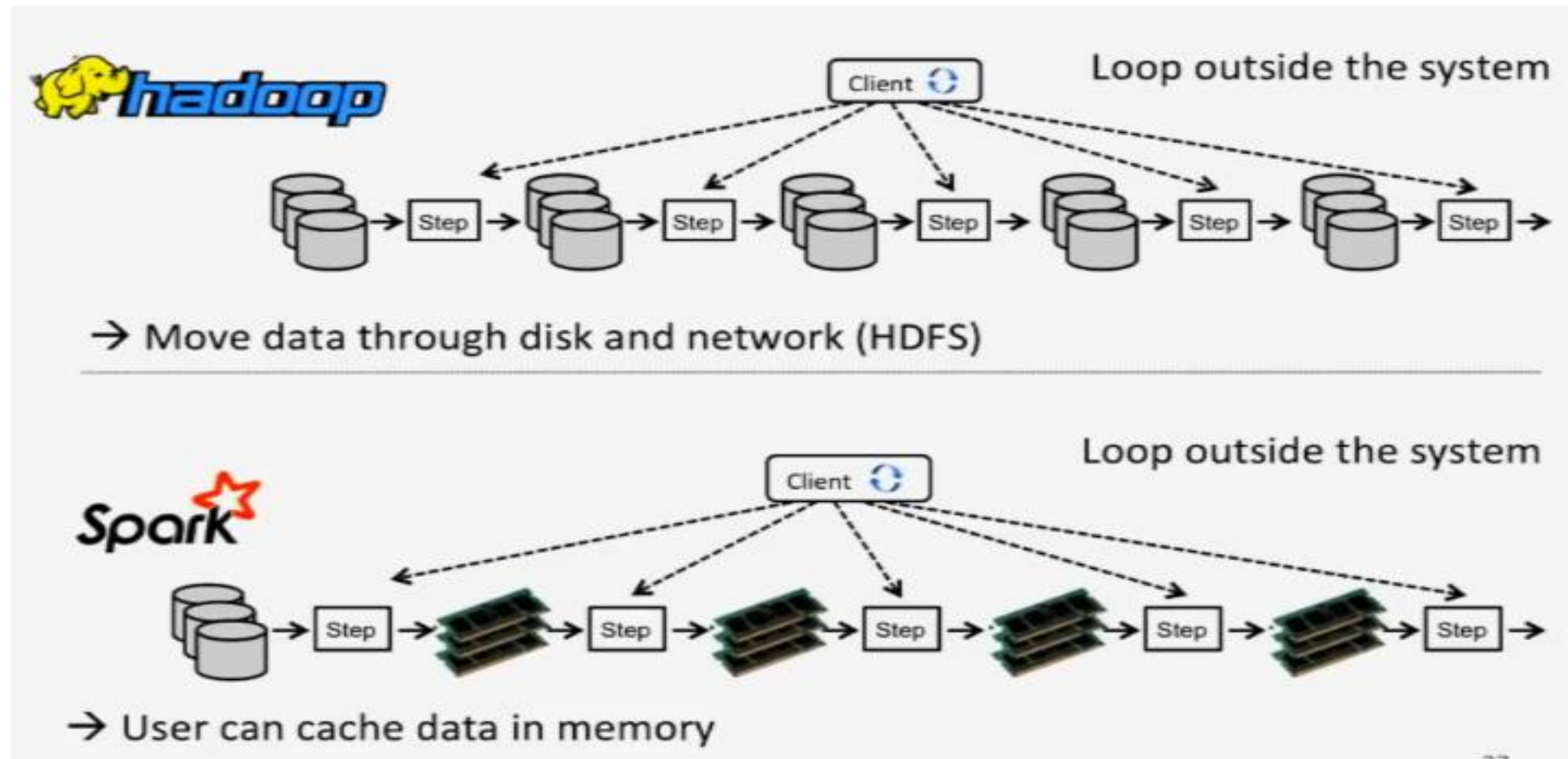
We observed that as the input data size increased, PySpark achieved the better performance result with limited resources, while Pandas crashed and failed to handle parquet files larger than 39GB.

Pandas VS PySpark: max value



Source : <https://www.databricks.com/blog/2018/05/03/benchmarking-apache-spark-on-a-single-node-machine.html>

SPARK CACHES DATA IN MEMORY.....

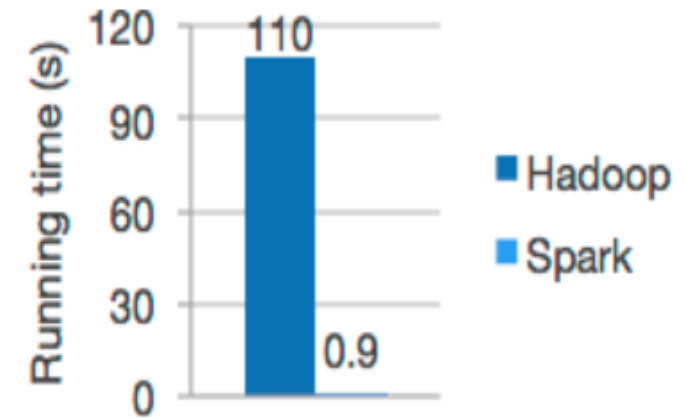


....MAKING IT REALLY FAST

Speed

Run workloads 100x faster.

Apache Spark achieves high performance for both batch and streaming data, using a state-of-the-art DAG scheduler, a query optimizer, and a physical execution engine.



Logistic regression in Hadoop and Spark

10 – 100 times faster than Hadoop/MapReduce

AND ITS FAULT TOLERANT.....

1) Each Spark unit (RDD) remembers the lineage of the operation that was used on input dataset to create it.

2) If due to a worker node failure any partition of an RDD is lost, then that partition can be re-computed using the lineage of operations.

3) To achieve fault tolerance for all the generated RDDs, the achieved data replicates among multiple Spark executors in worker nodes in the cluster.

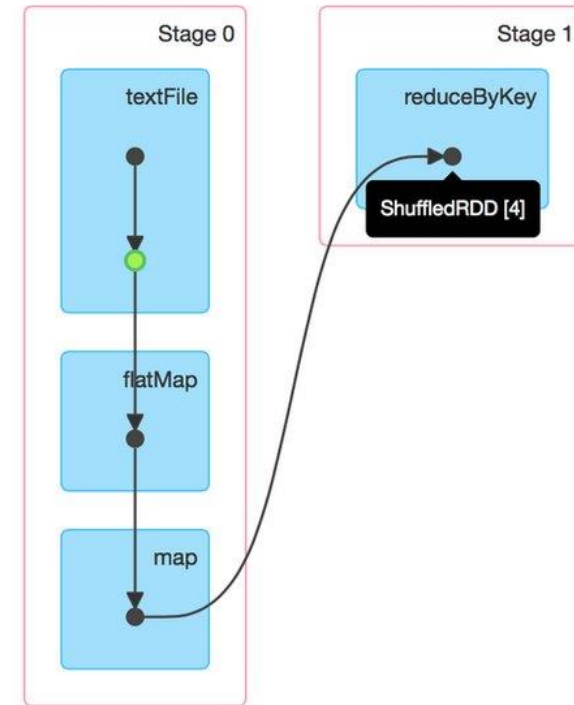
Details for Job 0

Status: SUCCEEDED

Completed Stages: 2

► Event Timeline

▼ DAG Visualization



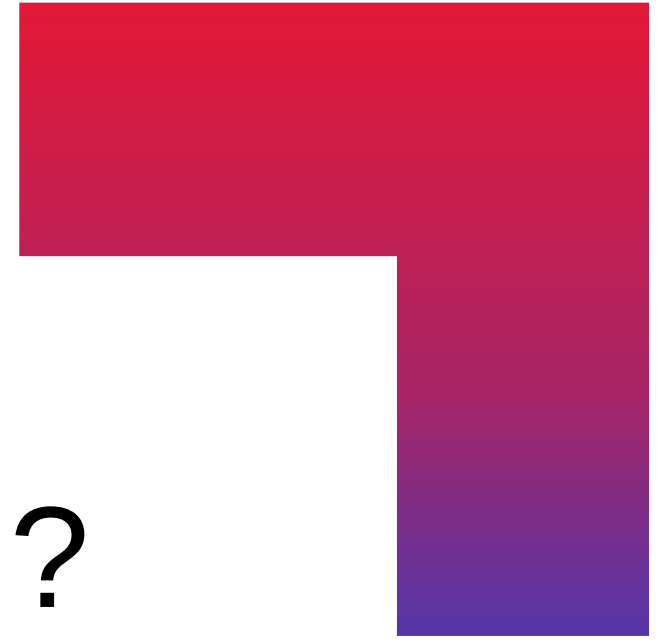
TO RESUME

- One tool where you can scale easily big data workloads quickly
- Distributed calculation = Speed = Time and Money saved
- Allows Python, SQL, R, Java, Scala commands to be used interactively
- Open Source, with a big user community
- Fault Tolerant and Reliable

CHAPTER OBJECTIVE REVIEW

- Understand what is Spark
- Understand why do we use Spark
- Understand key terms in Spark

Why Use Databricks ?

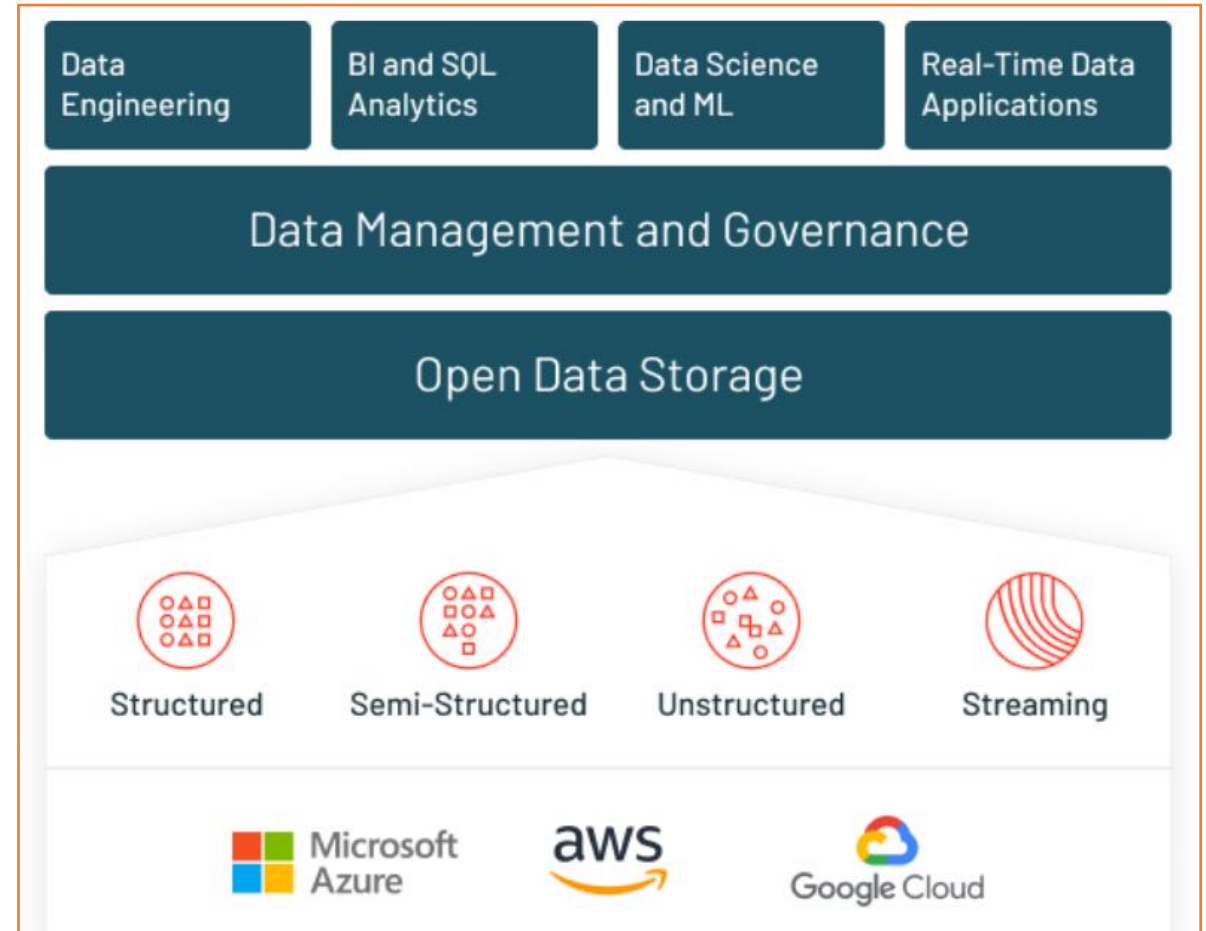


OBJECTIVES

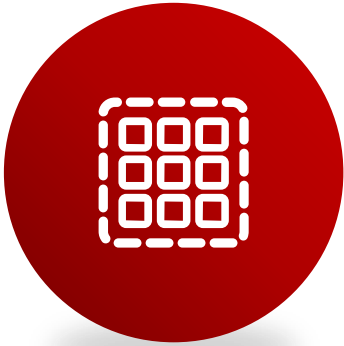
- Understand that Databricks is a Platform to run Spark in the cloud.
- Understand how Databricks responds to current problems in the world of data
- Understand advantages of Databricks

DATABRICKS IS AN UNIFIED ANALYTICS PLATFORM FOR RUNNING SPARK WORKLOADS

- Provisions Spark Clusters for Data Engineering/Machine Learning and SQL BI.
- Allows for Data management and governance using ACID Transactions in Delta
- Allows Data Storage in the platform for all types of data using Hive.
- Multi cloud (integrates well with Azure)



REQUIREMENTS OF A PLATFORM AS A SERVICE



One Platform
Many Workloads



Secure &
Governed Access
to All Data

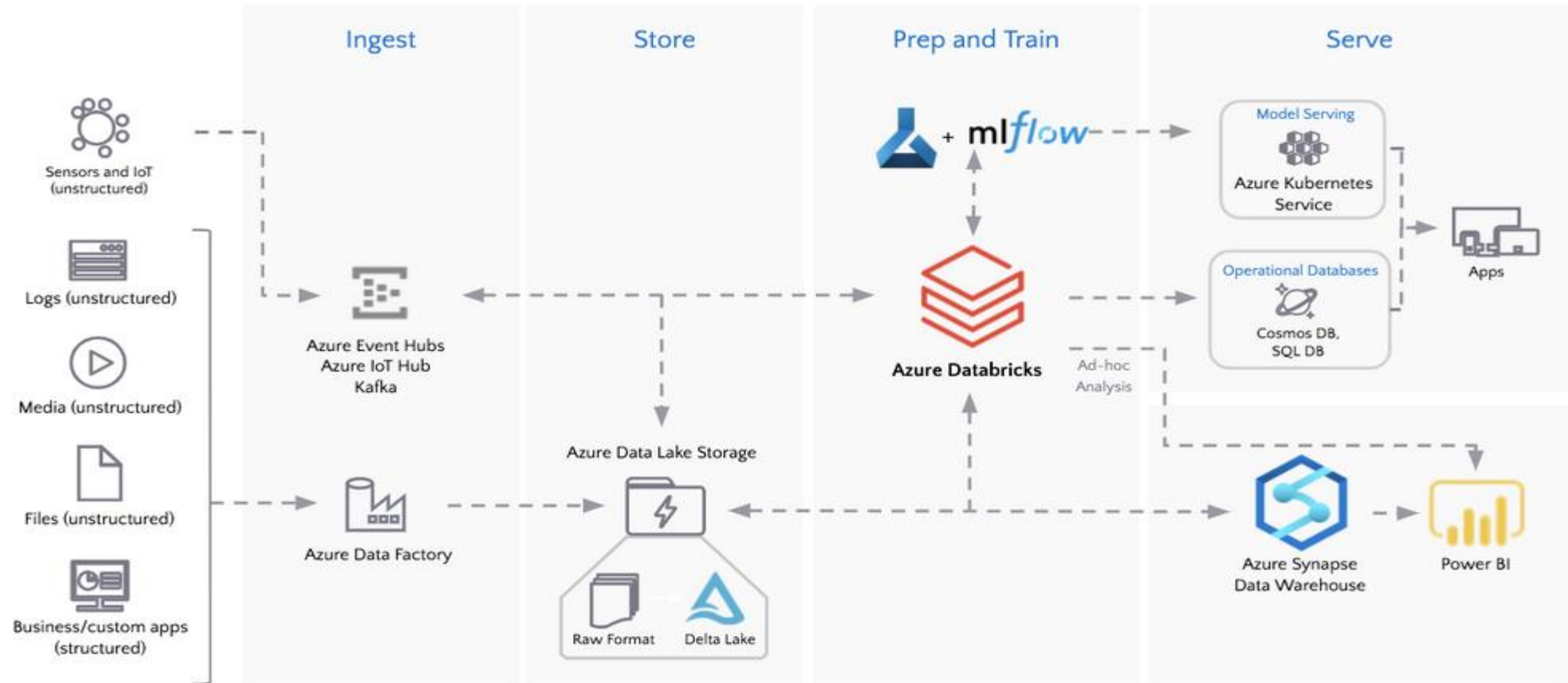


Near-zero
Maintenance,
as a Service



Unlimited
Performance
and ability to scale

MODERN DATA ARCHITECTURE WITH AZURE DATABRICKS



Source : <https://techcommunity.microsoft.com/t5/analytics-on-azure-blog/how-to-reduce-infrastructure-costs-by-up-to-80-with-azure/ba-p/1820280>

DATABRICKS IS CLOUD AGNOSTIC

Available on customer's cloud & region of choice

● AWS ● Azure ● Google ● Alibaba

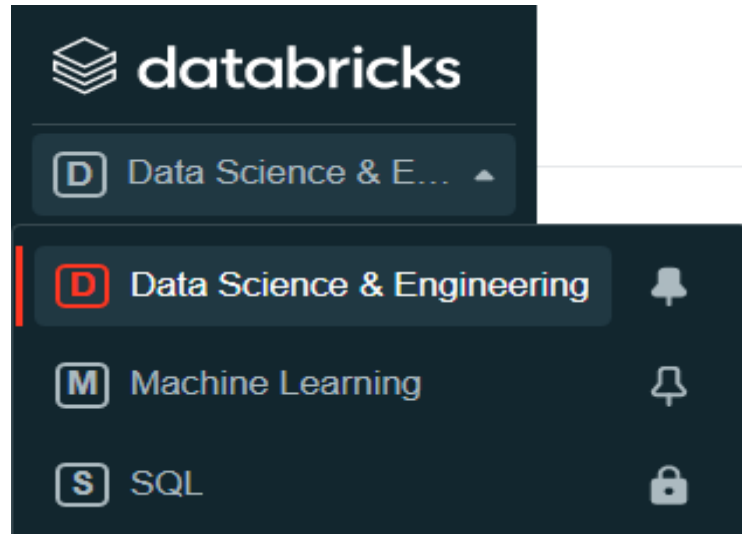
Generally Available

- US West (Oregon)
- US East (N. Virginia)
- Europe (France)
- Europe (Ireland)
- Asia Pacific (Sydney)
- Asia Pacific (Singapore)
- Canada Central (Montreal)
- US East (Ohio)
- Japan (Tokyo)
- AWS India (Mumbai)
- Seoul

© 2023 CGI inc.

ENABLING COLLABORATION

- Provides an environment that allows for collaboration between Data Engineers , Data Science and Data Analysts to become more data-driven.



DATABRICKS PRICING

	STANDARD	PREMIUM
	One platform for your data analytics and ML workloads	Data analytics and ML at scale across your business
JOBS COMPUTE Run data engineering pipelines to build data lakes and manage data at scale	\$0.15/ DBU	\$0.30/ DBU
SQL COMPUTE Run sql queries for BI reporting, analytics, and visualization to get timely insights from data lakes.	–	\$0.22/ DBU
ALL-PURPOSE COMPUTE Run interactive data science and machine learning workloads. Also good for data engineering, BI and data analytics.	\$0.40/ DBU	\$0.55/ DBU

COMPARING SPARK AND DATABRICKS

Comparing Apache Spark™ and Databricks



Apache Spark capabilities provide speed, ease of use and breadth of use benefits and include APIs supporting a range of use cases:

- Data integration and ETL
- Interactive analytics
- Machine learning and advanced analytics
- Real-time data processing



Databricks builds on top of Spark and adds:

- Highly reliable and performant data pipelines
- Productive data science at scale



Azure Data Lake Storage Gen2

A “no-compromises” Data Lake: secure, performant, massively-scalable Data Lake storage that brings the cost and scale profile of object storage together with the performance and analytics feature set of data lake storage



SECURE

- ✓ Support for fine-grained ACLs, protecting data at the file and folder level
- ✓ Multi-layered protection via at-rest Storage Service encryption and Azure Active Directory integration



MANAGEABLE

- ✓ Automated Lifecycle Policy Management
- ✓ Object Level tiering



FAST

- ✓ Atomic directory operations means jobs complete faster



SCALABLE

- ✓ No limits on data store size
- ✓ Global footprint (50 regions)



COST EFFECTIVE

- ✓ Object store pricing levels
- ✓ File system operations minimize transactions required for job completion

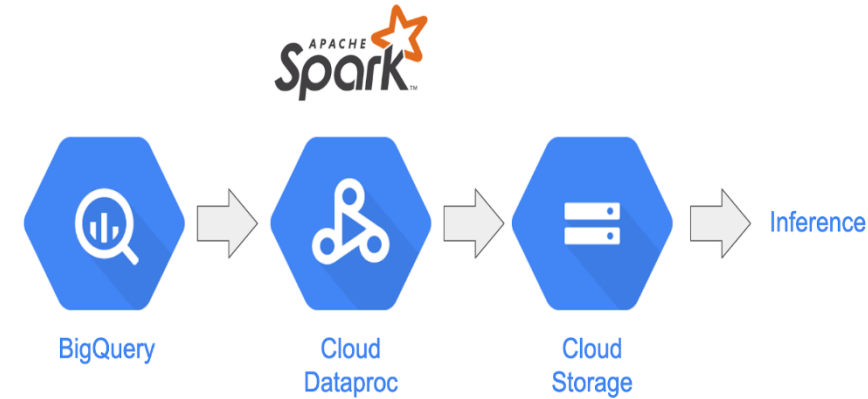
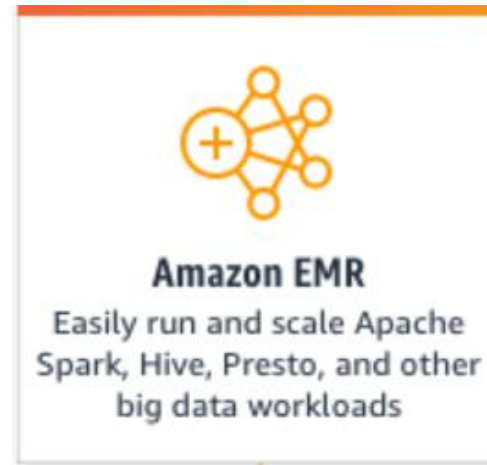


INTEGRATION READY

- ✓ Optimized for Spark and Hadoop Analytic Engines
- ✓ Tightly integrated with Azure end to end analytics solutions

ALTERNATIVES TO DATABRICKS FOR RUNNING SPARK IN THE CLOUD

- Amazon and Google both have their own offering of running a platform with Spark-as-a-Service.



- In Azure, Apache Spark powers both Synapse and Databricks.



CHAPTER OBJECTIVE REVIEW

- Understand that Databricks is a Platform to run Spark in the cloud.
- Understand how Databricks responds to current problems in the world of data
- Understand advantages of Databricks

Databricks Interface



CGI

Try Databricks for free

An open and unified data analytics platform for data engineering, data science, machine learning, and analytics. From the original creators of Apache Spark™, Delta lake, MLflow, and Koalas.



Databricks trial:

- Collaborative environment for data teams to build solutions together.
- Interactive notebooks to use Apache Spark™, SQL, Python, Scala, Delta Lake, MLflow, TensorFlow, Keras, Scikit-learn and more.
- Available as a 14-day full trial in your own cloud, or as a lightweight trial hosted by Databricks.

Used by:



Please tell us about yourself

First Name: *

Last Name: *

Company *

Company Email *

Title *

Phone Number

Country: *

Choose a cloud provider

 Amazon Web Services

 Microsoft Azure

 Google Cloud Platform

Get started

By clicking "Get started", you agree to the [Privacy Policy](#) and [Terms of Service](#)

Don't have a cloud account?

Community Edition is a limited Databricks environment for personal use and training.

[Get started with Community Edition](#)

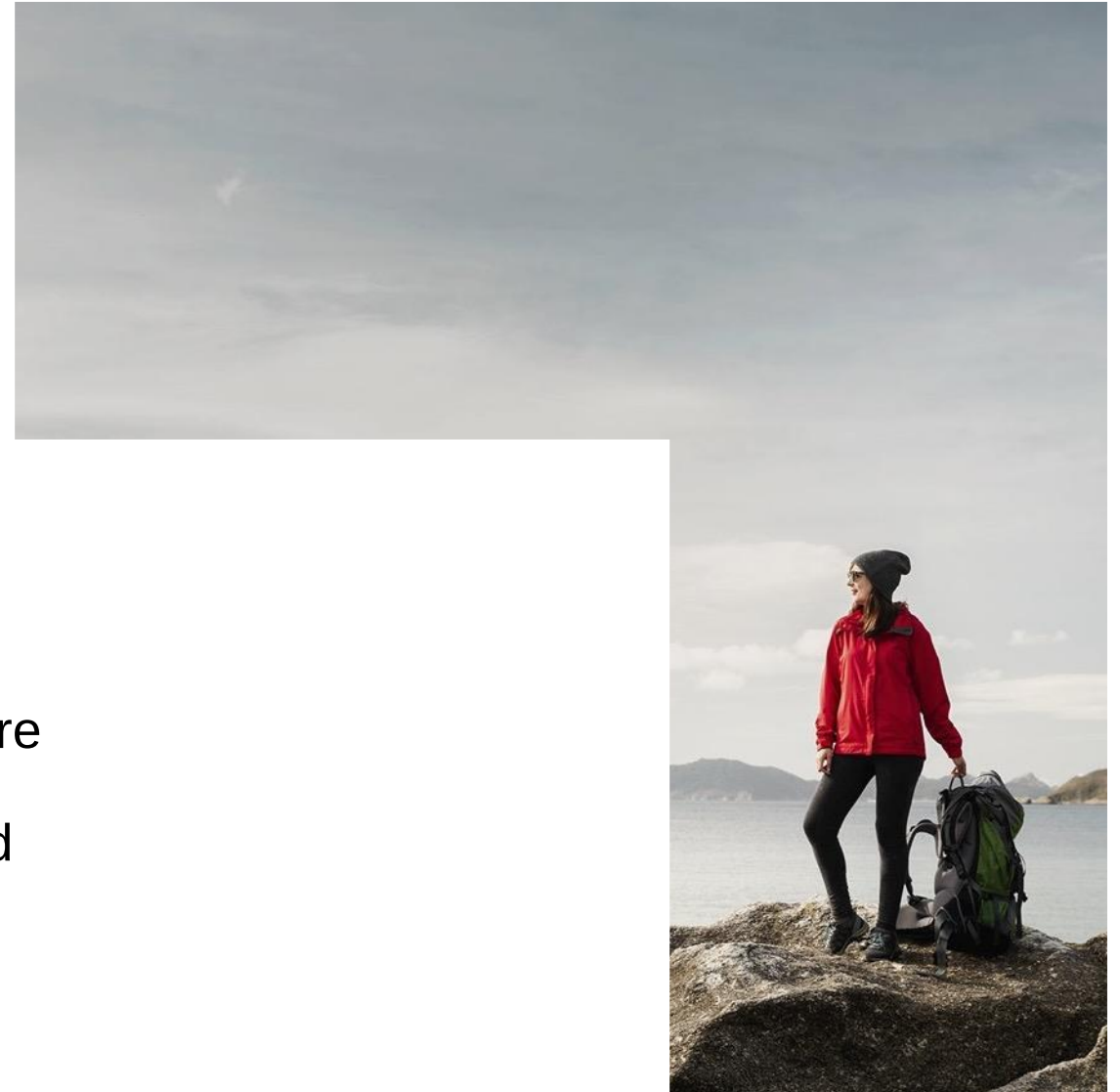
By clicking "Get started with Community Edition", you agree to the [Privacy Policy](#) and [Community Edition Terms of Service](#)

Demo Time

We will access the Databricks resource on Azure

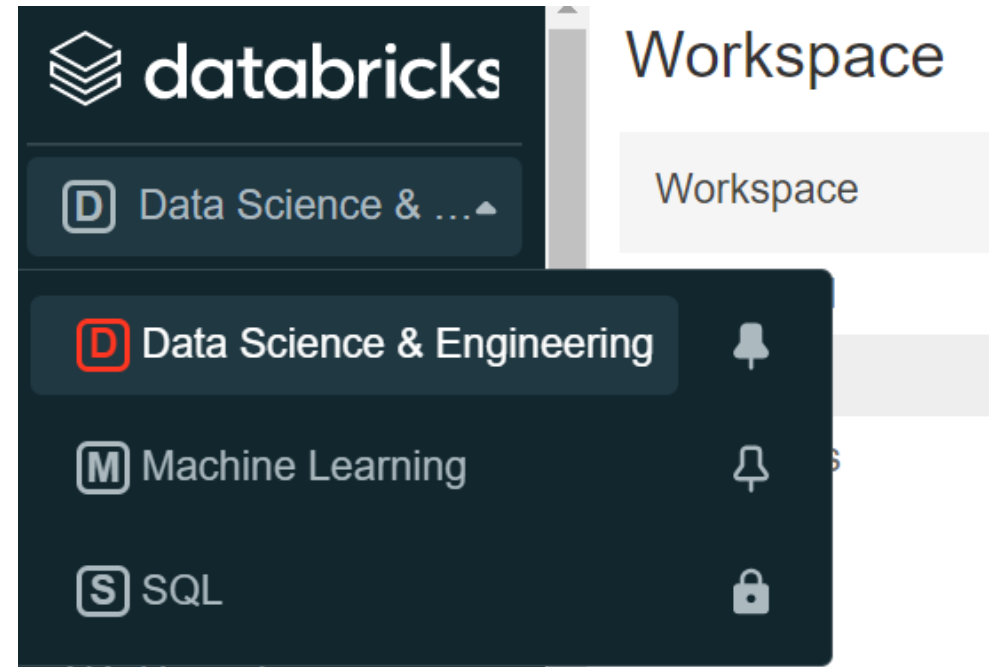
Import the Spark Programming Exercises found
on the Teams channel

And take a tour of the interface

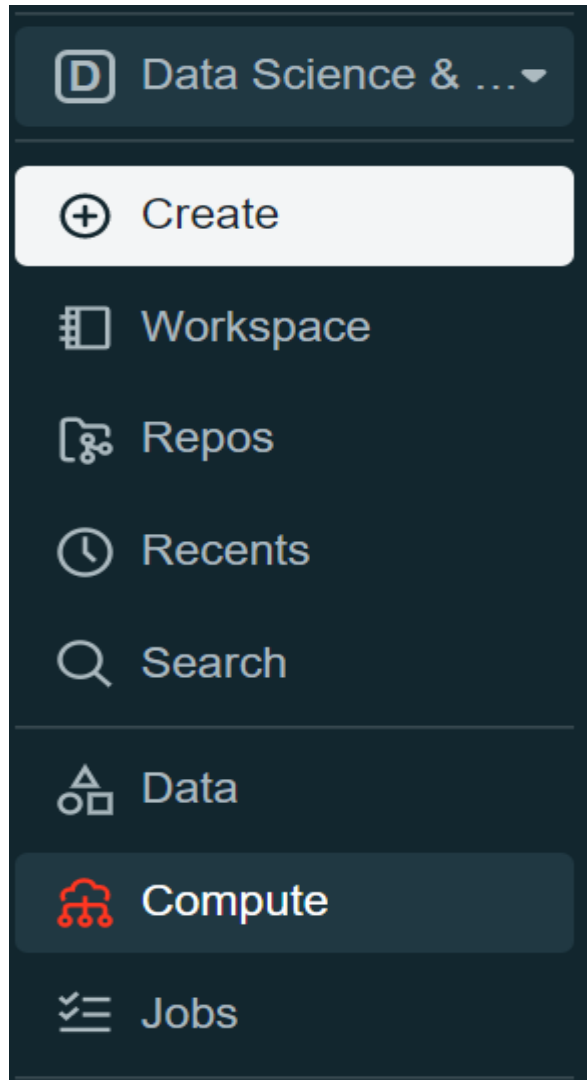


SEPARATE INTERFACES FOR ROLES

Just above the Databricks workspace, you have a one interface for Data Scientists & Engineering, a different one for Machine learning, and a third one focusing on SQL for primarily Data Analysts.



CREATE CLUSTER



Create Cluster

New Cluster Cancel Create Cluster

2-8 Workers: 28.0-112.0 GB Memory, 8-32 Cores, 1.5-6 DBU
1 Driver: 14.0 GB Memory, 4 Cores, 0.75 DBU Cost \$0.40 per DBU

Cluster Name

clf-for-aze-cluster1

Cluster Mode ?

Standard

Databricks Runtime Version ? [Learn more](#)

Runtime: 5.2 (Scala 2.11, Spark 2.4.0)

Python Version ?

3

New The default Python version for clusters was changed from major version 2 to 3.

Autopilot Options

☒ Enable autoscaling ?
☒ Terminate after minutes of inactivity ?

Worker Type

Standard_DS3_v2 14.0 GB Memory, 4 Cores, 0.75 DBU

Min Workers

2

Max Workers

8

Driver Type

Same as worker 14.0 GB Memory, 4 Cores, 0.75 DBU

▶ Advanced Options

CHAPTER OBJECTIVE REVIEW

- Take a tour of the most common parts of the Databricks interface
- Be able to start and import your Databricks notebooks
- Create your first cluster

Time for a break



CGI

Reading and Writing Files



OBJECTIVES

- Understand the different formats to read and write data with Spark
- Understand how to change different languages within a Databricks notebook

QUICK REVIEW: WHAT IS JSON

JSON stands for **J**ava**S**cript **O**bject **N**otation

JSON is a lightweight format for storing and transporting data

JSON is often used when data is sent from a server to a web page

JSON is "self-describing" and easy to understand

Curly braces hold objects

Square brackets hold arrays

```
{  
  "employees": [  
    {"firstName": "John", "lastName": "Doe"},  
    {"firstName": "Anna", "lastName": "Smith"},  
    {"firstName": "Peter", "lastName": "Jones"}  
  ]  
}
```

QUICK REVIEW: WHAT IS PARQUET

Apache Parquet is a binary, efficient columnar data format. It uses various techniques to store data in an efficient way like row groups, compression for pages in column chunks or dictionary encoding for columns.

	day	location	product	sale
row 1	2017-01-01	l1	p1	300
row 2	2017-01-01	l1	p2	40
row 3	2017-01-01	l2	p1	44
row 4	2017-02-01	l1	p1	200

Traditional Memory Buffer	
row 1	2017-01-01
	l1
	p1
	300
row 2	2017-01-01
	l1
	p2
	40
row 3	2017-01-01
	l2
	p1
	44

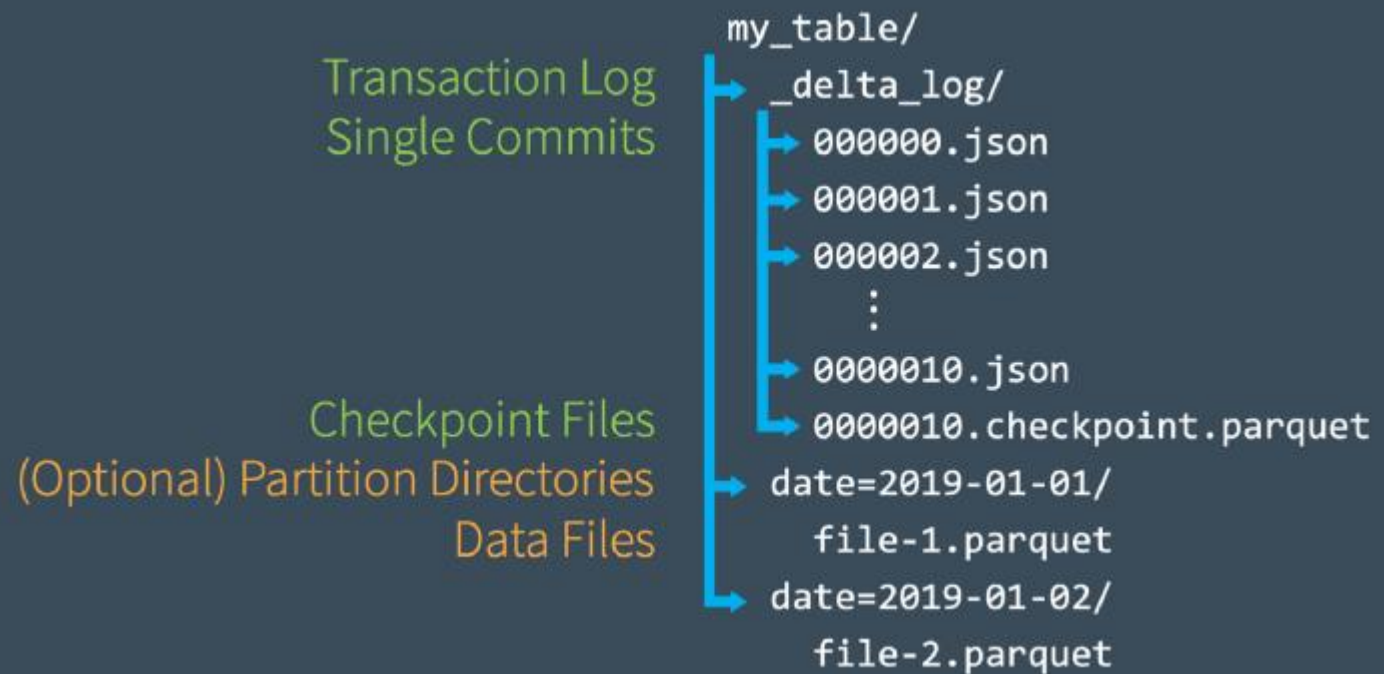
Columnar Storage	
day	2017-01-01
	2017-01-01
	2017-01-01
	2017-01-02
location	l1
	l1
	l2
	l1
product	p1
	p2
	p1
	p1

QUICK REVIEW: WHAT IS DELTA

Delta is a form of optimized Parquet, with a Transaction Log to store and record history. This transaction log differs it from traditional parquet files.

Delta format is made of many components:

- Parquet data files organized or not as partitions
- Json files as transaction log
- Checkpoint files



READING DIFFERENT FILE FORMATS

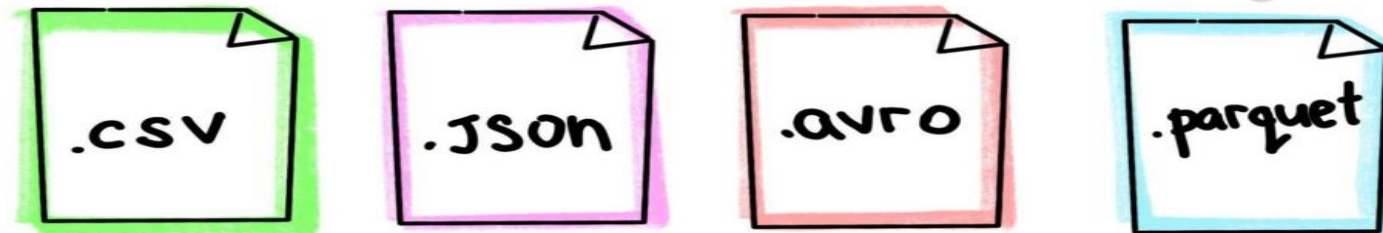
- Apache Spark supports a wide range of data formats, including the popular CSV format and the convenient JSON Web format, as well as Apache Parquet and Apache Avro.

`spark.read.csv(filepath, sep = ; , header = True)`

`spark.read.format('avro').load(filepath)`

`spark.read.json(filepath)`

`spark.read.parquet(filepath)`



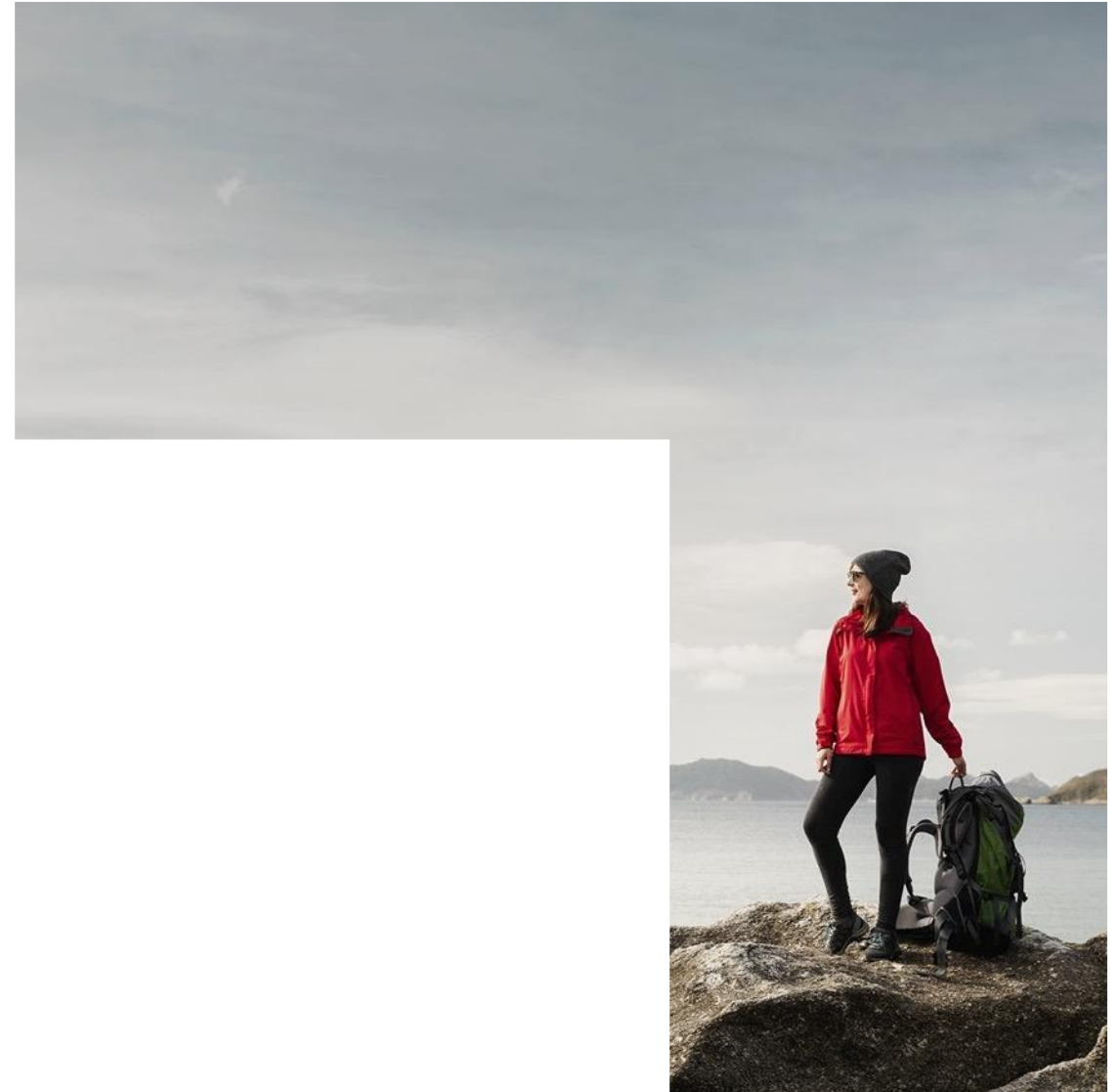
WRITING IN DIFFERENT FILE FORMATS

- Just as we can read data easily, Spark allows you to write data in specific formats.
- We will define a custom function for writing in csv format or json format in this course, but for parquet files, we can simply use the native method (`spark.write.parquet`)
- Another useful way of writing is `saveAsTable("Table_Name")` which allows you to save the data as a table.

Exercise

Let's work on Spark Programming Exercises :

1.1 Reading & Writing Files



CHAPTER OBJECTIVES REVIEW

- Understand the different formats to read and write data with Spark
- Understand how to change different languages within a Databricks notebook



Go Deeper

- Databricks Documentation
<https://docs.databricks.com/>

For Pyspark and SQL help:

<https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql.html>



Rappels



3V OF DATA

Volume

The amount of data being generated



Velocity

The speed at which data is generated



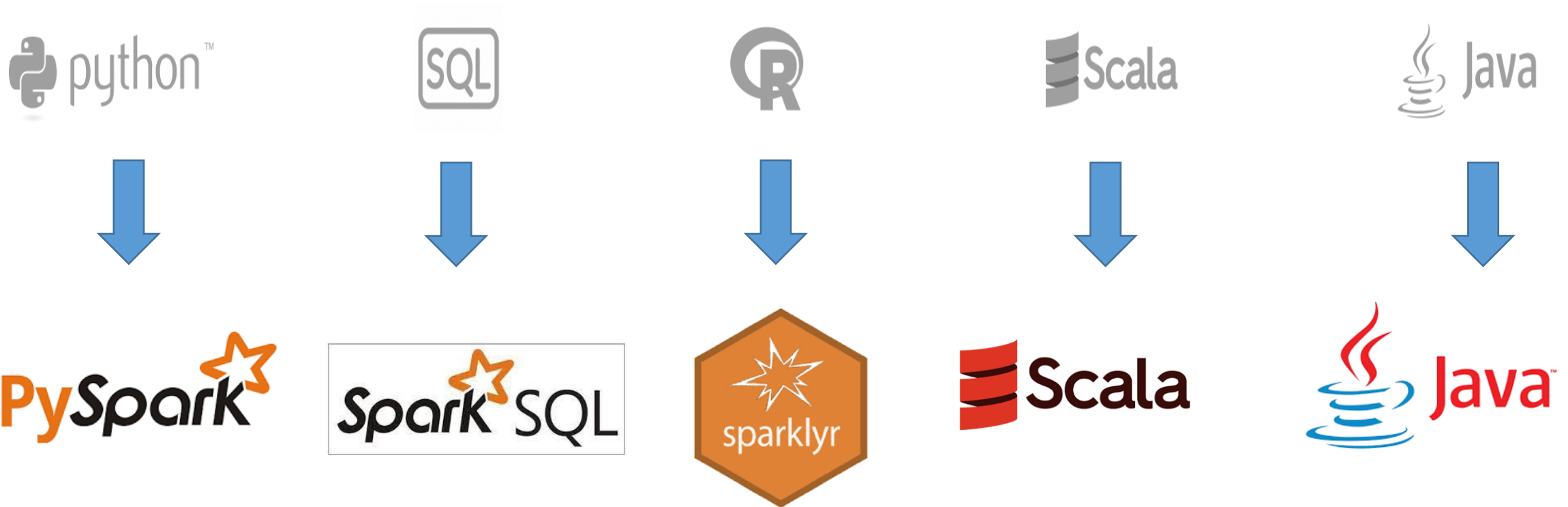
Variety

The different types of data generated



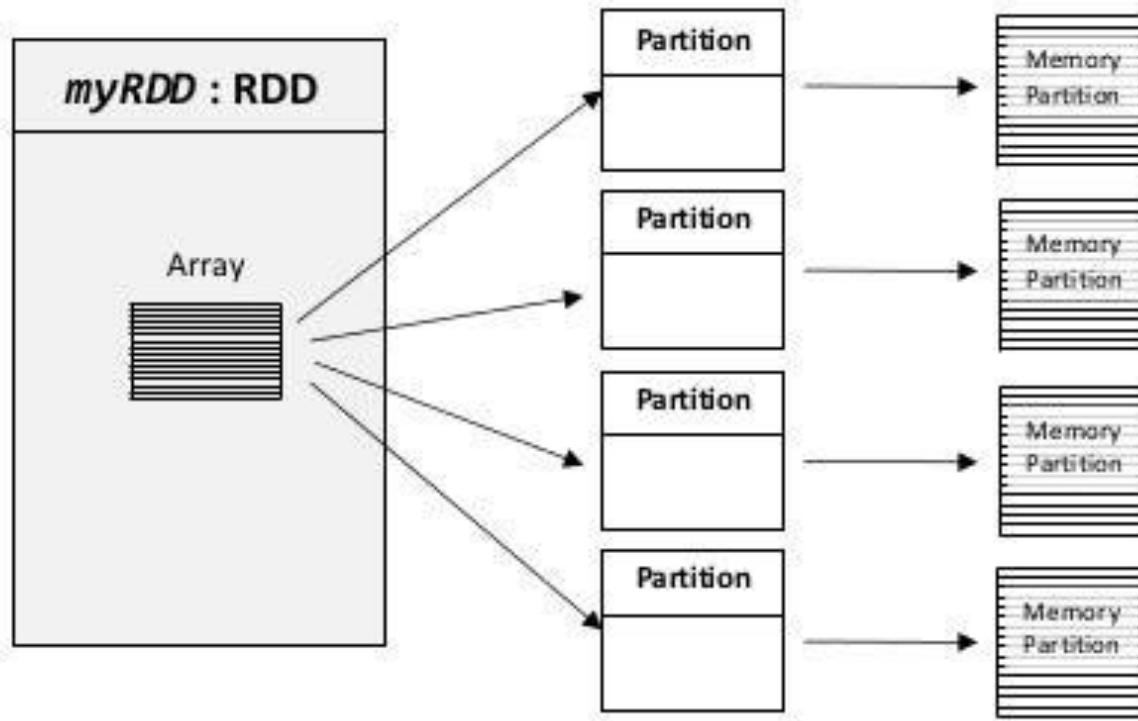
CODE IN THE LANGUAGE OF YOUR CHOICE

Spark allows to distribute calculation on these languages to make them more efficient at treating data



SPARK HAS RESILIENT DISTRIBUTED DATASETS

What is an RDD?



Some RDD Characteristics

- Hold references to Partition objects
- Each Partition object references a subset of your data
- Partitions are assigned to nodes on your cluster
- Each partition/split will be in RAM (by default)

Source:

<https://www.slideshare.net/sparkInstructor/apache-spark-rdd-101>

REVIEW : KEY TERMS IN SPARK



CLUSTER

A **Spark cluster** is a collection of machines or nodes in the public cloud or on-premises in a private data center on which Spark is installed.



RDD

RDD (Resilient Distributed Dataset) is the fundamental data structure of Apache Spark which are an immutable collection of objects which computes on the different node of the cluster.



PERSIST

Persist or cache means to store data in memory where Spark does its computation

REVIEW : KEY TERMS IN SPARK



SPARK DRIVER

The **driver** program distributes Spark tasks to each worker's Executor. The driver also receives computed results from each Executor's tasks.



SPARK WORKER

Each **Worker** node consists of one or more Executor(s) who are responsible for running the Task.

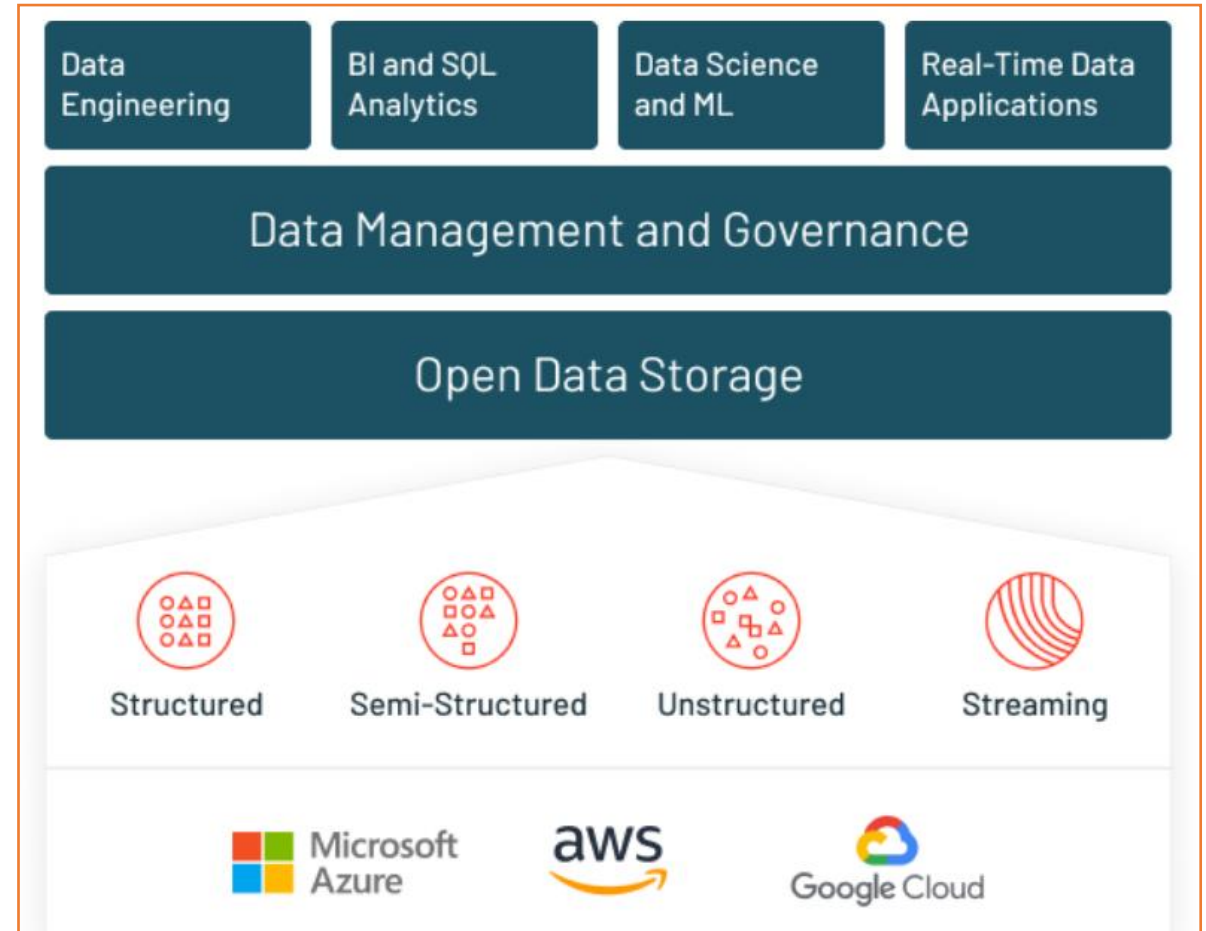


SPARK EXECUTOR

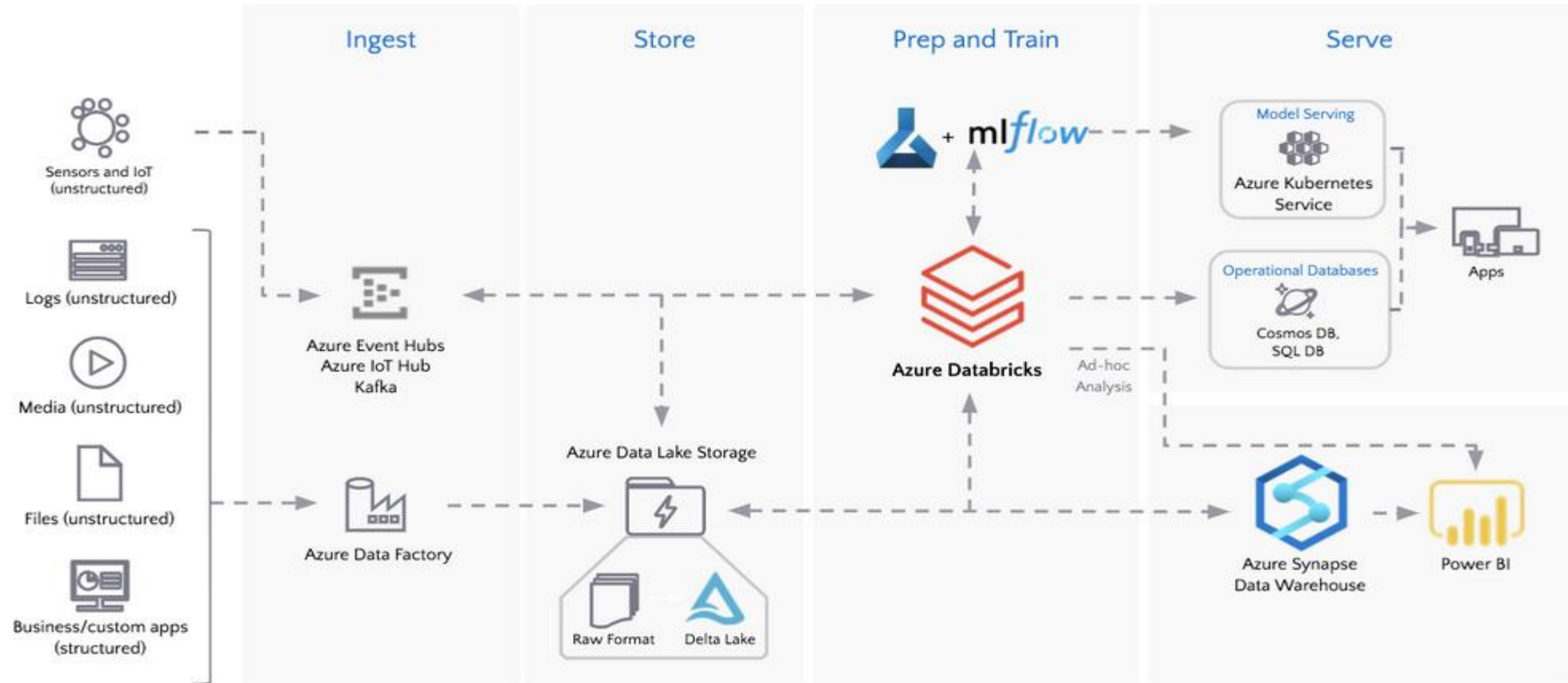
Executor resides in the Worker node. Executors are launched at the start of a Spark Application in coordination with the Driver. They do the actual jobs

DATABRICKS IS AN UNIFIED ANALYTICS PLATFORM FOR RUNNING SPARK WORKLOADS

- Provisions Spark Clusters for Data Engineering/Machine Learning and SQL BI.
- Allows for Data management and governance using ACID Transactions in Delta
- Allows Data Storage in the platform for all types of data using Hive.
- Multi cloud (integrates well with Azure)



MODERN DATA ARCHITECTURE WITH AZURE DATABRICKS



Source : <https://techcommunity.microsoft.com/t5/analytics-on-azure-blog/how-to-reduce-infrastructure-costs-by-up-to-80-with-azure/ba-p/1820280>

Transform data with Pyspark



OBJECTIVES

- Understand what is PySpark and what is a DataFrame
- Learn and review the basic syntax of Pyspark
- Practice basic transformations , selection and joins using pyspark.
- Contrast the syntax for simple transformations between Pyspark and spark SQL

WHAT IS PYSPARK ?

PySpark is the Python API for Apache Spark, an open source, distributed computing framework and set of libraries for real-time, large-scale data processing.

It is an object-oriented language that is dynamically typed.

WHAT IS PYSPARK DATAFRAME ?

The key data type used in PySpark is the Spark dataframe. This object can be thought of as a table distributed across a cluster, and has functionality that is similar to dataframes in R and Pandas.

If you want to do distributed computation using PySpark, then you'll need to perform operations on Spark dataframes and not other Python data types.

One of the key differences between Pandas and Spark dataframes is eager versus lazy execution. In PySpark, operations are delayed until a result is actually requested in the pipeline.

THE IMPORTS YOU NEED TO KNOW

```
from pyspark.sql.session import SparkSession
```

```
from pyspark.sql.types import *
```

```
from pyspark.sql.functions import *
```

```
from pyspark.sql.window import Window
```

ACTIONS AND TRANSFORMATIONS

Transformations <i>(lazy)</i>	Actions
select	show
distinct	count
groupBy	collect
sum	save
orderBy	display
filter	
limit	

LAZY EVALUATION

- *Lazy Evaluation* means triggering processing only when a *Spark action* is run and not a *Spark transformation*.

WAYS TO DEAL WITH COLUMNS

```
from pyspark.sql.functions import col #replace col by * if you want all functions
```

```
df = df.filter(col("column1") < value)
```

```
import pyspark.sql.functions as F
```

```
df = df.filter(F.col("column1") < value)
```

Warning : col keyword is always needed for filters

PYSPARK SELECT

```
df.select("column1")
```

The select column is a very important functionality on a PYSPARK data frame which gives us the privilege of selecting the columns of our need in a PySpark making the data more defined and usable.

With the select column, we can have the option of selecting the column we need and leaving the rest of the columns that are not needed in a PySpark data frame.

PYSPARK CREATE A NEW COLUMN

```
df = df.withColumn("NewColumn", *condition)
```

To add a new column, we must use the With Column syntax.

Even if you want to do just change the type of same column, you must use the WithColumn syntax on the same column. This is because Spark Dataframes are immutable.

PYSPARK WHERE

```
df = df.where(df.column == "x")
```



```
df = df.filter(df.column == "x")
```

The filter condition is similar to **where** condition in SQL where it filters data based on the condition provided. Both the where and filter conditions do the same thing in pyspark.

PYSPARK RENAME A COLUMN

```
df = df.withColumnRenamed("CurrentCol", "NewCol")
```

The `.withColumnRenamed` function is one simple way for us to rename a column. Remember, the current column is first, then the column you want to rename.

PYSPARK LIT

```
df = df.withColumn("LiteralValue", lit("I love  
Spark"))
```

The lit function is used to assign a literal value as a text to a column . For example, it could be used to assign True/False values to a column.

PYSPARK WHEN

```
df.withColumn("condition1", when(col("x")== "111",  
lit("Found")).otherwise(lit("Not Found")))
```

The when function is used instead of if and else or case when conditions .

Most frequent use case is to create a new column B that creates a literal value if column A has a specific value. We can chain when conditions by adding otherwise at the end.

PYSPARK ORDER BY

```
df.orderBy(desc("col_Name")).show()
```

PySpark Orderby is a spark sorting function used to sort the data frame / RDD in a PySpark Framework.

The orderBy clause is used to return the row in a sorted Manner. It guarantees the total order of the output. The order by function can be used with one column as well as more than one column can be used in OrderBy. It takes two parameters : Asc for ascending and Desc for Descending order.

PYSPARK UNION

```
df1.unionByName(df2)
```

The union is the same as concatenation of two dataframes : it will guard all the rows from df1 and all the rows from df2.

PYSPARK DISTINCT

```
df1.select(columns).distinct()
```

Remember, the distinct will be applied on the dataframe columns that you select

PYSPARK DROP DUPLICATES

```
df1.select(columns).dropDuplicates()
```

Similar to the distinct function , will keep only unique values .

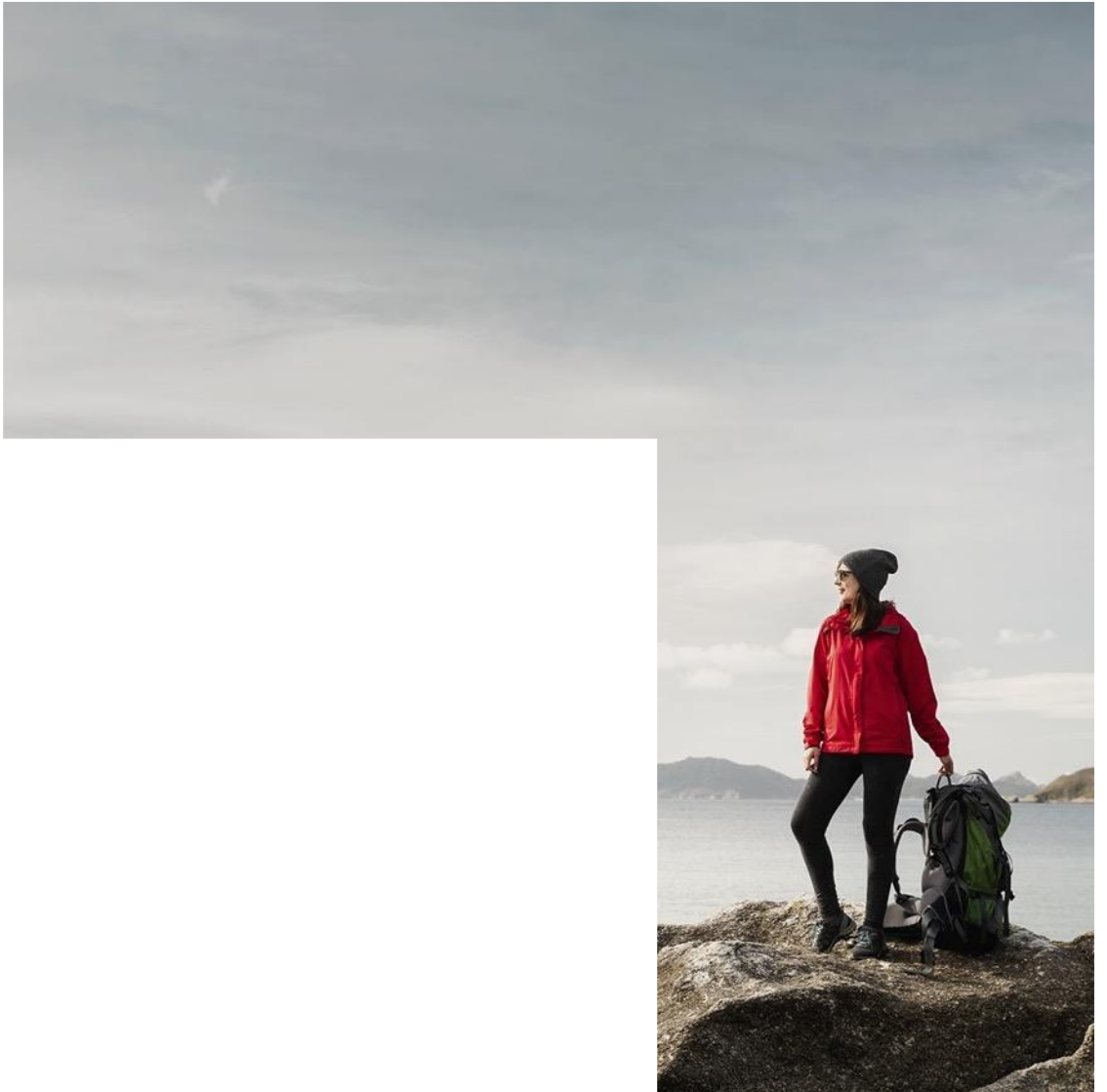
CHEAT SHEET PYSPARK

Go to the following link;

https://s3.amazonaws.com/assets.datacamp.com/blog_assets/PySpark_SQL_Cheat_Sheet_Python.pdf

Exercise

2.0 Pyspark



CHAPTER OBJECTIVE REVIEW

- Understand what is PySpark and what is a DataFrame
- Learn and review the basic syntax of Pyspark
- Practice basic transformations , selection and joins using pyspark.
- Contrast the syntax for simple transformations between Pyspark and Spark SQL

Go to Kahoot.it



PYSPARK GROUPBY

```
df.groupBy ("Column")
```

PySpark GroupBy Count is a function in PySpark that allows to group rows together based on some columnar value and count the number of rows associated after grouping in spark application.

The group By Count function is used to count the grouped Data, which are grouped based on some conditions and the final count of aggregated data is shown as the result. In simple words, if we try to understand what exactly groupBy count does in PySpark is simply grouping the rows in a Spark Data Frame having some values and count the values generated.

PYSPARK FUNC & AGG

A best practice is to aggregate a dataframe after a groupBy using the **.agg** function. This helps us to know that we are aggregating a dataframe.

Once we start aggregating, we must use a library of Pyspark functions to help us to make useful notions of our aggregations.

```
agg_df = df.groupBy("Ville").agg(sum("salaire"),  
                                  countDistinct("Maison_Id"),  
                                  mean("Age"))
```

- For example:
 - F.min,
 - F.max,
 - F.count

PYSPARK GROUPBY & COUNT

```
agg_df = df.groupBy("id").agg(func.count("id"))
```

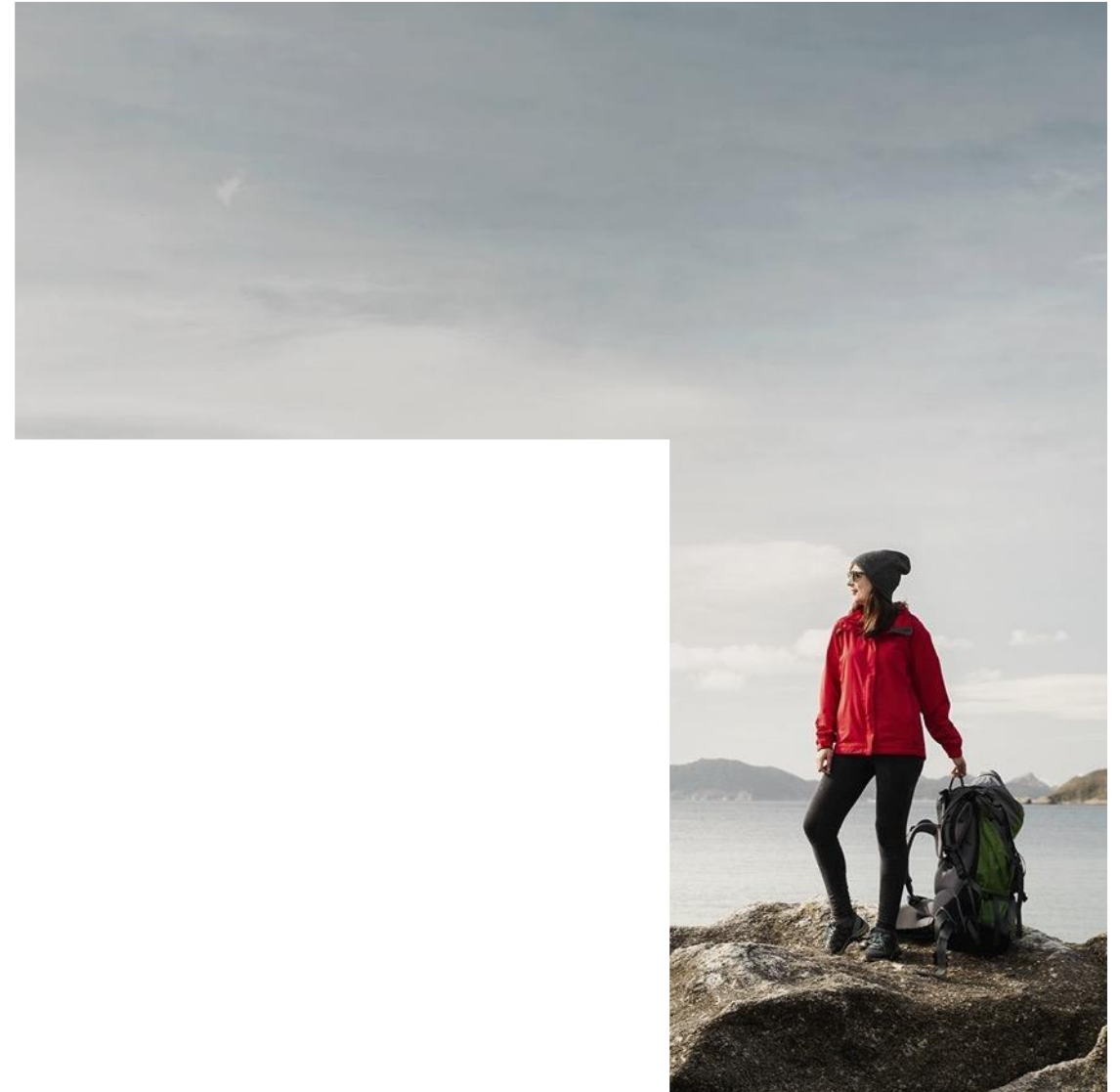
The way to do a group by and count is simply by doing a group by followed by an aggregate `func.count` of the column you want to count on.

Exercises

2.1 Aggregation

2.2 Datetimes

2.3 Additional functions



Go to Kahoot.it



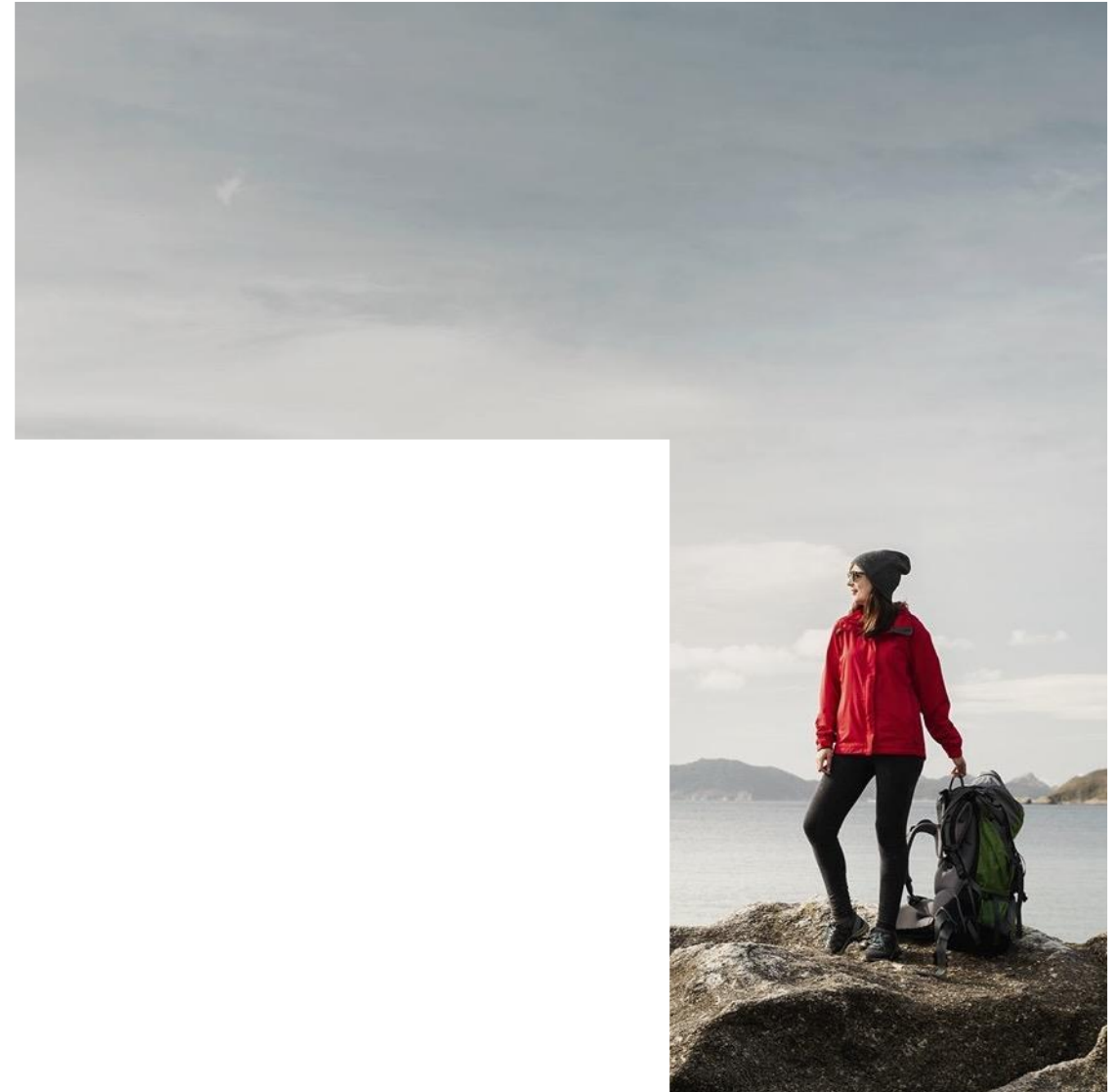
CGI

Exercise Bonus

2.1 Aggregation

2.2 Datetimes

2.3 Additional functions



**End of the day
=
ROTI**



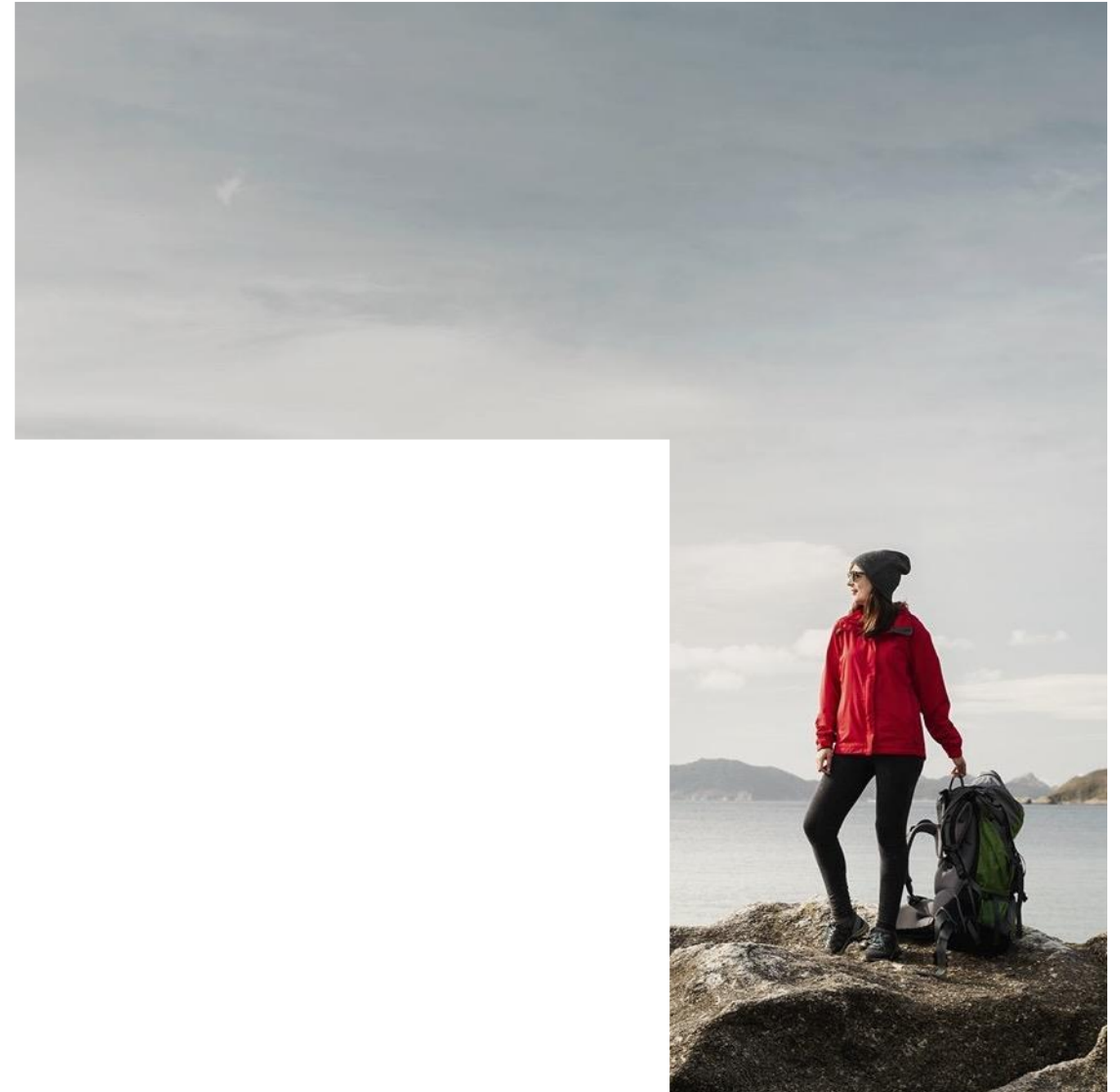
CGI

Exercise Bonus

2.1 Aggregation

2.2 Datetimes

2.3 Additional functions



JOINS

**Joins in Pyspark are easy to read
and easy to write.**

**Here is the basic syntax:
datafram1.join(dataframe2,
on = [columns to join on],
how = "join string ")**

Join String	Equivalent SQL Join
inner	INNER JOIN
outer, full, fullouter, full_outer	FULL OUTER JOIN
left, leftouter, left_outer	LEFT JOIN
right, rightouter, right_outer	RIGHT JOIN
cross	
anti, leftanti, left_anti	
semi, leftsemi, left_semi	

JOINS (SINGLE COMMON COLUMN)

Pyspark

```
Df1.join(DF2, on = ["CAI_CODE"], how = "inner")
```

Spark SQL

```
Select a.* from table1 a  
Inner join table2 b on  
a.CAI_CODE = b.CAI_CODE
```

JOINS (TWO COMMON COLUMN)

Pyspark

```
Df1.join(DF2, on = ["CAI_CODE", "DATE" ], how =  
"inner")
```

Spark SQL

```
Select a.* from table1 a  
Inner join table2 b on  
a.CAI_CODE = b.CAI_CODE  
And a.DATE = b.DATE
```

JOINS (ONE COLUMN, DIFFERENT NAMES)

Pyspark

```
Df1.join(DF2, DF1.ITEM == DF2.CAI_CODE, how  
= "inner")
```

Spark SQL

```
Select a.* from table1 a  
Inner join table2 b on  
a.ITEM = b.CAI_CODE
```

JOINS (TWO COLUMNS, DIFFERENT NAMES)

Pyspark

#first rename the columns

```
DF2 = DF2.withColumnRenamed("DT", "DATE" )  
.withColumnRenamed("ITEM", "CAI_CODE" )
```

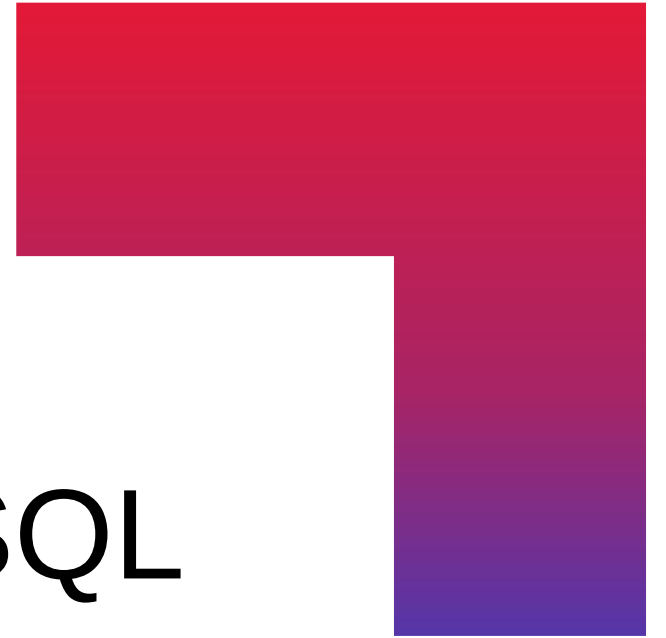
#then join

```
Df1.join(DF2, on = ["CAI_CODE", "DATE" ], how =  
"left")
```

Spark SQL

```
Select a.* from table1 a  
left join table2 b on  
a.ITEM = b.CAI_CODE  
And a.DT = b.DATE
```

Query data with Spark SQL



OBJECTIVES

- Understand how to save a temporary table and run SQL queries on spark dataframes
- Understand the use of `spark.sql()` operation to query data and convert to Dataframe

WHAT IS SPARK SQL



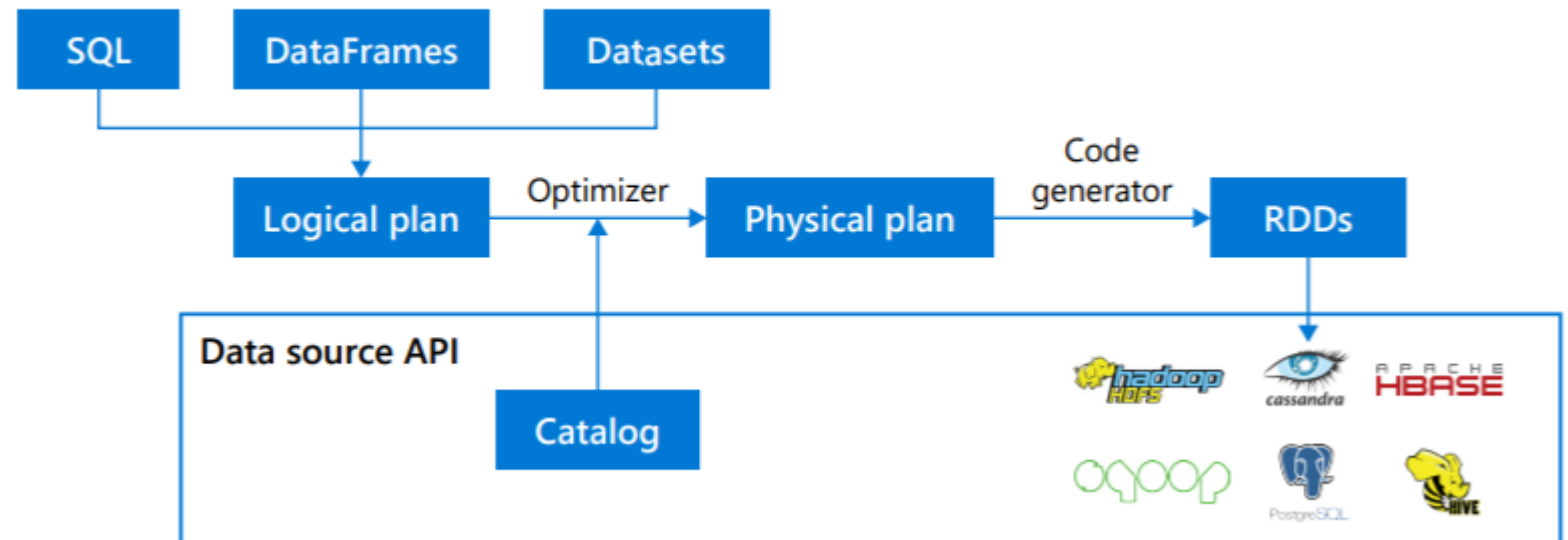
- Spark SQL is one of the modules of Spark for structured data processing. Concretely, Spark SQL will allow developers to run SQL queries against imported data. It allows also a simple conversion from SQL back to Dataframes.

For more info: <https://spark.apache.org/docs/latest/sql-programming-guide.html>

HOW IS SPARK SQL OPTIMIZED ?

- Spark will first create a logical plan of your code written in SQL , and try to optimize it using the help of Spark catalog.
- This will generate several different physical plans, where another optimizer will choose the physical plan that costs the least number of resources.
- Finally, there is a code generator to convert these into RDDs, the fundamental building blocks of Spark.

Spark SQL Architecture



Source:

<https://spark.apache.org/docs/>

HOW TO REGISTER TEMP TABLES

- `df.registerTempTable(« table_name »)` : allows to create a temporary view from a dataframe to do SQL queries on.

For more info:

<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.DataFrame.registerTempTable.html>

SPARK SQL: CONVERT BETWEEN SQL AND PYSPARK

- The magic of Spark SQL is it allows us to convert data between pure SQL into a Spark Dataframe that we can manipulate using PySpark.
- We need to use `Dataframe = spark.sql("""" query """)` and then register a temporary table.

```
df_books = spark.read.parquet(path)
```

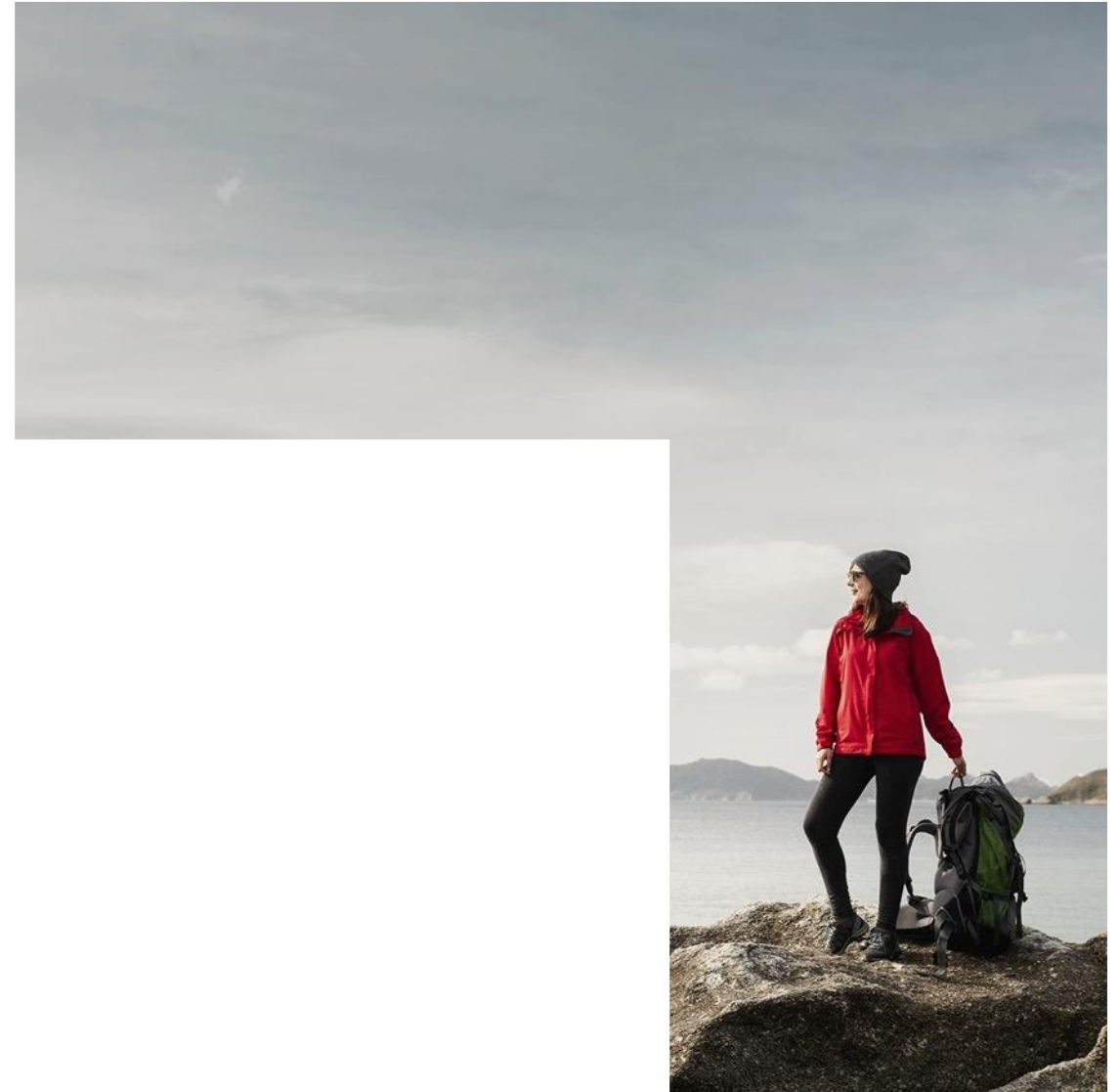
```
df_books.registerTempTable("tbl_books")
```

```
df_aggregated = spark.sql("select product_id,count(star_rating)  
as total_rating,max(star_rating) as best_rating, min(star_rating)  
from tbl_books t1  
group by product_id order by total_rating desc limit 10")
```

Exercise

Let's work on Spark Programming Exercises :

1.2 Query data with Spark SQL



CHAPTER OBJECTIVE REVIEW

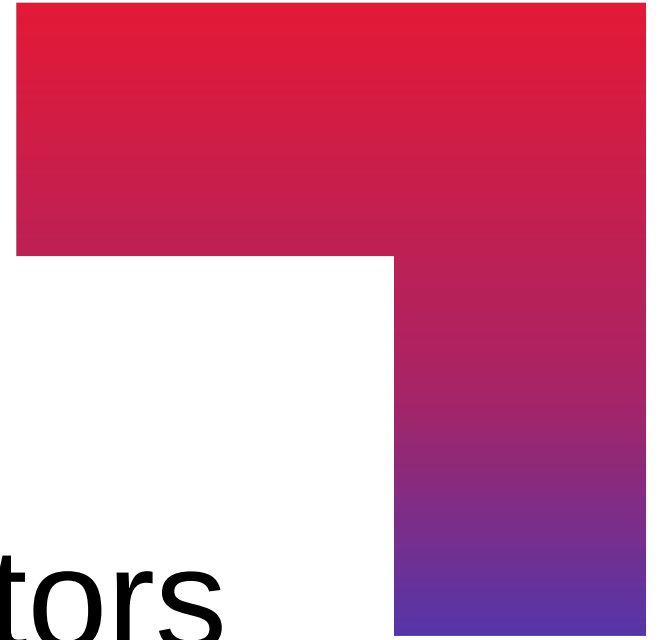
- Understand how to save a temporary table and run SQL queries on spark dataframes
- Understand the use of `spark.sql()` operation to query data and convert to Dataframe

DAY 2 BEGINS



CGI

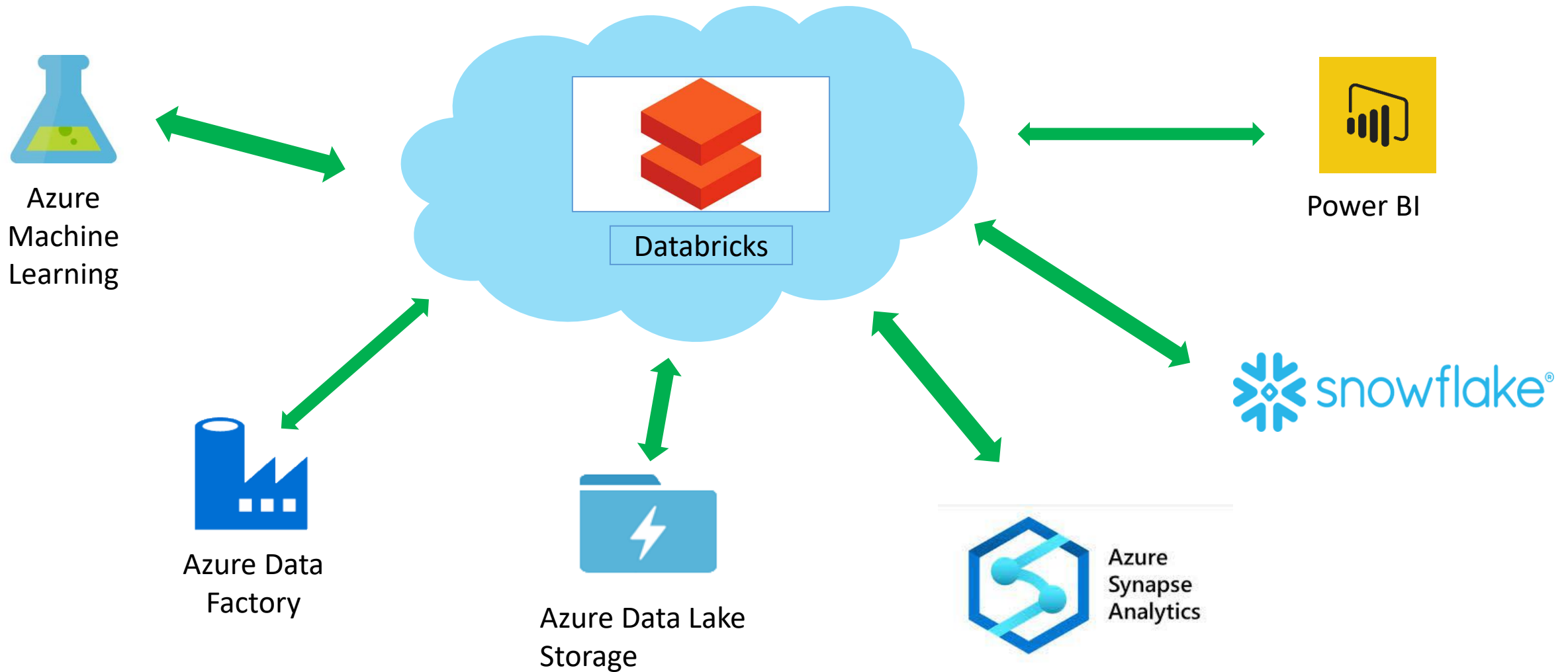
Databricks Connectors



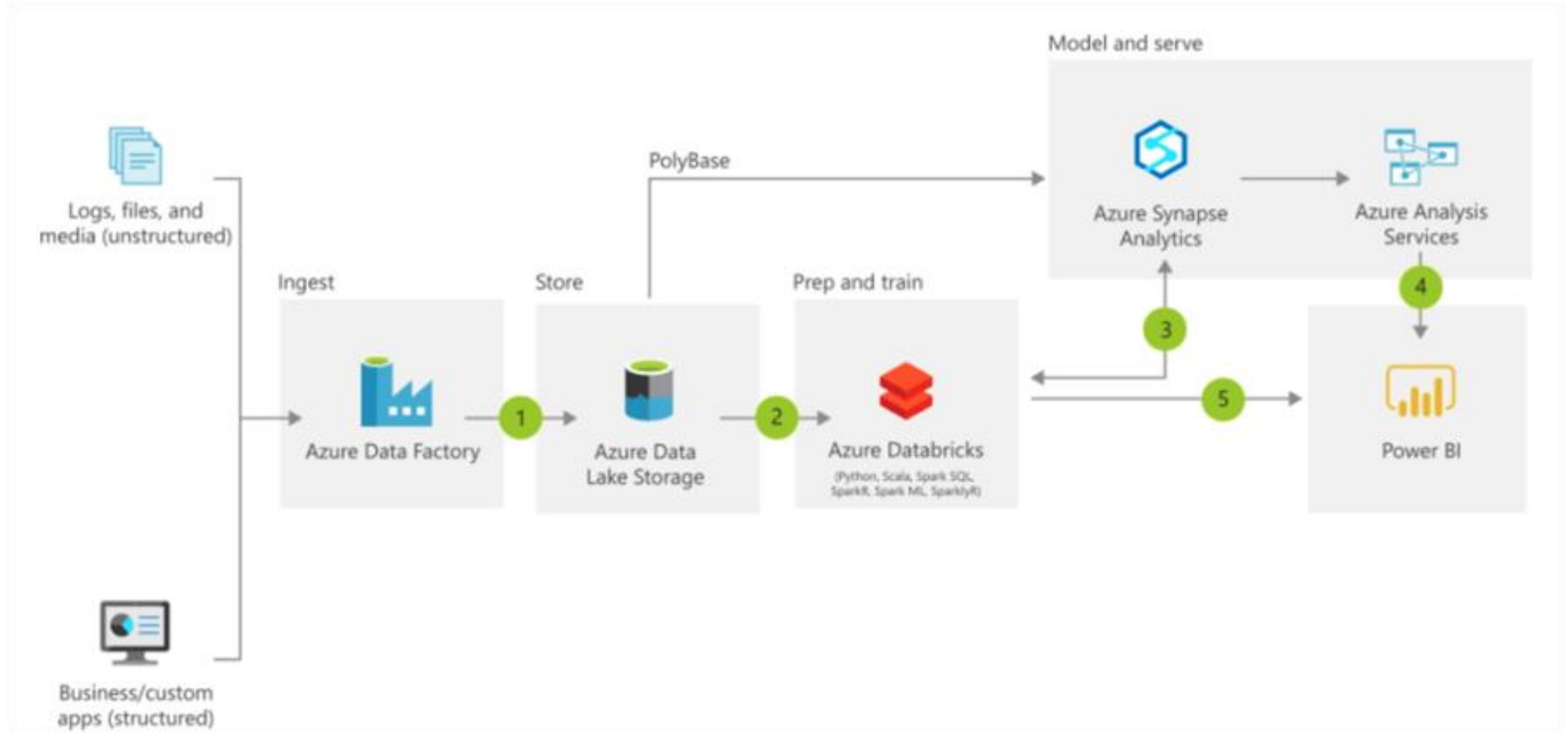
OBJECTIVES

- Understand how Databricks fits in the Azure cloud
- Look at how to connect Databricks with Datafactory and Datalake
- Understand the 3 principal ways to connect Databricks to another Azure component.
- Look at how to connect Databricks to BI Tools like Power BI and Microstrategy

Compatibility matrix between components



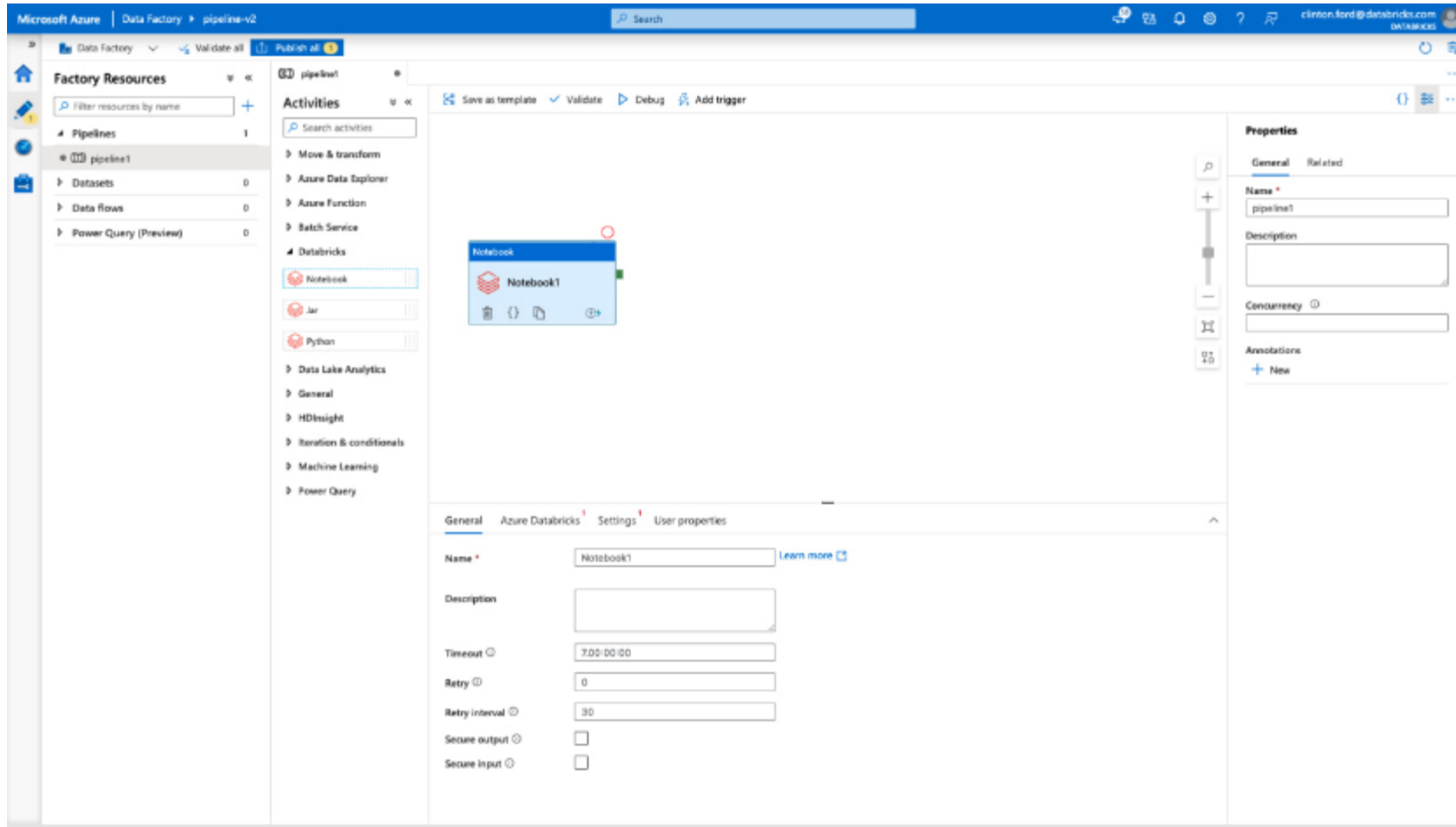
Typical Integration of Databricks in Azure



DATABRICKS CONNECTORS

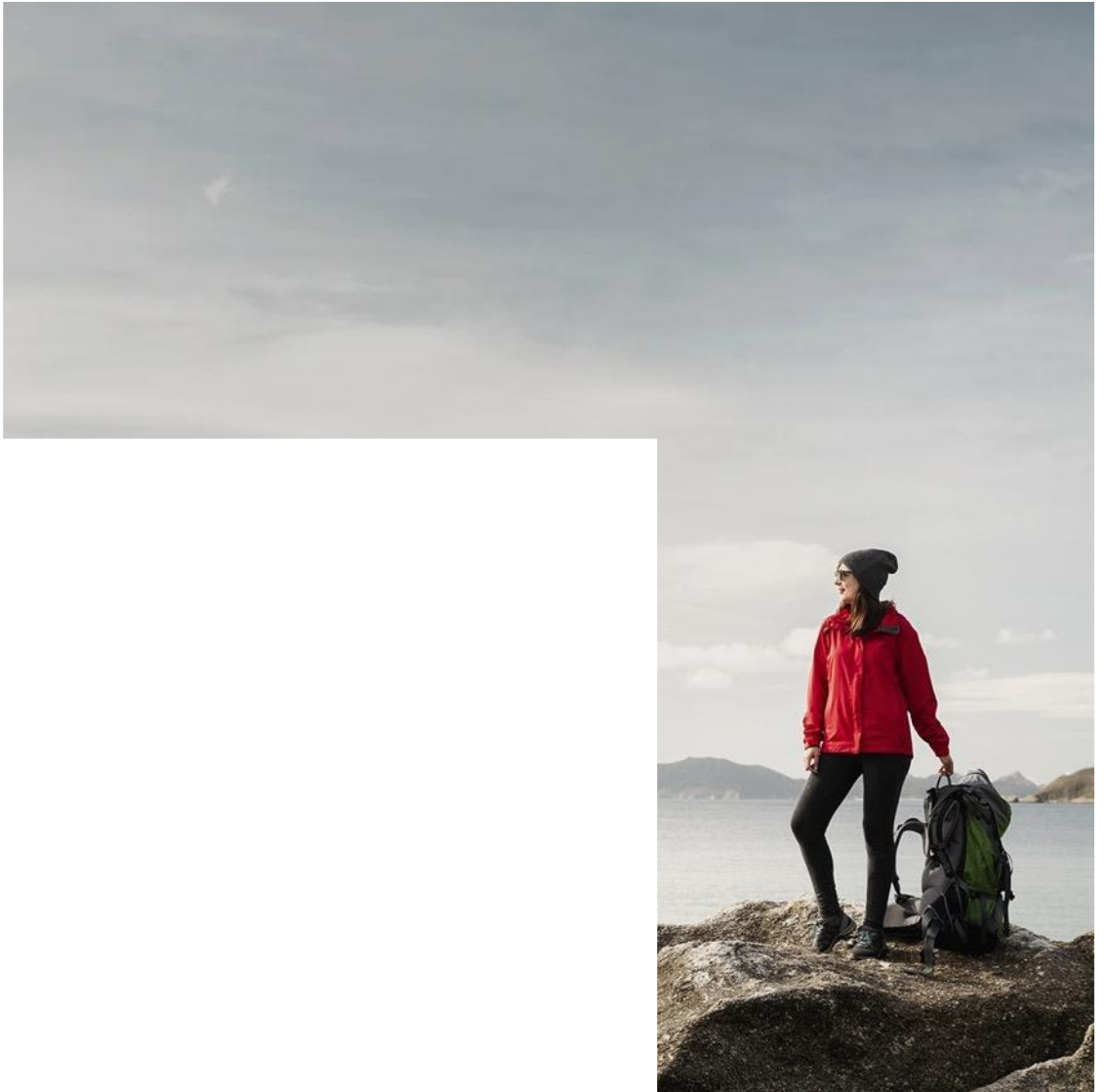
- 1.ETL Tools:** Databricks can be scheduled and run from ETL tools like Azure Data Factory
- 2.DATABASES:** Databricks can connect to: PostGres, Azure SQL Database, Snowflake.
- 3.DATALAKES:** Databricks can connect to datalakes in the cloud to read and to write data (requires a mounting point action to be done first).
- 4.BI TOOLS:** Databricks can connect to BI Tools like Power BI ad Microstrategy using the Databricks connector.

Azure Databricks with Azure Data Factory

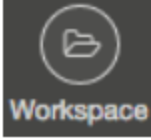


DEMO

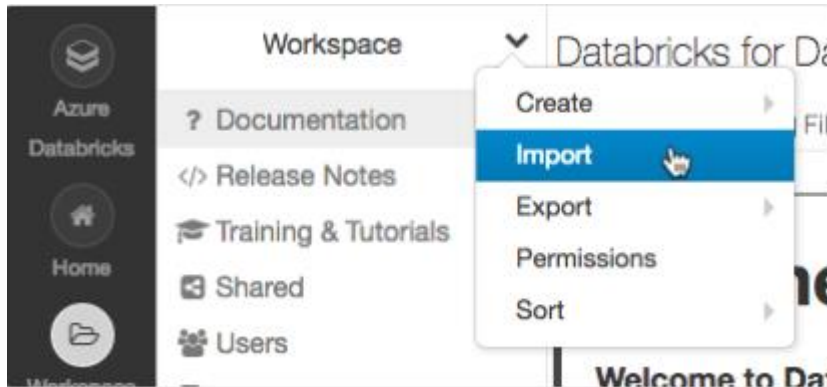
Connect to Azure Data Factory



Use Azure Data Factory with Databricks

1. Click the **Workspace** button  or the **Home** button  in the sidebar. Do one of the following:

- Next to any folder, click the  on the right side of the text and select **Import**.
- In the Workspace or a user folder, click  and select **Import**.



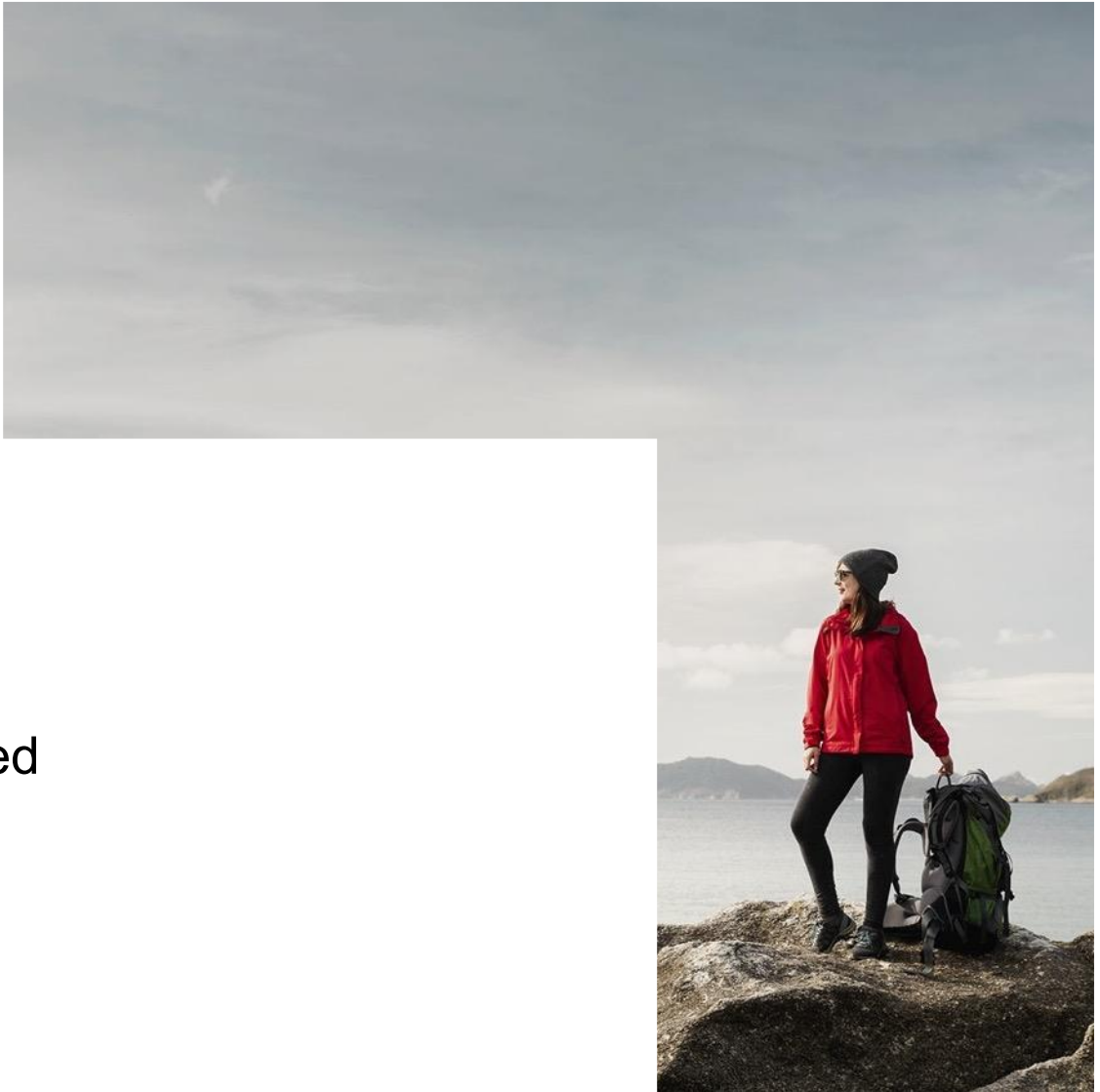
2. Specify the URL or browse to a file containing a supported external format.
<https://github.com/MicrosoftDocs/mslearn-data-ingestion-with-azure-data-factory/blob/master/DBC/03-Data-Ingestion-Via-ADF.dbc?raw=true>
3. Click **Import**.

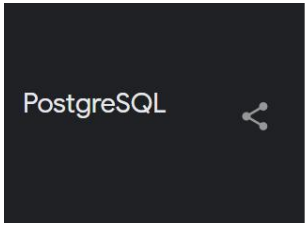
Use Azure Data Factory with Databricks

- **01 Getting Started** - This notebook contains instructions for setting up your storage account and Azure Data Factory (ADF). If you've already set up these in the previous unit, you can skip this notebook.
- **02 Data Ingestion** - In this notebook, you create a Data Factory v2 pipeline to ingest data from a public dataset into your storage account. After the data is ingested, you use a Databricks notebook function to examine the data.
- **03 Data Transformation** - This notebook contains instructions to create connectivity between your Azure Data Factory and Databricks workspace. You will add a Databricks Notebook activity to your ADF pipeline that will run a sample notebook to transform and restructure your data. You'll also perform some basic aggregation on the sample dataset to generate required reports.

YOUR TURN

Open Azure DataFactory and let's create a linked service and launch pipelines





CONNECTION TO DATABASES



- We are able to write Databricks data to a PostgreSQL Database or even Data warehouses like Snowflake.
- Using the native Databricks connector, we can even read Databricks metastore data in these databases or Datawarehouses

SECURITY FIRST : PRINCIPAL MODES OF CONNECTION

It's important that we when we connect to databases and datalakes that we don't leave passwords within the notebooks themselves. There are secure ways of creating connections between Azure components and Databricks.

- MOUNT POINT:** Databricks provides a method to create a mount point to seamlessly connect using secrets to a datalake or a database for all users in a workspace.
- SPARK CONFIG:** It's possible also to add configurations into the Azure databricks cluster advanced parameters to access datalakes and databases, at a notebook level.
- AZURE AD CREDENTIAL PASSTHROUGH** :allows you to authenticate to the Azure Datalake from Databricks clusters using the same Azure AD identity that you use to log in.

For more info: please see https://docs.databricks.com/_static/notebooks/data-import/azure-data-lake-store.html



CONNECTION TO BI TOOLS



- We must have tables stored in the HIVE metastore before we are able to connect a BI Tool directly to a table in Databricks.
- This can be in any format : csv, delta, parquet etc, as long as we are using the native Databricks connector.

Power BI

Import



Great for small data sources and personal data discovery
Fine for CSV files, spreadsheet data and summarized OLTP data

DirectQuery



The enterprise solution
Avoid data movement
Delegate query work to the back-end source; take advantage of Azure Synapse's advanced features

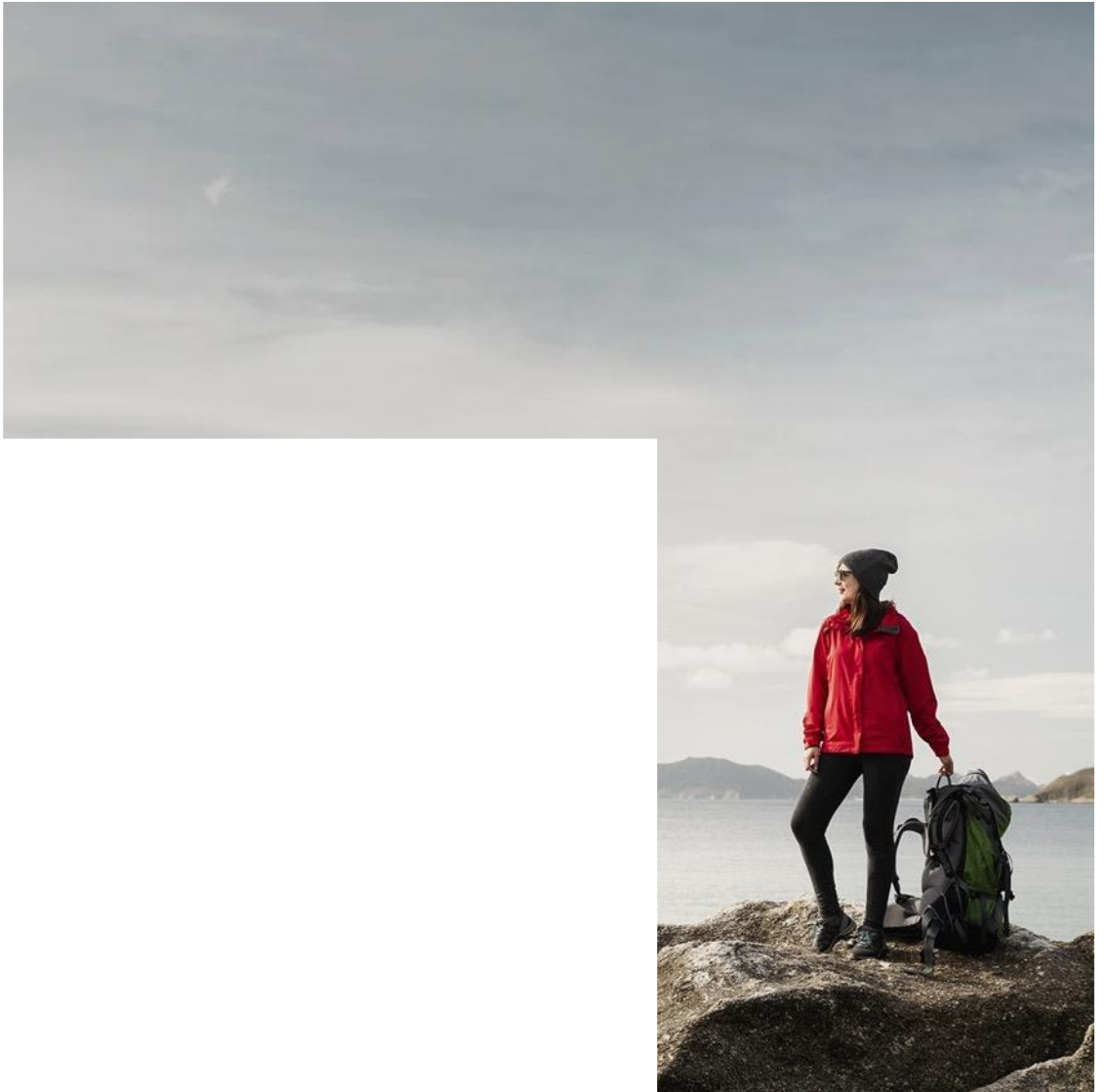
Composite Models & Aggregation Tables



Why choose? Import and DirectQuery in a single model
Keep summarized data local; get detail data from the source

DEMO

Connect to BI Tools



CONNECTION TO POWER BI



- Get the [Server Hostname and HTTP Path](#).
- Start Power BI Desktop.
- Click **Get data** or **File > Get data**.
- Click **Get data to get started**.
- Search for **Databricks**, choose the **Azure Databricks** connector, and click **Connect**.
- Enter the **Server Hostname** and **HTTP Path**.
- Select your **Data Connectivity mode**. Click **OK**.
- Enter your authentication credentials:
 - Personal Access Token**: Enter your Databricks personal access token
 - Username / Password**: Enter your Databricks username (typically your email address) and password from the [Requirements](#).
- Username and password authentication may be disabled if your Databricks workspace is [enabled for single sign-on \(SSO\)](#).
- If you cannot log in by using your Databricks username and password, try using the **Personal Access Token** option instead.
- Azure Active Directory**: Not applicable.
- Click **Connect**.
- Select the Databricks data to query from the Power BI **Navigator**.

CONNECTION TO POWER BI



Navigator

Display Options ▾

https://...:443/sql/...

- ☐ bmathew.gas_sales
- ☐ bmathew.global_page_views
- ☐ bmathew.sales
- ☐ bmathew.us_sales
- ☐ cleaned_taxes
- ☐ maddie_db.exploit
- ☐ maddie_db.something
- ☐ maddie_db2.acl_demo_table
- ☐ power_plant
- ☒ prices
- ☐ sales_long
- ☐ state_income_9f7c5_csv
- ☐ stockprice
- ☐ stockvolumes

prices

id	time	price	date
0	1/1/2017 12:00:00 AM	10	1/1/2017
1	1/1/2017 12:00:01 AM	10.0000006	1/1/2017
2	1/1/2017 12:00:02 AM	10.0000012	1/1/2017
3	1/1/2017 12:00:03 AM	10.0000018	1/1/2017
4	1/1/2017 12:00:04 AM	10.0000024	1/1/2017
5	1/1/2017 12:00:05 AM	10.000003	1/1/2017
6	1/1/2017 12:00:06 AM	10.0000036	1/1/2017
7	1/1/2017 12:00:07 AM	10.0000042	1/1/2017
8	1/1/2017 12:00:08 AM	10.0000048	1/1/2017
9	1/1/2017 12:00:09 AM	10.0000054	1/1/2017
10	1/1/2017 12:00:10 AM	10.000006	1/1/2017
11	1/1/2017 12:00:11 AM	10.0000066	1/1/2017
12	1/1/2017 12:00:12 AM	10.0000072	1/1/2017
13	1/1/2017 12:00:13 AM	10.0000078	1/1/2017
14	1/1/2017 12:00:14 AM	10.0000084	1/1/2017
15	1/1/2017 12:00:15 AM	10.000009	1/1/2017
16	1/1/2017 12:00:16 AM	10.0000096	1/1/2017
17	1/1/2017 12:00:17 AM	10.0000102	1/1/2017

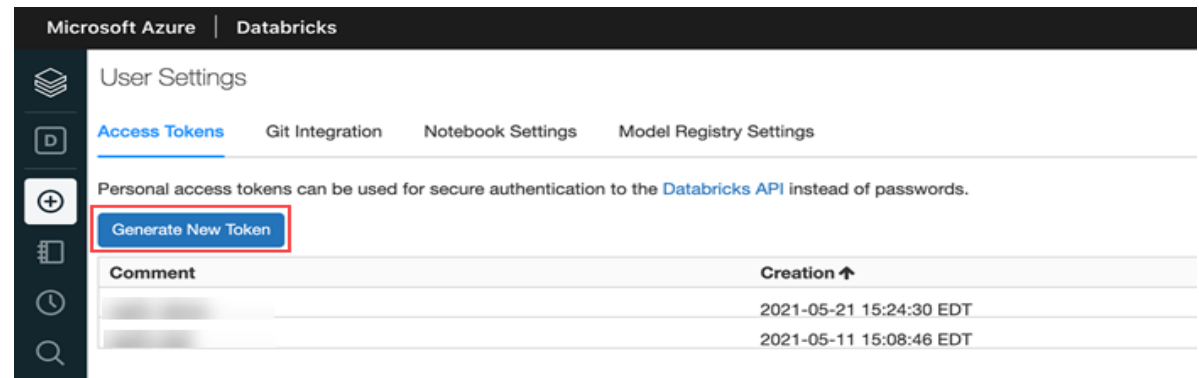
Load Edit Cancel

CONNECTION TO MICROSTRATEGY

STEP 1

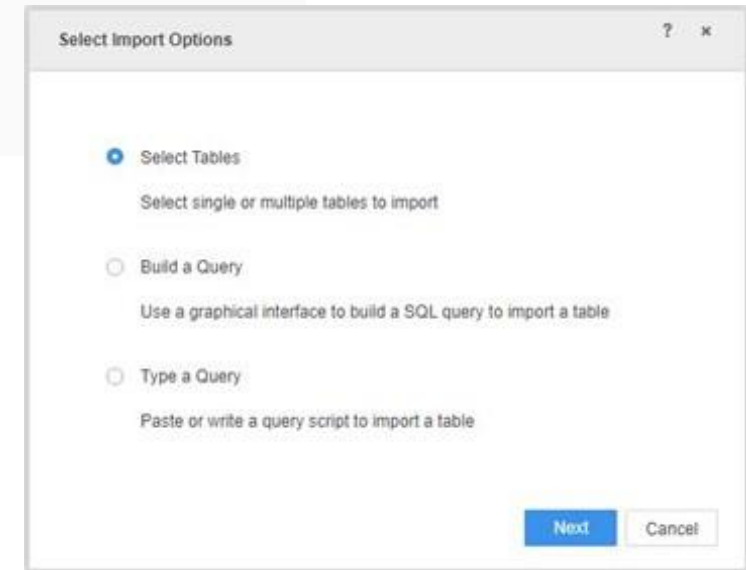
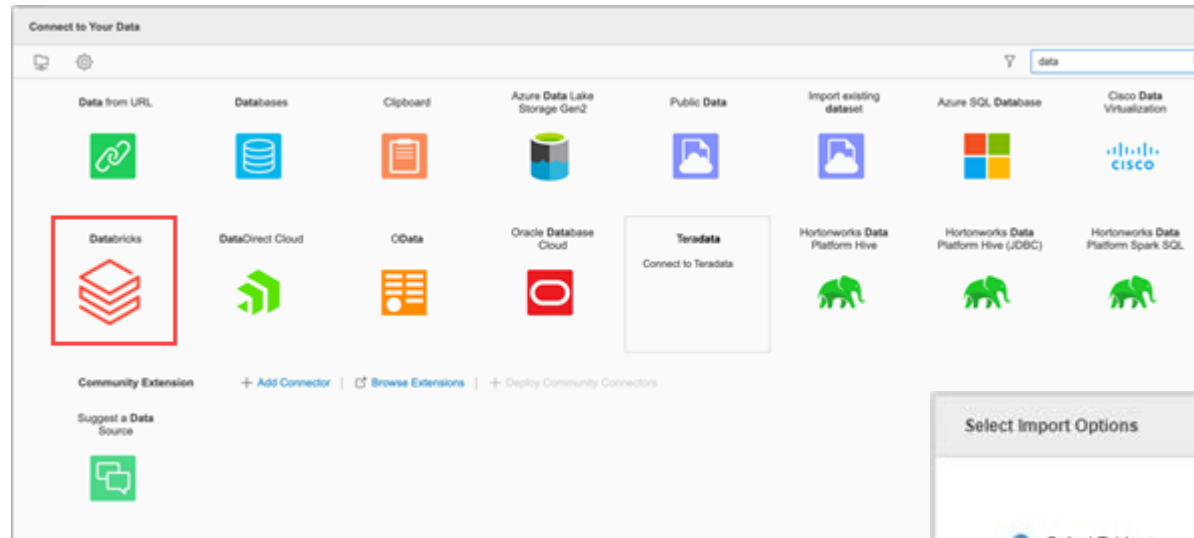
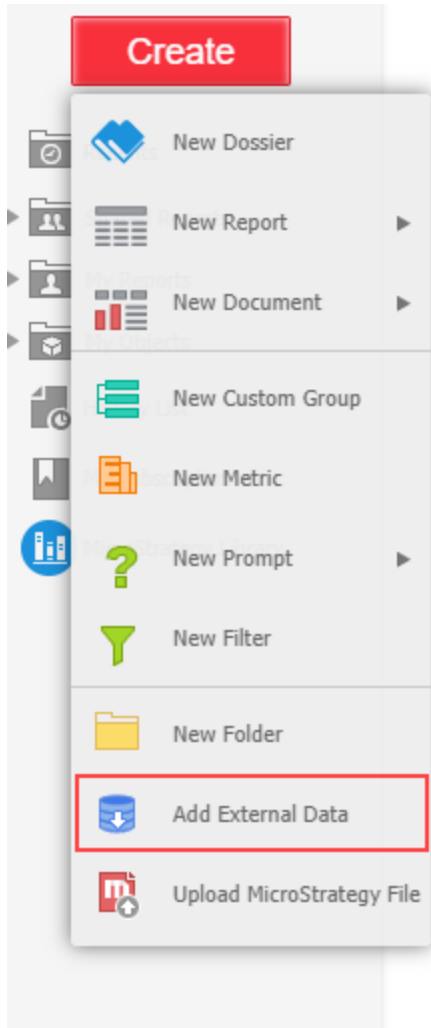
The MicroStrategy logo is displayed in white text on a red square background.

1. Log into your Databricks workspace
2. Select your cluster to go to the Clusters management page.
3. Click the **Advanced Options** drop-down.
4. Copy the JDBC URL for later use.
5. Click the user profile icon in the top right of your workspace.
6. Click **User Settings**.
7. Under the Access Tokens tab, click the **Generate a New Token** button.
8. Complete the required fields for the new token.
9. Click the **Generate** button.
10. Copy the generated token for later use.



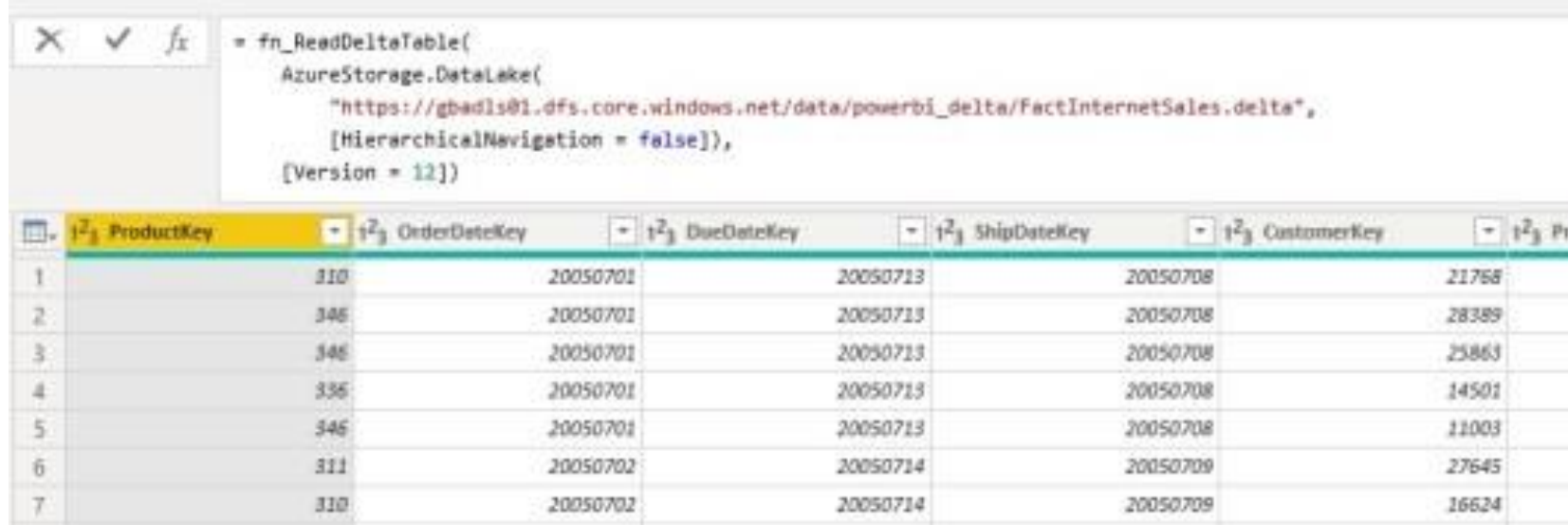
CONNECTION TO MICROSTRATEGY STEP 2

MicroStrategy



PROBLEM OF COST : CONNECTIONS TO BI TOOLS

- Power BI takes **too long** while connecting in mode Direct Query to Delta tables. Furthermore, the cluster must always be on , either for import or for direct query mode. **This becomes extremely expensive to maintain.**



The screenshot shows a Power BI interface. At the top, the DAX formula bar contains the function `fn_ReadDeltaTable` with arguments for an Azure Storage Data Lake, a URL to a Delta table, and options for hierarchical navigation and version. Below the formula bar, a table is displayed with columns for ProductKey, OrderDateKey, DueDateKey, ShipDateKey, CustomerKey, and Product Name. The table contains 7 rows of data.

	t ² ProductKey	t ² OrderDateKey	t ² DueDateKey	t ² ShipDateKey	t ² CustomerKey	t ² Product Name
1	310	20050701	20050713	20050708	21758	
2	346	20050701	20050713	20050708	28389	
3	346	20050701	20050713	20050708	25863	
4	336	20050701	20050713	20050708	14501	
5	346	20050701	20050713	20050708	11003	
6	311	20050702	20050714	20050709	27645	
7	310	20050702	20050714	20050709	16624	

SOLUTION : USE DIRECT DELTA CONNECTOR

https://github.com/gbrueckl/connectors/blob/master/powerbi/fn_ReadDeltaTable.pq

- Just copy the code within the link into a blank query in Power BI desktop, and it will create the function to read a Delta table stored in the Datalake.
- Works with GCP buckets, ADLS Gen 1 and Gen 2, and AWS S3 Buckets

CHAPTER OBJECTIVE REVIEW

- Understand how Databricks fits in the Azure cloud
- Look at how to connect Databricks with Datafactory and Datalake
- Understand the 3 principal ways to connect Databricks to another Azure component.
- Look at how to connect Databricks to BI Tools like Power BI and Microstrategy

Go to Kahoot.it



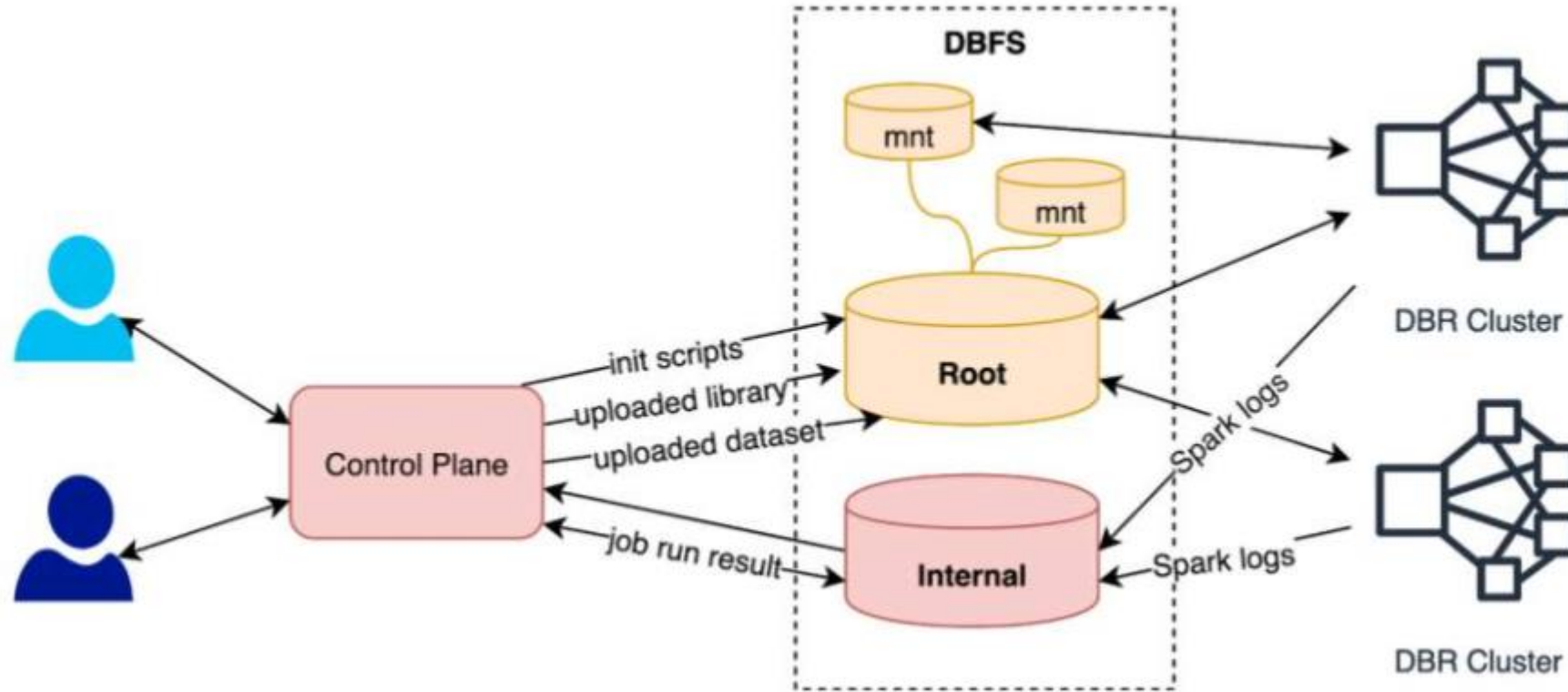
Databricks Storage



OBJECTIVES

- Understand how to store data in Databricks
- Understand what is DBFS
- Understand when not to use DBFS

WHAT IS DATABRICKS FILE SYSTEM ?



Databricks File System (DBFS) is a distributed file system mounted into an Azure Databricks workspace and available on Azure Databricks clusters. DBFS is an abstraction on top of scalable object storage and offers many benefits

WHAT IS INSIDE DATABRICKS FILE SYSTEM ?

The Default storage location is known as DBFS root.
We have several types of data stored here:

/FileStore : Imported Data Files

/databricks-datasets : Sample Public Datasets

/databricks-results: Files generated by downloading the full results of a query

/databricks/init: Global and Cluster Init Scripts

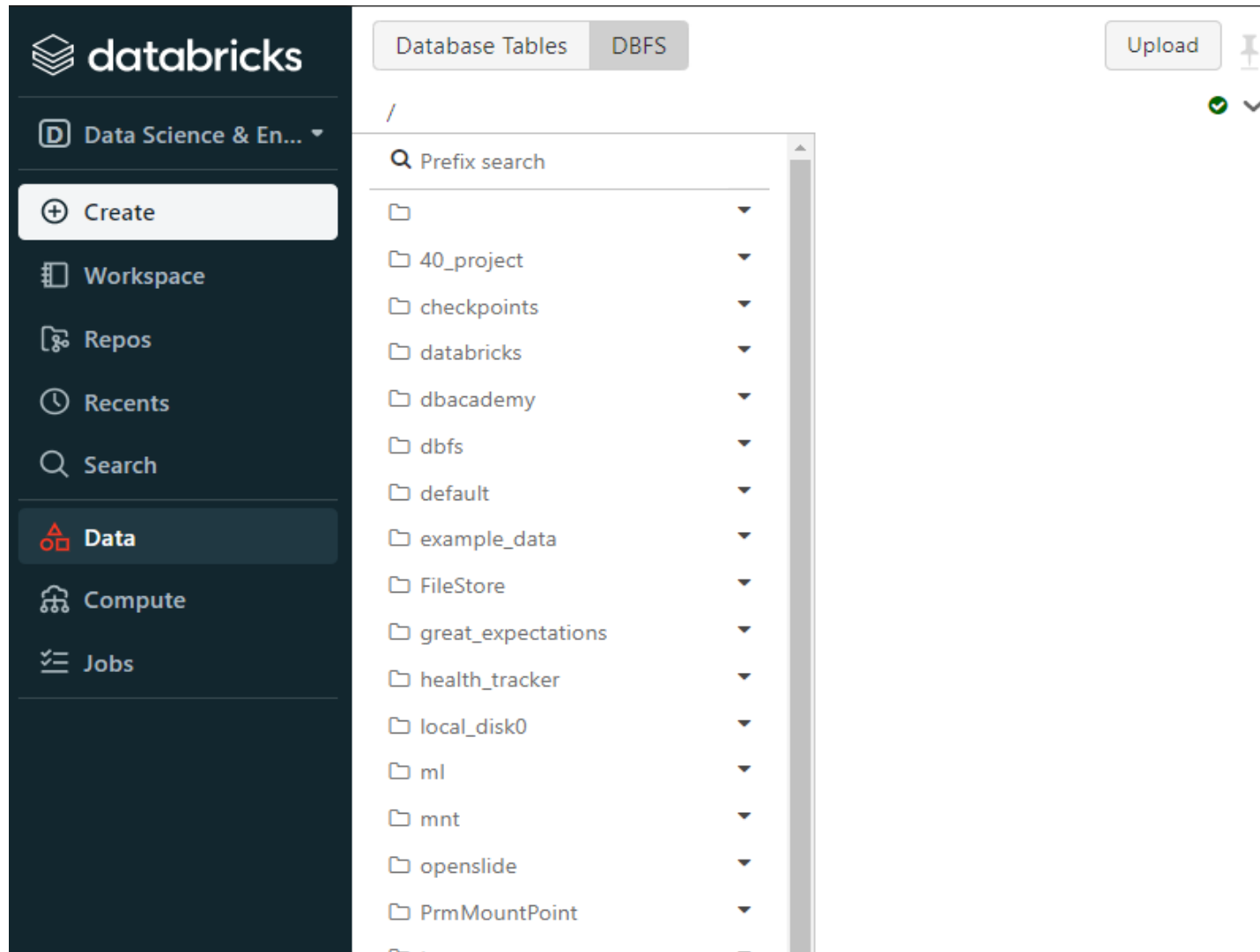
/user/hive/warehouse: Data and Metadata for Hive tables

path	name	size
dbfs:/FileStore/	FileStore/	0
dbfs:/databricks-datasets/	databricks-datasets/	0
dbfs:/databricks-results/	databricks-results/	0
dbfs:/tmp/	tmp/	0

ADVANTAGES OF DATABRICKS FILE SYSTEM

- Allows you to mount storage objects so that you can seamlessly access data without requiring credentials.
- Allows you to interact with object storage using directory and file semantics instead of storage URLs.
- Persists files to object storage, so you won't lose data after you terminate a cluster.

HOW TO ACCESS THE DBFS



HOW TO UPLOAD DATA TO THE DBFS

Upload Data to DBFS

DBFS Target Directory ?

/FileStore

(optional)

Select

Files uploaded to DBFS are accessible by everyone who has access to this workspace. [Learn more](#)

Files ?

Drop files to upload, or click to browse

Done

WHEN NOT TO USE DBFS

Note: It is recommended to not use DBFS for production data storage.

It shouldn't be used also to store large files or ML models for a long period of time.
Lack of replication means you can lose critical data
Use Datalake or Blob Storage instead.

CHAPTER REVIEW OBJECTIVES

- Understand how to store data in Databricks
- Understand what is DBFS
- Understand when not to use DBFS

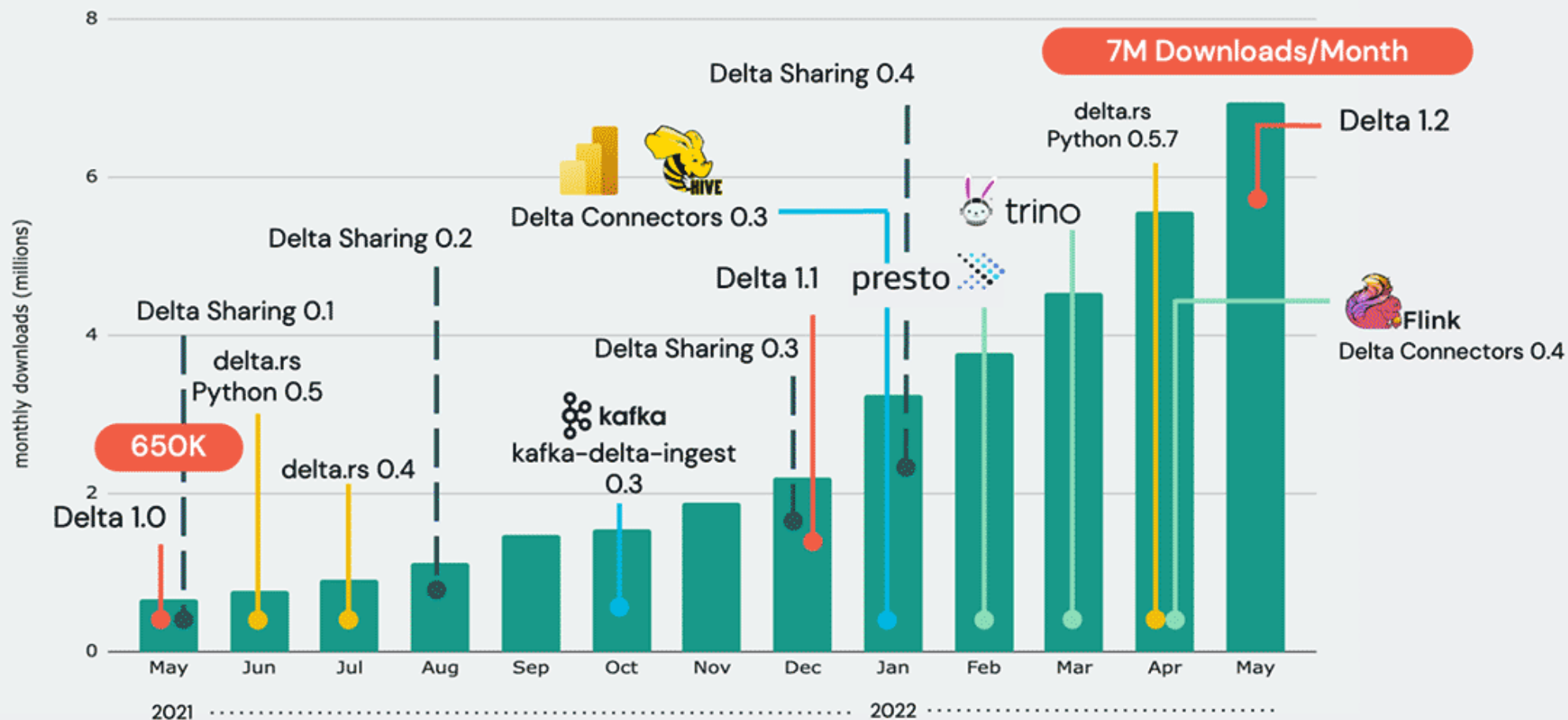
Delta, Delta, and more Delta



OBJECTIVES

- Understand what is Delta Lakehouse and Delta format
- Understand why Delta is useful
- Know the major differences between Delta and Parquet
- Learn how to create, use and delete database with Delta tables
- Learn interesting features of Delta

HISTORY OF DELTA LAKE



DELTA FILE FORMAT



Delta Lake is a data format based on Apache Parquet.

It's an open source project (<https://github.com/delta-io/delta>), delivered with Databricks runtimes and it's the default table format from runtimes 8.0 onwards on Databricks .

You can read and write VERY quickly with delta , as it is a columnar format and hence fitted for analytic workloads.

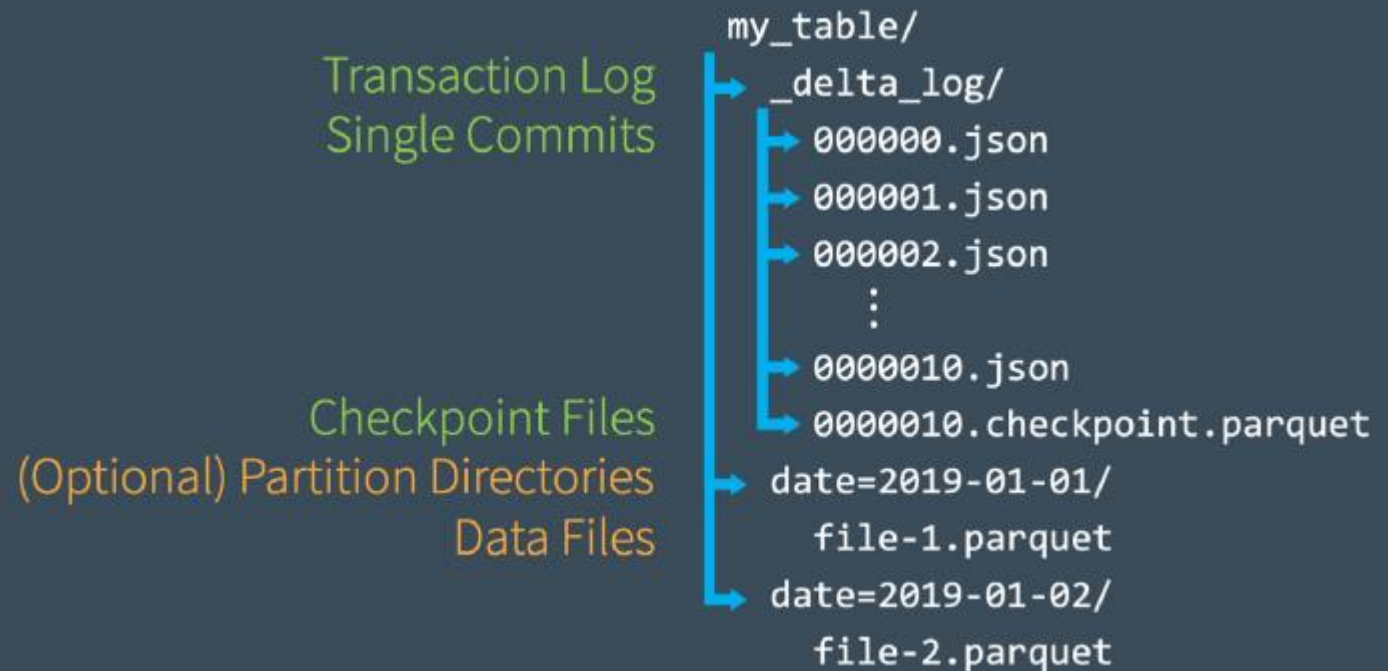
Delta file format is built to work on high data volumes
... by design.

DELTA FILE FORMAT



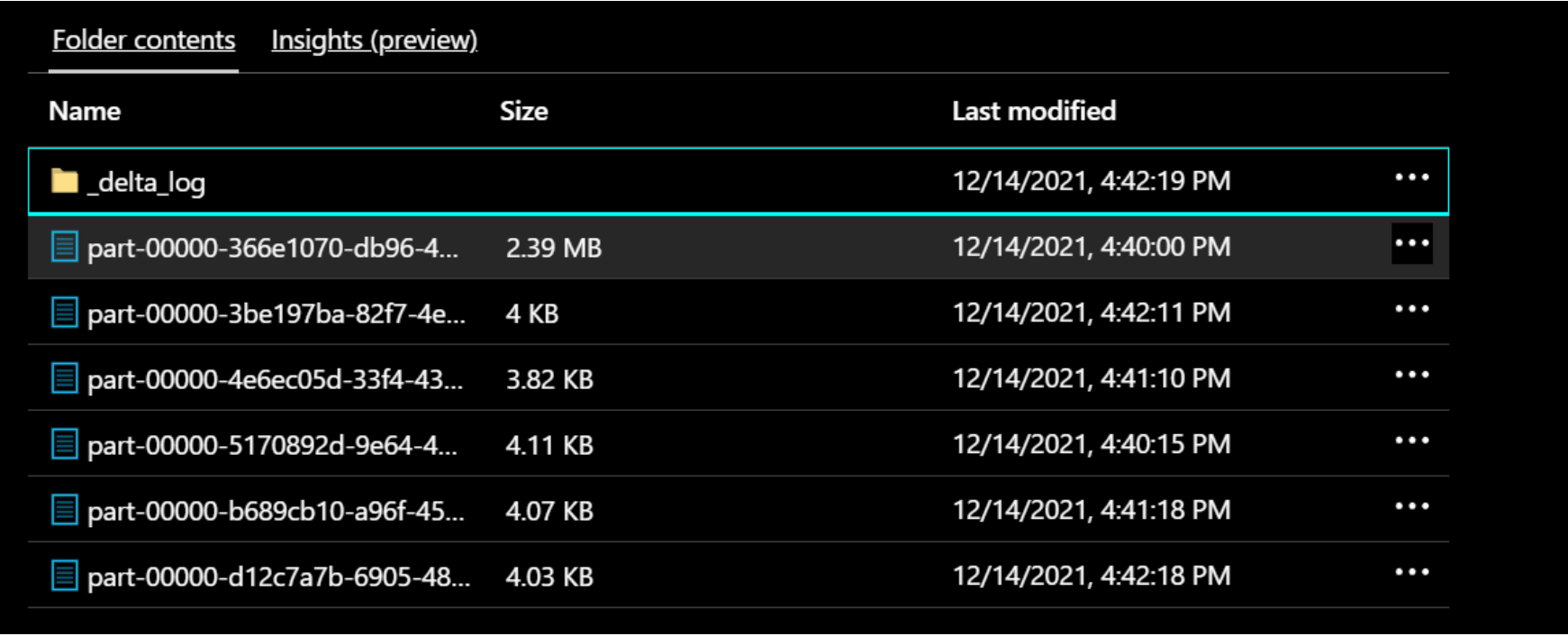
Delta Lake is made of many components:

- Parquet data files organized or not as partitions
- Json files as transaction log
- Checkpoint files



HOW TO DIFFERENTIATE PARQUET FROM DELTA IN OUR DATALAKE ?

The delta log is an ordered record of every transaction made on a delta table since inception.
If you see a delta log, it means the table is in delta format, not in parquet.
So try `spark.read.format(« delta »)` instead of `spark.read.format(« parquet »)` when trying to read the file ;)



<u>Folder contents</u>		<u>Insights (preview)</u>
Name	Size	Last modified
 _delta_log		12/14/2021, 4:42:19 PM
 part-00000-366e1070-db96-4...	2.39 MB	12/14/2021, 4:40:00 PM
 part-00000-3be197ba-82f7-4e...	4 KB	12/14/2021, 4:42:11 PM
 part-00000-4e6ec05d-33f4-43...	3.82 KB	12/14/2021, 4:41:10 PM
 part-00000-5170892d-9e64-4...	4.11 KB	12/14/2021, 4:40:15 PM
 part-00000-b689cb10-a96f-45...	4.07 KB	12/14/2021, 4:41:18 PM
 part-00000-d12c7a7b-6905-48...	4.03 KB	12/14/2021, 4:42:18 PM

BENEFITS OF DELTA LAKE

Key Technical Features

ACID Transactions

- Ensures data integrity
- Serializability

Scalable Metadata Handling

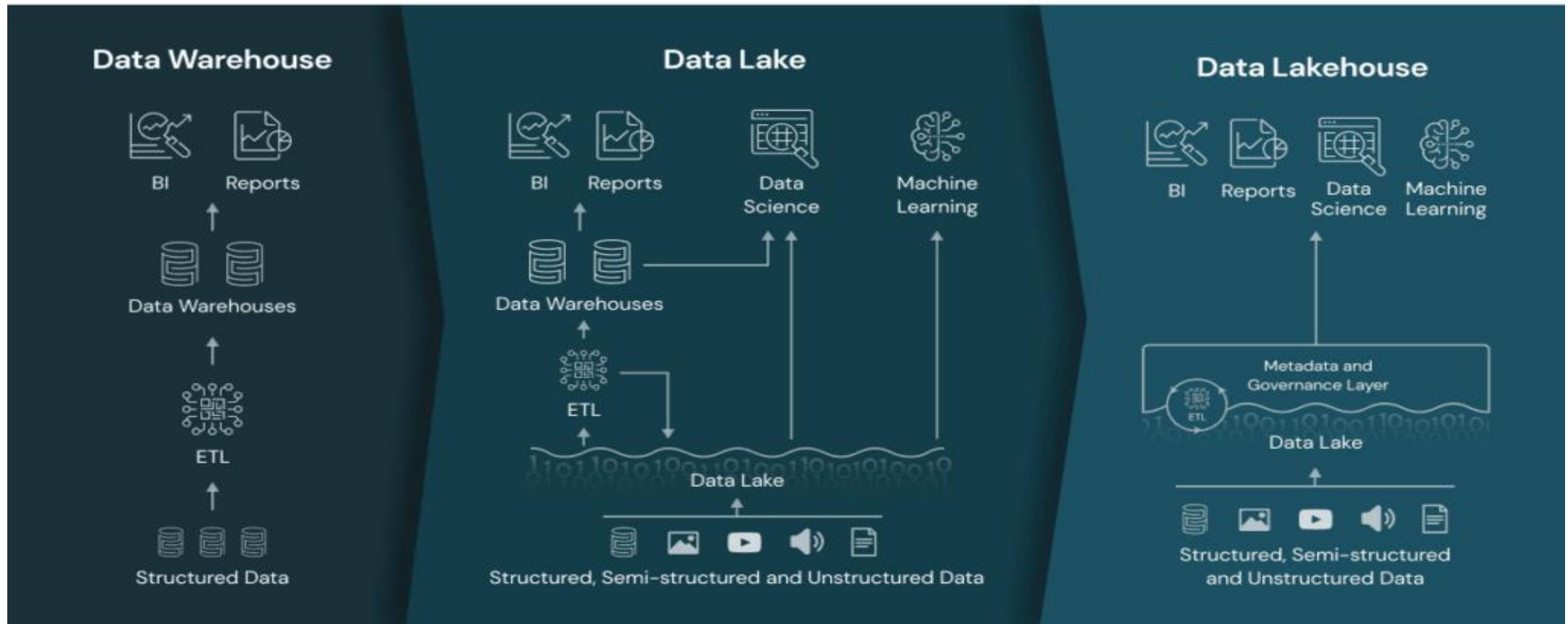
- Petabyte scale with billions of partitions
- Spark processing

Unified Batch and Streaming

- Batch table
- Streaming source and sink

DELTA LAKEHOUSE

With the benefits of ACID transactions, scalable metadata handling, Delta Lake enables organizations to build Data Lakehouses.



BENEFITS OF DELTA LAKE

- But Delta Lake does not stop there. Today, it is the most comprehensive Lakehouse format used by over 7,000 organizations, processing exabytes of data per day.
- Beyond core functionality that enables seamlessly ingesting and consuming streaming and batch data in a reliable and performant manner, one of the most important capabilities of Delta Lake is Delta Sharing, which enables different companies to share data sets in a secure way.
- Let's dive into a few features...

USEFUL DELTA FEATURES: MERGE INTO

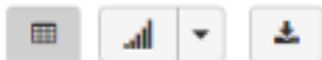
MERGE INTO : merges a set of updates, insertions, and deletions based on a source table into a target Delta table. Very useful when you only want to insert/delete or update rows that match a certain condition.

```
1 %sql
2 MERGE INTO demodelta.customer2 as c2
3 USING demodelta.customer as c
4 ON c.C_CUSTKEY=c2.C_CUSTKEY
5 WHEN NOT MATCHED
6 THEN INSERT *
```

► (6) Spark Jobs

	num_affected_rows ▲	num_updated_rows ▲	num_deleted_rows ▲	num_inserted_rows ▲
1	8396742	0	0	8396742

Showing all 1 rows.



USEFUL DELTA FEATURES: TIME TRAVEL

Examples

Select * from table_name
VERSION AS OF 76000

Select * from table_name
TIMESTAMP AS OF «2021-01-01 »

Table: operations

operations Refresh

Shared Autoscaling

Details History

Filter

version	timestamp	userId	userName	operation	operationParameters
76874	2019-01-24 02:45:32	null	null	STREAMING UPDATE	{ "outputMode": "Append", "queryId": "29693a5d-b4aa-4390-8983-554081730a22", "epochId": "10350" }
76873	2019-01-24 02:45:09	null	null	STREAMING UPDATE	{ "outputMode": "Append", "queryId": "29693a5d-b4aa-4390-8983-554081730a22", "epochId": "10349" }
76872	2019-01-24 02:44:04	null	null	STREAMING UPDATE	{ "outputMode": "Append", "queryId": "29693a5d-b4aa-4390-8983-554081730a22", "epochId": "10348" }
76871	2019-01-24 02:42:56	null	null	STREAMING UPDATE	{ "outputMode": "Append", "queryId": "29693a5d-b4aa-4390-8983-554081730a22", "epochId": "10347" }
76870	2019-01-24 02:41:53	null	null	STREAMING UPDATE	{ "outputMode": "Append", "queryId": "29693a5d-b4aa-4390-8983-554081730a22", "epochId": "10346" }
76869	2019-01-24 02:40:26	null	null	STREAMING UPDATE	{ "outputMode": "Append", "queryId": "29693a5d-b4aa-4390-8983-554081730a22", "epochId": "10345" }

USEFUL DELTA FEATURES: INSERT INTO

- Imagine that you want to insert some rows into a Delta table, without having to delete it and create it again. INSERT INTO my_table allows this type of operation.
- When combined with Delta Time Travel, it is also very versatile: imagine that we have a certain user whose data we want to reconstitute from yesterday (as being the correct version of the truth): we can use a combination of INSERT INTO and TIME TRAVEL to do this.

```
INSERT INTO my_table  
SELECT * FROM my_table TIMESTAMP AS OF date_sub(current_date(), 1)  
WHERE userId = 111
```

USEFUL DELTA FEATURES: VACUUM

- Vacuum is a form of a specialised Delete: it removes files no longer in last state of transaction log and older than a retention threshold. It allows you to keep the table name and the latest version, while simply getting rid of old versions.
- Default period of VACCUM is **7 days**: guideline set by editor.

```
%sql
-- #Script to delete and drop a table
set spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM SCAI_TABLES_SHARED.CUST_ORDER_HISTORY RETAIN 0 HOURS;
```

USEFUL DELTA FEATURES: OPTIMIZE

Use it to adjust the distribution of partitions of data inside your datalake. Helps in tackling small partitions problems, and overall addressing network latency and volume of file metadata.

Cmd 8

```
1 OPTIMIZE business.sales
```

► (14) Spark Jobs

path ▼	metrics
null	► {"numFilesAdded":2378,"numFilesRemoved":40545,"filesAdded":{"min":11840,"max":1033764,"avg":452523,"totalFiles":2378,"totalSize":1076099954},"filesRemoved":{"min":9961,"max":115670,"avg":36686,"totalFiles":40545,"totalSize":1487439611},"partitionsOptimized":2509,"zOrderStats":null,"numBatches":6}



ADVANTAGES DELTA OVER PARQUET



- **PREVENT DATA CORRUPTION:** If an incompatible schema change is detected, Delta Lake will throw an exception and prevent the table from being corrupted with columns of incompatible types.
- **ALLOWS TIME TRAVEL** to visit previous versions of a table , in order to capture changes and undo unwanted changes.
- **CAN ENFORCE NOT NULL** constraints on columns in Delta, which is not possible in Parquet.
- **ALLOWS OPTIMIZATION** using ZORDER algorithm to skip ingestion of files that do not match query predicate.
- **WORKS WITH SPARK STRUCTURED STREAMING** to allow built-in checkpointing between delta tables.
- **CHANGE DATA CAPTURE** is possible in Databricks Delta using some advanced parameters.

Source : <https://databricks.com/fr/blog/2020/08/21/top-5-reasons-to-convert-your-cloud-data-lake-to-a-delta-lake.html>

Delta Lake 2.0: We are open-sourcing all of Delta

Unlock the power of Delta Lake



Coming Soon!



ACID Transactions



Scalable Metadata



Time Travel



Schema Evolution



OPTIMIZE



OPTIMIZE ZORDER



Change data feed



Generated column support w/ partitioning



Clones



Unified Batch/Streaming



Schema Enforcement



Audit History



DML Operations



Table Restore



S3 Multi-cluster writes



Data Skipping via Column Stats



Identity Columns



Iceberg to Delta converter



Compaction



MERGE Enhancements



Stream Enhancements



Simplified Logstore



Generated Columns



Multi-part checkpoint writes



Column Mapping



Subqueries in deletes and updates



Fast metadata only deletes

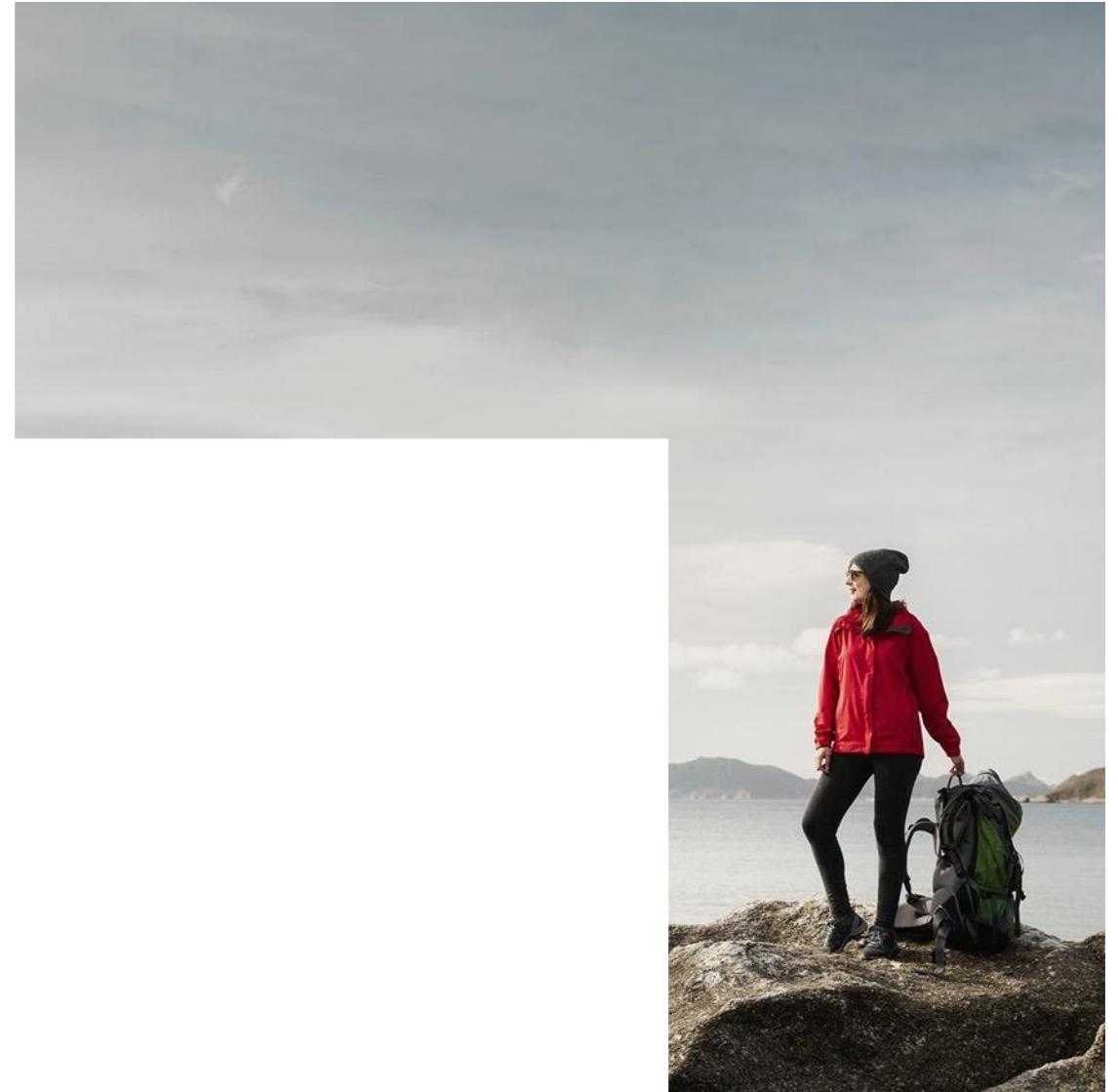
BRONZE, SILVER AND GOLD TABLES

The Delta architecture design pattern



Exercise

4.1 Delta Lake



Go to Kahoot.it



RECAP OF COMMANDS

Data Definition Language

- CREATE TABLE sales USING DELTA
- DELETE TABLE IF EXISTS sales

Delta Special

- OPTIMIZE sales ZORDER BY primary_id
- SELECT * from sales VERSION AS OF x
- VACUUM sales

Data Manipulation Language

- INSERT INTO sales...
- UPDATE sales...
- MERGE INTO sales...

CHAPTER OBJECTIVE REVIEW

- Understand what is Delta Lakehouse and Delta format
- Understand why Delta is useful
- Know the major differences between Delta and Parquet
- Learn how to create, use and delete database with Delta tables
- Learn interesting features of Delta

WORKFLOWS



CGI

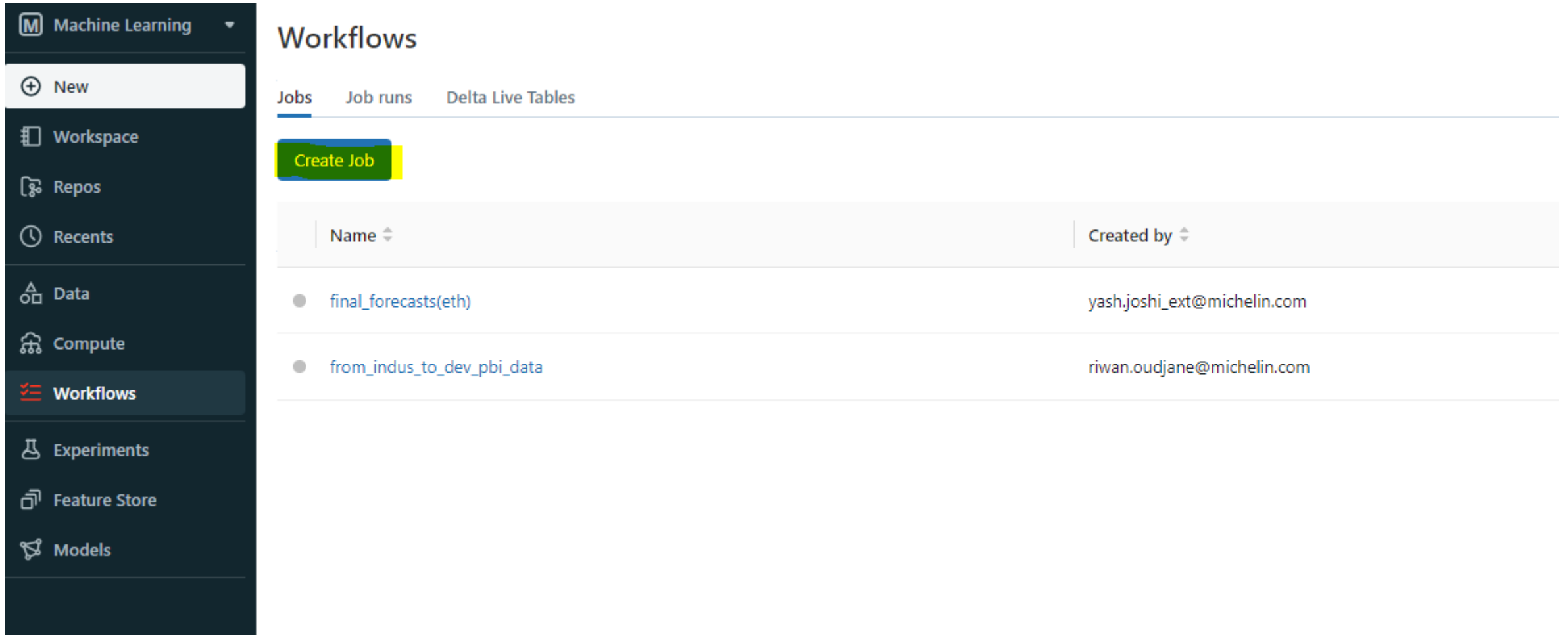
OBJECTIVES

- Understand how to set up your first workflows
- Understand how to set up email alerts on workflows

WHAT ARE WORKFLOWS ?

- Workflows allows users to build ETL pipelines that are automatically managed, including ingestion, and lineage, directly within Databricks ;
- Workflows is available across GCP, AWS, and Azure, giving you full flexibility and cloud independence.

SET UP WORKFLOWS



The screenshot displays the Machine Learning interface. On the left is a dark sidebar with a menu containing: Machine Learning (with a dropdown arrow), New, Workspace, Repos, Recents, Data, Compute, Workflows (highlighted with a red checkmark icon), Experiments, Feature Store, and Models. The main content area is titled 'Workflows' and has three tabs: Jobs (selected), Job runs, and Delta Live Tables. Below the tabs is a green 'Create Job' button. Underneath the button is a table with two columns: 'Name' and 'Created by'. The table contains two entries:

Name	Created by
final_forecasts(eth)	yash.joshi_ext@michelin.com
from_indus_to_dev_pbi_data	riwan.oudjane@michelin.com

Go to the Workflows tab , and then click Create Job.

SET UP WORKFLOWS

Fill in the Details.

Click Add under Parameters to get the Emails field

Task name * 

TestWorkflow

Type *

Source * 

Notebook | v

Workspace | v

Path * 

Select Notebook | v

Cluster * 

Cluster_SAI_ST 96 GB · 48 Cores · DBR 10.4 LTS ML · Spark 3.2.1 · Scala 2.12  | v

 Jobs running on all-purpose clusters are considered all-purpose compute. [Learn more](#)

Parameters 

UI | JSON

Add

Emails 

yash.joshi_ext@michelin.com ☐ Start ☐ Success ☒ Failure 

Add

Retries 

No retries 

MONITOR WORKFLOWS

Workflows > Jobs > RUN_BO >

RUN_BO

Run now

Runs Tasks

Runs

Start time from → Start time to

Table

Matrix

Refresh

Active runs

Start time	Run ID	Launched	Duration	Status	Actions
------------	--------	----------	----------	--------	---------

Run now / Run now with different parameters

Completed runs (past 60 days)

Latest successful run (refreshes automatically)

Refresh

Start time	Run ID	Launched	Duration	Status	Actions
Oct 10 2022, 9:00 AM CEST	1162368	By scheduler	37m 45s	✓ Succeeded	
Oct 6 2022, 6:14 AM CEST	1109866	Manually	36m 21s	✓ Succeeded	
Oct 4 2022, 11:37 AM CEST	1091362	Manually	33m 10s	✗ Failed	



Job details

Job ID 404040224697084

Creator yash.joshi_ext@michelin.com

Run as yash.joshi_ext@michelin.com

Tags + Tag



Schedule

At 08:00 AM, only on Monday (UTC+01:00 — Dublin, Edinburgh, Lisbon, L...

Edit schedule

Pause



Compute

Cluster_A2A

Driver: Standard_DS13_v2 · Workers: Standard_DS13_v2 · 0 workers · from pool · 10.4 LTS (includes Apache Spark 3.2.1, Scala 2.12)

View cluster

Swap

Spark UI

Logs

Metrics



Notifications

RUN_BO

Runs

→ Start time to

Table

Matrix

Job Runs

Tasks

RUN_BO_UPDATE_CONSUMPTION

RUN_BO_AGGREGATE

RUN_BO_RETARD_PORTEFEUILLE

RUN_BO_ORDER_REPORTING

RUN_BO_REF_ITEM

38m 12s

19m 6s

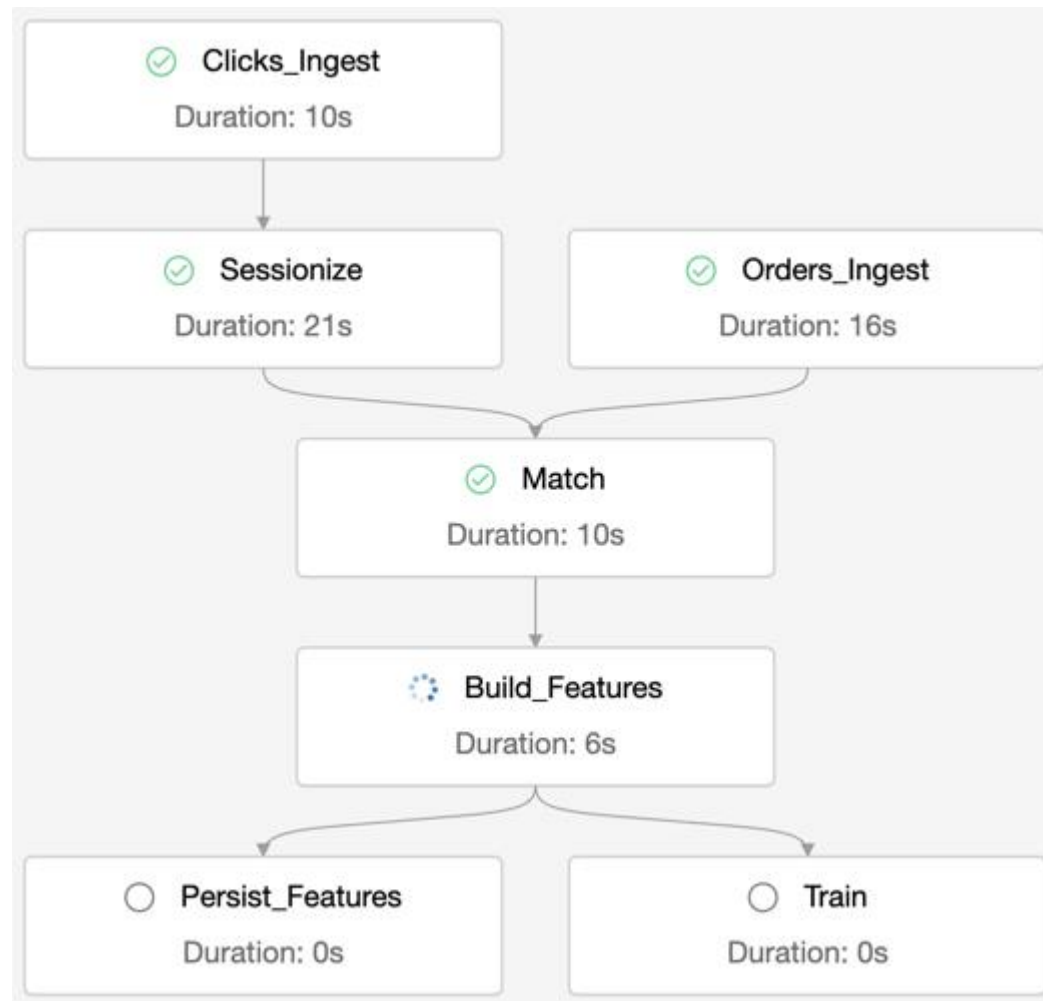
Aug 15, 09:00

Sep 26, 09:00

Oct 10, 09:00

© 2023 CGI inc.

WORKFLOWS GRAPHS HELP TO SHOW LINEAGE



WORKFLOWS FOR MACHINE LEARNING USE CASES



CHAPTER OBJECTIVE REVIEW

- Understand how to set up your first workflows
- Understand how to set up email alerts on workflows

Time for a break



CGI

Spark Optimization

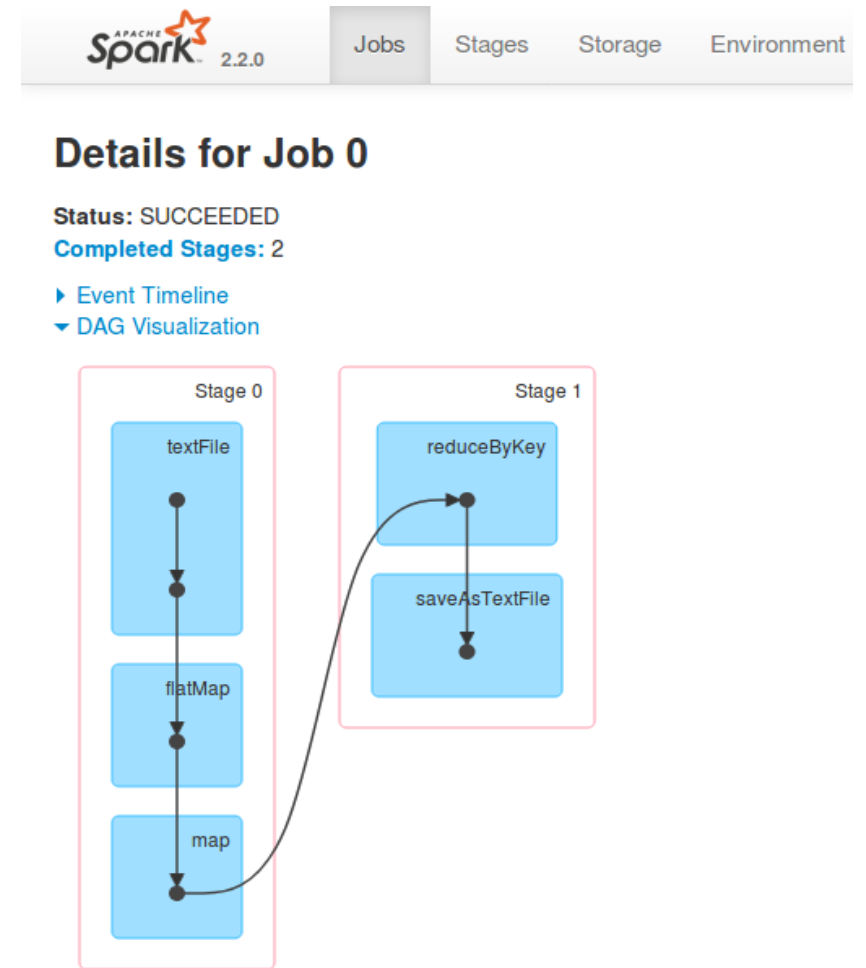


OBJECTIVES

- Understand what is a DAG in Spark
- Understand how to access and read the Spark UI
- Understand and define the most common Spark problems: Skew, Spill and Shuffle.
- Look at how to optimize performance

WHAT IS A DAG ?

- A **DAG** is a directed graph in which there are no cycles or loops, i.e., if you start from a node along the directed branches, you will never visit the already visited node by any chance.
- Spark **Driver** builds a logical flow of operations that can be represented in a graph which is directed and acyclic, hence **DAG** (Directed Acyclic Graph).
- Spark will construct all the steps of data manipulation, also known as **transformations**, in its logical plan, then we will demand the execution of the DAG using an **action**.
- This is why Spark is called a **lazy** framework, as it only applies the execution of the DAG when it finds an action.



WHAT IS ADAPTIVE QUERY EXECUTION ?

Adaptive Query Execution (AQE) is an optimization technique in Spark SQL that makes use of the runtime statistics to choose the most efficient query execution plan, which is enabled by default since Apache Spark 3.2.0. Spark SQL can turn on and off AQE by spark.

Enable AQE

```
set spark.sql.adaptive.enabled = true;
```

	key ▲	value ▲
1	spark.sql.adaptive.enabled	true

Showing all 1 rows.

ADAPTIVE QUERY EXECUTION FUNCTIONALITIES

1. Change Join Strategies from SortMerge to BroadcastHash
2. Dynamic Coalesce Shuffle Partitions
3. Handle Skew Partitions in the Join operation
4. Dynamic Partition Pruning

How do you know if AQE is enabled?
Look for 'AdaptiveSparkPlan' in Spark UI
using df.explain()



```
(14) AdaptiveSparkPlan  
Output [3]: [NeighborhoodDis  
Arguments: isFinalPlan=true
```

© 20

AQE's like having a Superhero
turbo-charging your queries



SORT MERGE JOIN

Problem: `SortMergeJoin` is Poor Performer

Here I force `SortMergeJoin` strategy to view Performance

```
# Lab 01
spark.conf.set("spark.sql.autoBroadcastJoinThreshold",-1)
spark.conf.set("spark.sql.adaptive.enabled",False)
```

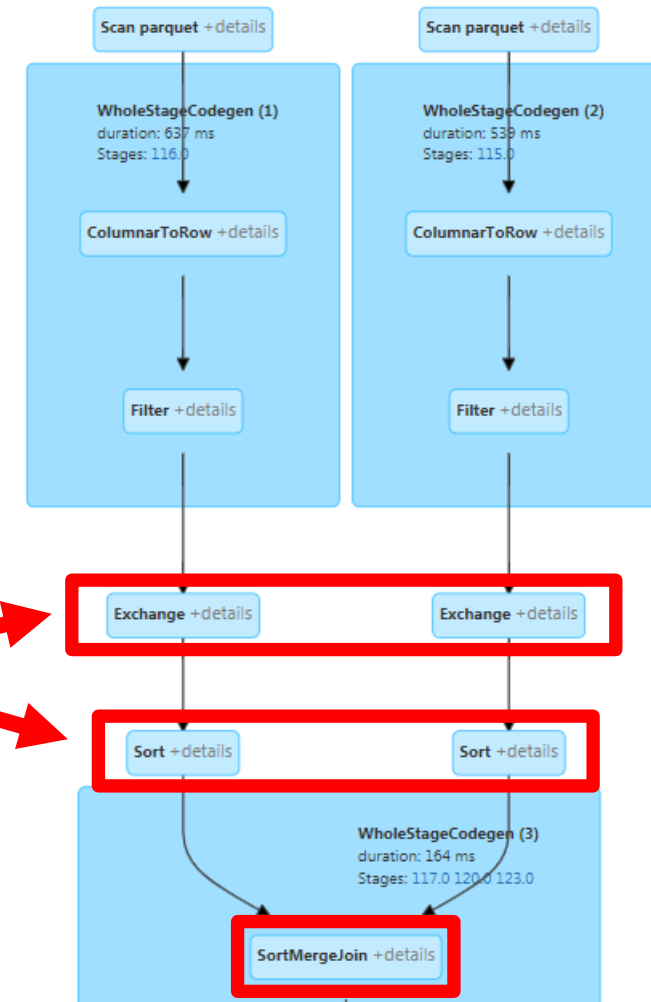
```
empDF.join(deptDF, "dept").select("last_name", "dept", "dept_name").limit(4).show()
```

► (3) Spark Jobs

```
+-----+-----+
|last_name|dept|dept_name|
+-----+-----+
| Ratzlaff| 501|marketing sales|
|  Runyon| 501|marketing sales|
|  Wilson| 501|marketing sales|
|  Rabbit| 501|marketing sales|
+-----+-----+
```

Notice the **Sort** and the **Exchange** (Shuffle)
These are both time-intensive operations

```
Command took 1.64 seconds
```



BROADCAST HASH JOIN



I enable BroadcastHashJoin and AQE and re-Run query

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 10485760)
spark.conf.set("spark.sql.adaptive.enabled", True)

display(empDF.join(deptDF, "dept").select("last_name", "dept", "dept_name"))
```

There is no **Sort** and no **Exchange** (Shuffle).

There is a **BroadcastExchange** where smaller table is sent to each Executor

SortMerge

```
Command took 1.64 seconds
```

BroadcastHash

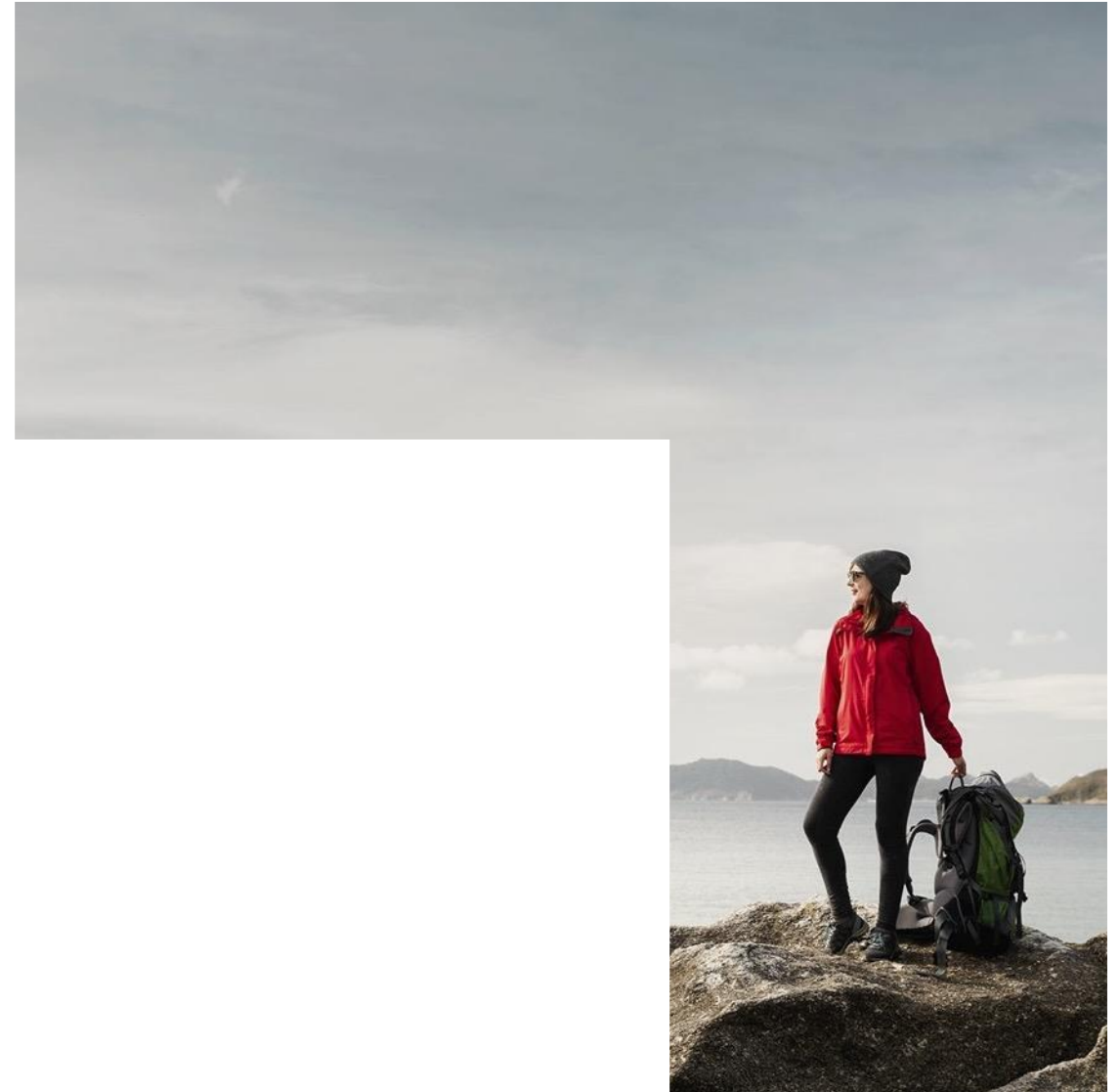
```
Command took 0.89 seconds
```

```
(16) AdaptiveSparkPlan
Output [3]: [last_name#
Arguments: isFinalPlan=
```



The Three S: Skew, Spill, and Shuffle

Let's see the big 3 Spark problems and how to solve them

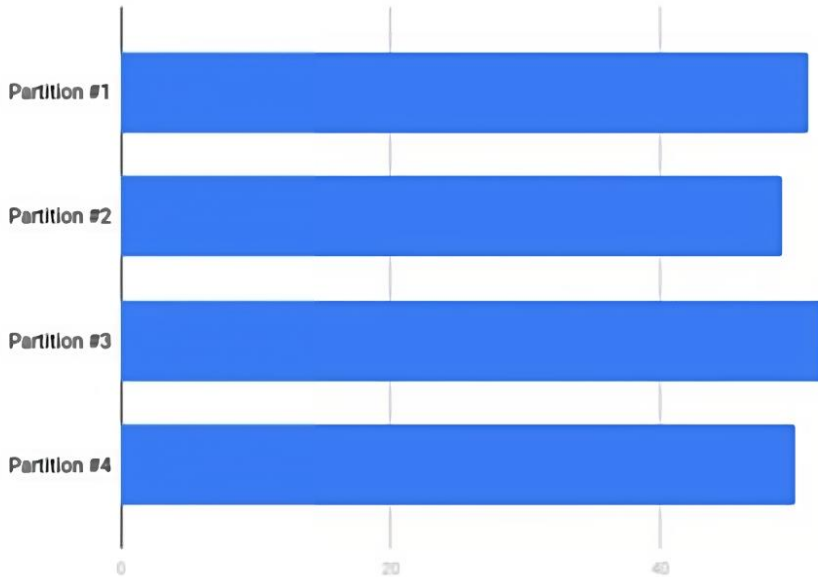


SKEW

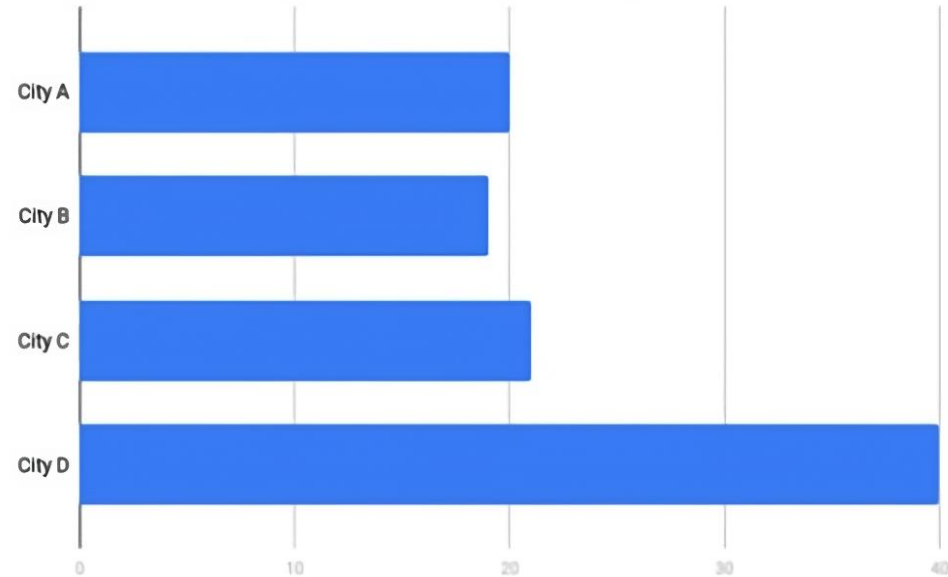
Skew - Before & After



Before aggregation



After aggregation by city



As the data is transformed (aggregated), it is possible to have significantly more records in one partition than in another. This significantly uneven distribution of records in partitions is called Skew.

SOLVING SKEW

The simplest solution today is to enable Adaptive Query Execution: which automatically does the reoptimizing and adjusting of query plans based on runtime statistics collected in the process of query execution.

AQE can be enabled by setting the following in a notebook or advanced parameters of a cluster . Only available in Spark 3.

```
spark.conf.set("spark.sql.adaptive.enabled", true)
spark.conf.set("spark.sql.adaptive.skewedJoin.enabled", true)
spark.conf.set("spark.sql.adaptive.localShuffleReader.enabled", true)
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", true)
```

SPILL

Search:

Time	Total Tasks	Failed Tasks	Killed Tasks	Succeeded Tasks	Blacklisted	Shuffle Read Size / Records	Spill (Memory)	Spill (Disk)
min	401	0	0	401	false	73.8 GiB / 1339865622	55.9 GiB	26 GiB
min	424	0	0	424	false	74.3 GiB / 1348549767	52.1 GiB	24.2 GiB

For every task there is a corresponding partition.

If that task cannot process that partition (due to memory allocated), the partition is spilled to disk or written to disk and read back again.

SOLVING SPILL

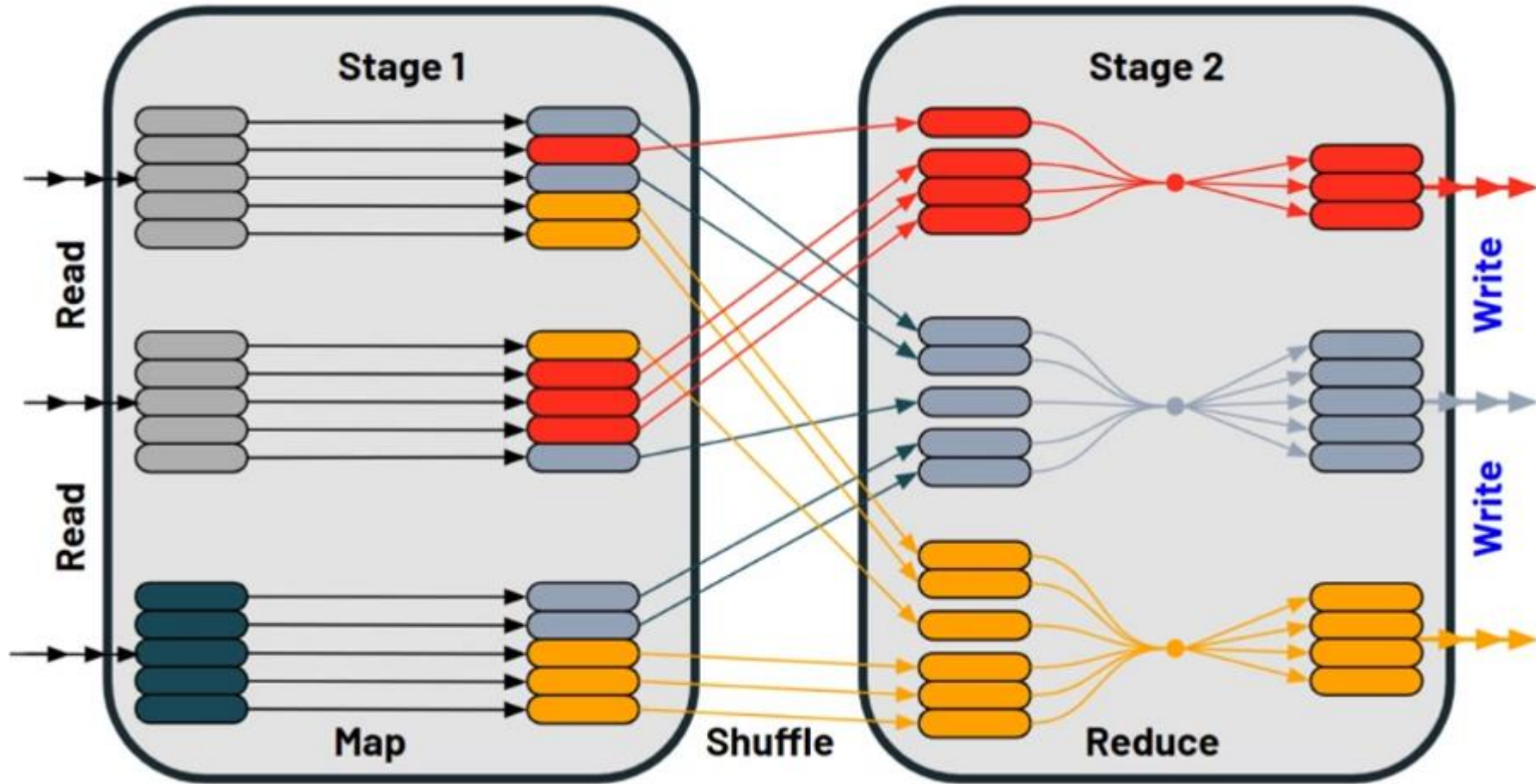
Time	Total Tasks	Failed Tasks	Killed Tasks	Succeeded Tasks	Blacklisted	Shuffle Read Size / Records	Spill (Memory)	Spill (Disk)
min	401	0	0	401	false	73.8 GiB / 1339865622	55.9 GiB	26 GiB
min	424	0	0	424	false	74.3 GiB / 1348549767	52.1 GiB	24.2 GiB

In case of Spill caused by skew address the skew and the Spill will be addressed.

→ Decrease the size of each partition by increasing the number of partitions

- By managing `spark.sql.shuffle.partitions`
- By explicitly repartitioning
- By managing `spark.sql.files.maxPartitionBytes`

SHUFFLE

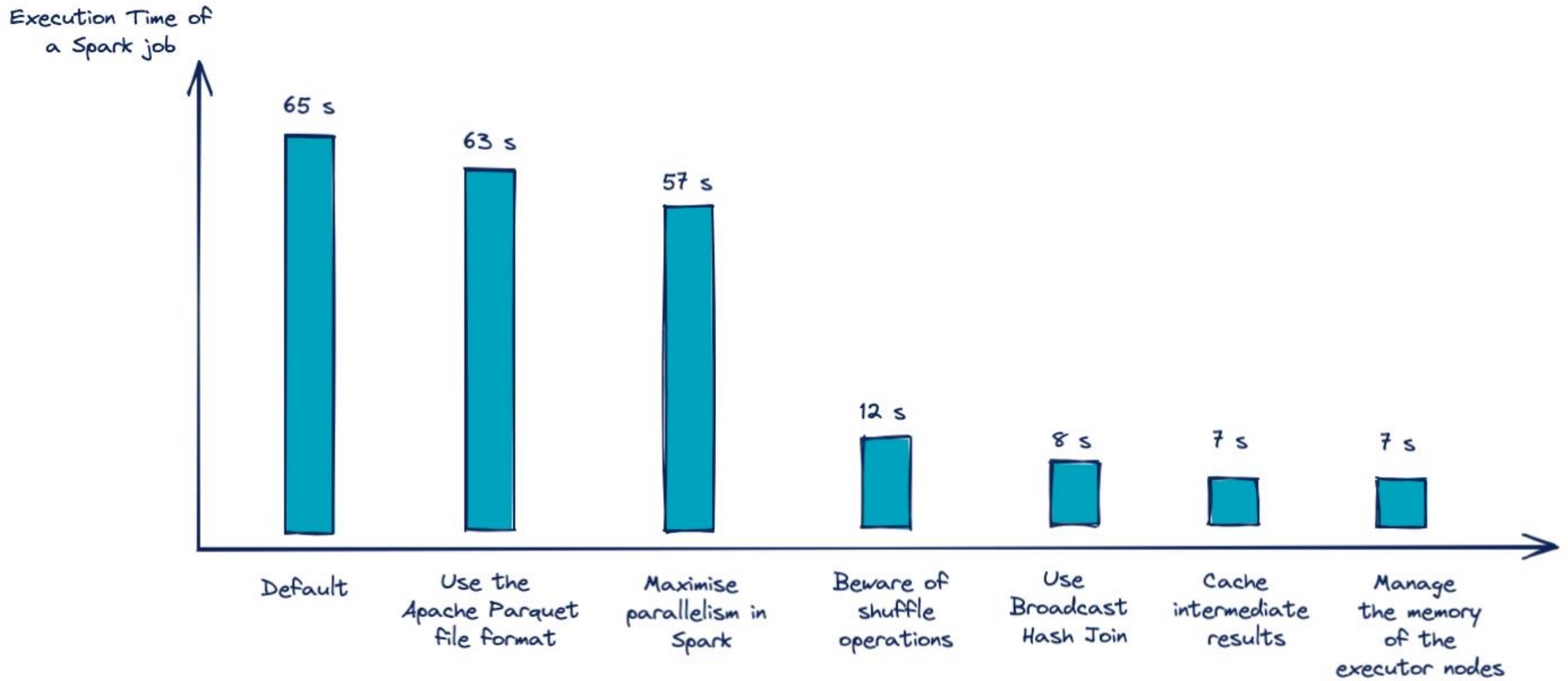


- This requires expensive disk and network IO.

SOLVING SHUFFLE

- 1.Reduce network IO by using fewer and larger workers.
- 2.Reduce the amount of data being shuffled : get rid of the columns that you don't need, filter out unnecessary records, optimize data ingestion.
- 3.De-normalize the datasets specifically if the shuffle is caused by a join.

REDUCING THE EXECUTION TIME



Source: <https://towardsdatascience.com/6-recommendations-for-optimizing-a-spark-job-5899ec269b4b>

TIPS : PERSIST DATA

In this example we read the order file and put it in the orders dataframe

```
order_file_name = "/mnt/adls/" + "40_project/test/" + "_Customer_order_concatenation.parquet"  
orders = spark.read.parquet(order_file_name)
```

Here we declare that at the first action the dataframe will be put in memory

```
orders.persist(StorageLevel.MEMORY_AND_DISK)
```

Here we concretely put it in memory by doing this action which is a No-operation action

```
orders.write.mode("overwrite").format("noop").save()
```

Action can also be a count().

```
orders.count()
```


TIPS : PERSIST DATA

Only persist datasets you will use repeatedly throughout the script.

Example. You call a dimension table you will use three to four times during the script to do jointures. It is better to persist this dataframe as soon as you have read the file to improve performance.

Be careful: persisting large dataframes like orders tables can pose problems (overload the memory).

Pyspark

```
Df.persist(StorageLevel.MEMORY_AND_DISK)
```

Spark SQL

```
Spark.sql('cache table1')
```

TIPS : UNPERSIST DATA

An important point is to free the memory in the notebook once you have finished to use the dataframe :

```
orders.unpersist()
```

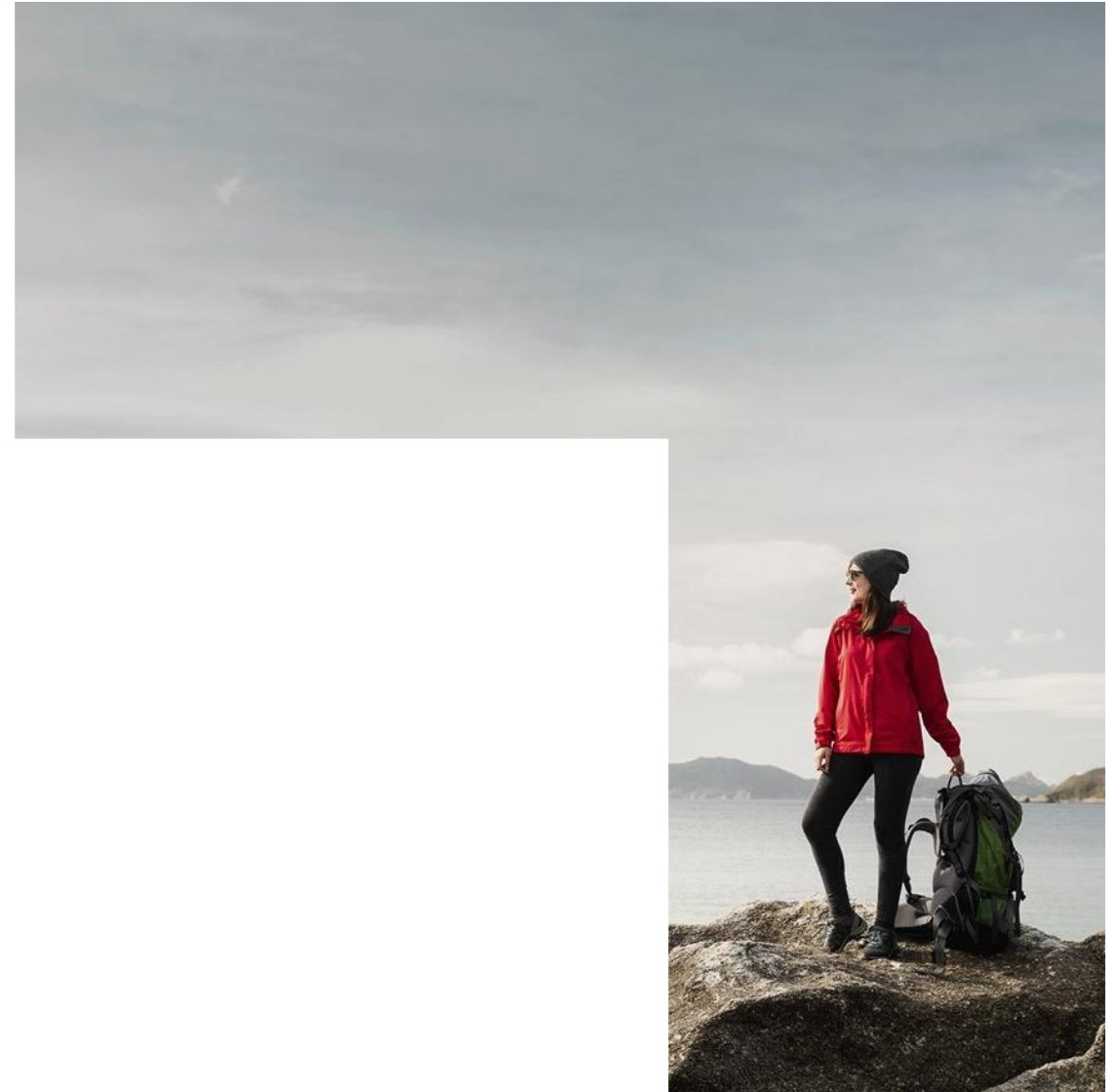
you can decide to free all the memory used with this command :

```
spark.catalog.clearCache()
```

It's like a unpersist for all dataframes

Exercise

4.2 Partitioning



CHAPTER OBJECTIVE REVIEW

- Understand what is a DAG in Spark
- Understand how to access and read the Spark UI
- Understand and define the most common Spark problems: Skew, Spill and Shuffle.
- Look at how to optimize performance

Pandas and PySpark Pandas



OBJECTIVES

- When to use 'Pandas' on Databricks
- Know how to import PySpark Pandas on Databricks
- Understand Pandas UDFs
- Know the limitations of PySpark Pandas

PANDAS

- **Pandas** is a fast, powerful, flexible and easy to use open-source data analysis and manipulation tool, built on top of the Python programming language.
- **Pandas** is mainly used for **data analysis**. Pandas allows importing data from various file formats such as comma-separated values, JSON, SQL database tables or queries, and Microsoft Excel.
- **Pandas** is built on top of two core Python libraries—matplotlib for data visualization and NumPy for mathematical operations. Pandas acts as a wrapper over these libraries



WHEN TO USE PANDAS ?

- For small datasets (less than 1 GB) , we can use pandas effectively.
- In Databricks, recommended to use pandas for :
 - 1) Reading excel files (easier in Pandas than in Spark)
 - 2) Pandas functions are useful for machine learning, such as one-hot encoding using the `pd.get_dummies` and is simpler to read and to code than the SparkML equivalent.

READING EXCEL WITH PANDAS

#install library

- `dbutils.library.installPyPI("xlrd",version="1.2.0")`
- `dbutils.library.restartPython()`

#Read file in pandas

- `PIPO = pd.read_excel("/dbfs" + PIPO_DIR_PATH + PIPO_FILE_NAME)`
- #Convert to spark dataframe
- `PIPO = spark.createDataFrame(PIPO, schema=None)`

SPEED UP WITH ARROW

```
from pyspark.sql.functions import rand
df = spark.range(1 << 22).toDF("id").withColumn("x", rand())
df.printSchema()
```

```
root
 |-- id: long (nullable = false)
 |-- x: double (nullable = false)
```

```
%time pdf = df.toPandas()
```

```
CPU times: user 10.6 s, sys: 714 ms, total: 11.3 s
Wall time: 16.1 s
```

```
spark.conf.set("spark.sql.execution.arrow.enabled", "true")
```

```
%time pdf = df.toPandas()
```

```
CPU times: user 102 ms, sys: 134 ms, total: 236 ms
Wall time: 1.04 s
```

```
pdf.describe()
```

	id	x
count	4.194304e+06	4.194304e+06

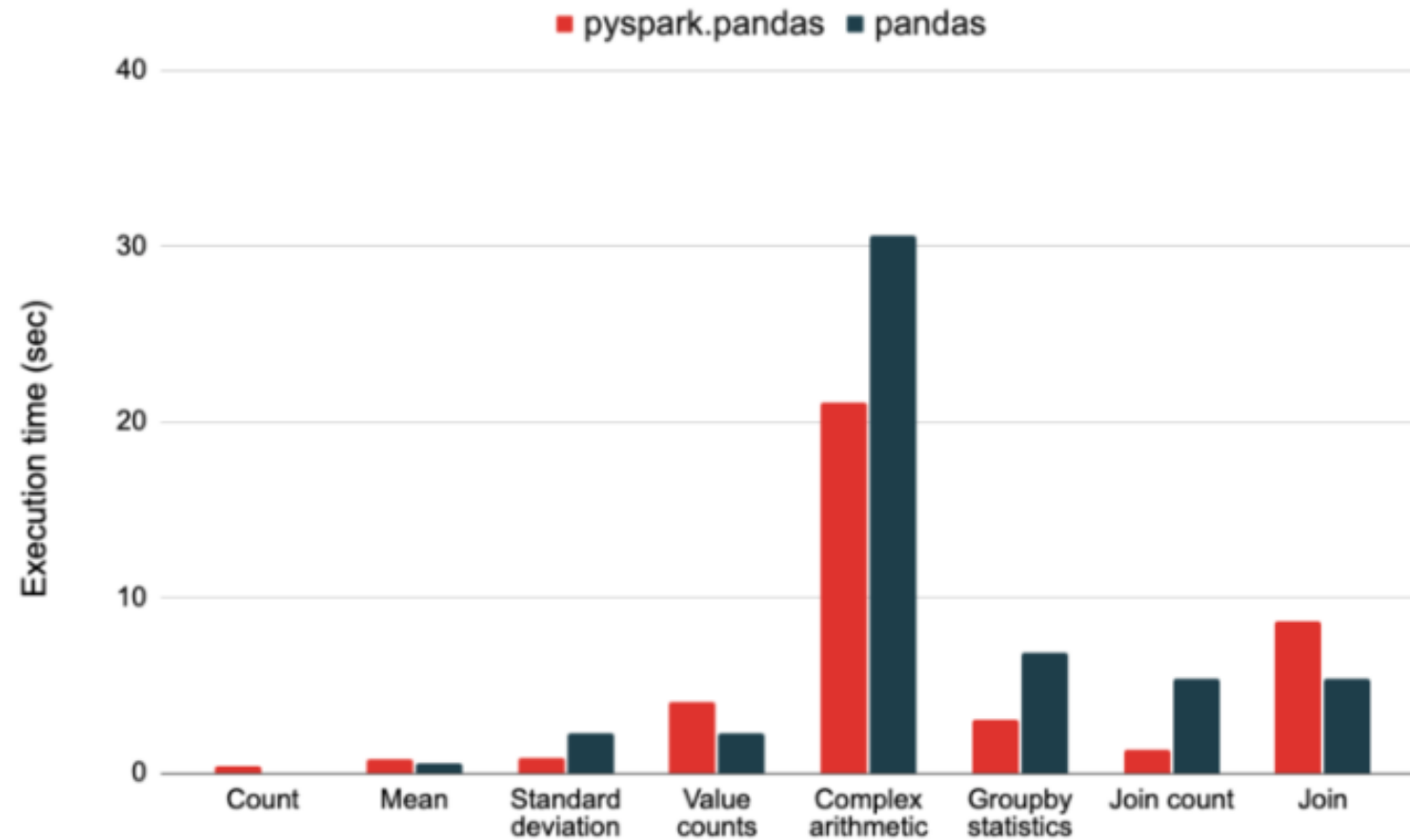
SPARK TO PANDAS ON SPARK

```
>>> %pip install pyspark.pandas

>>> import pandas as pd
>>> import pyspark.pandas as ps
>>> pdf = pd.DataFrame({"a": [1, 3, 5]}) # pandas DataFrame
>>> sdf = spark.createDataFrame(pdf) # PySpark DataFrame
>>> psdf = sdf.to_pandas_on_spark() # pandas-on-Spark DataFrame
>>> # Query via SQL
... ps.sql("SELECT count(*) as num FROM {psdf}")
```

Performance Comparison - 31 GB

Lower is better



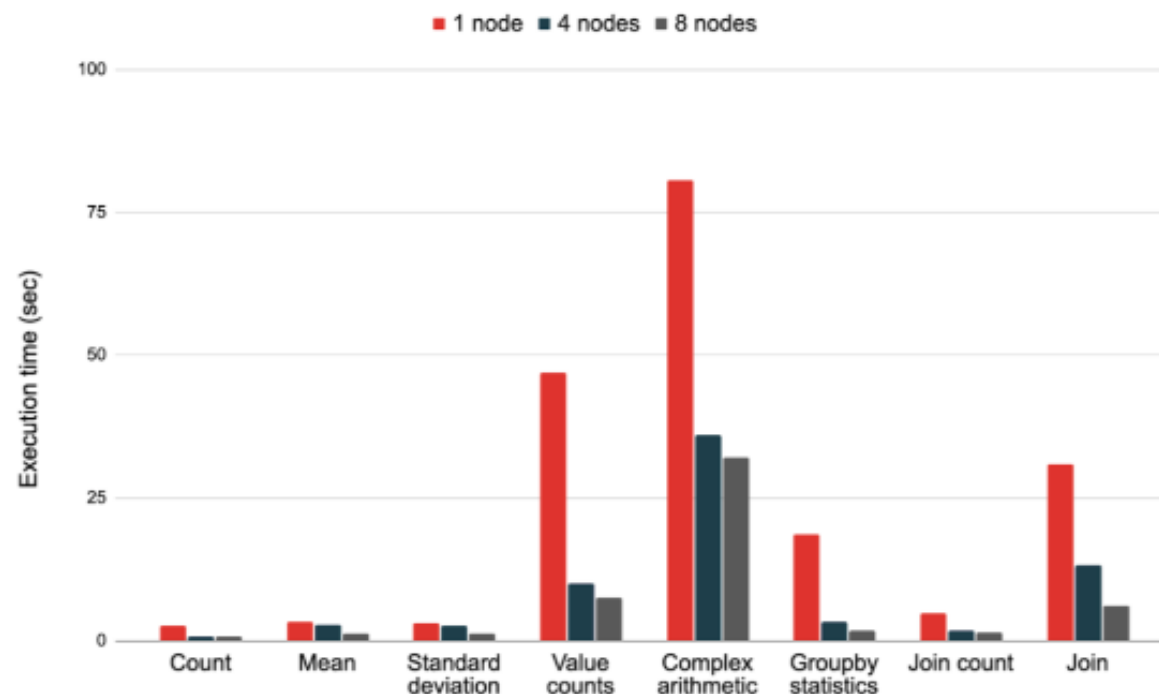
Performance Comparison - 95 GB

Lower is better

pandas

pyspark.pandas

💡 **ConnectException error:** This is often caused by an OOM error that causes the connection to the Python REPL to be closed. Check your query's memory usage. [Spark tip settings](#)



ADVANTAGES OF PANDAS ON SPARK

- It also supports string interpolation syntax to interact with Python objects naturally:

```
>>> pred = range(4)
>>> # String interpolation with Python instances
... ps.sql("SELECT * from {psdf} WHERE a IN {pred}")
```

- Users can easily call the scalable machine learning libraries in Spark:

```
>>> from pyspark.ml.feature import StringIndexer
>>> sdf = psdf.to_spark() # PySpark DataFrame
>>> indexer = StringIndexer(
...     inputCol="category", outputCol="categoryIndex")
>>> indexed = indexer.fit(sdf).transform(sdf)
>>> indexed.show()
```

DIFFERENCES TO KNOW

Firstly we must know that the class changes : `pandas.core.frame.DataFrame` for pure Pandas and `pyspark.pandas.frame.DataFrame` for Pandas on Spark.

1.You cannot work with a pandas and a pandas-on-spark dataframe together. So `toPandas()` must be replaced with `to_pandas_on_spark()`

2.The `concat` and `merge` function in Pandas on Spark does not allow lists, but only a `ps.DataFrame` or `ps.Series`. So you have to make a list into a Dataframe using `ps.DataFrame(list)` before trying to use `ps.concat` or `ps.merge`.

3.When reading excel files on Pandas on Spark, we must precise the data type, other `read_excel` function does not work. For example;

```
df_mag_reference = spd.read_excel(MAG_REFERENTIAL_FILE_PATH, dtype=str)
```

4.Normally in Pandas dataframe, we can make a list directly from a DataFrame column. But in Pandas on Spark, we are obliged to add `to_numpy()` before making this treatment.

TIPS ON CONVERSION

We can convert pandas-on-spark dataframe to pandas using `toPandas()` and to Spark using `to_spark()`.

When pandas-on-Spark Dataframe is converted from Spark DataFrame, it loses the index information, which results in using the default index in pandas API on Spark DataFrame. The default index is inefficient in general comparing to explicitly specifying the index column. Specify the index column whenever possible.

Time for a break



CGI

CHAPTER REVIEW OBJECTIVES

- Understand the concept 'Pandas' and 'Koalas'
- Know how to import Pandas and Koalas on Databricks
- Understand when Pandas and Koalas can be an alternative to PySpark .

Advanced Query with Pyspark



QUICK REMINDER

Transformations <i>(lazy)</i>	Actions
<code>select</code>	<code>show</code>
<code>distinct</code>	<code>count</code>
<code>groupBy</code>	<code>collect</code>
<code>sum</code>	<code>save</code>
<code>orderBy</code>	
<code>filter</code>	
<code>limit</code>	

WINDOW FUNCTION

```
from pyspark.sql.window import Window
```

```
from pyspark.sql.functions as F
```

```
wind = Window.partitionBy("department").orderBy("sales")
```

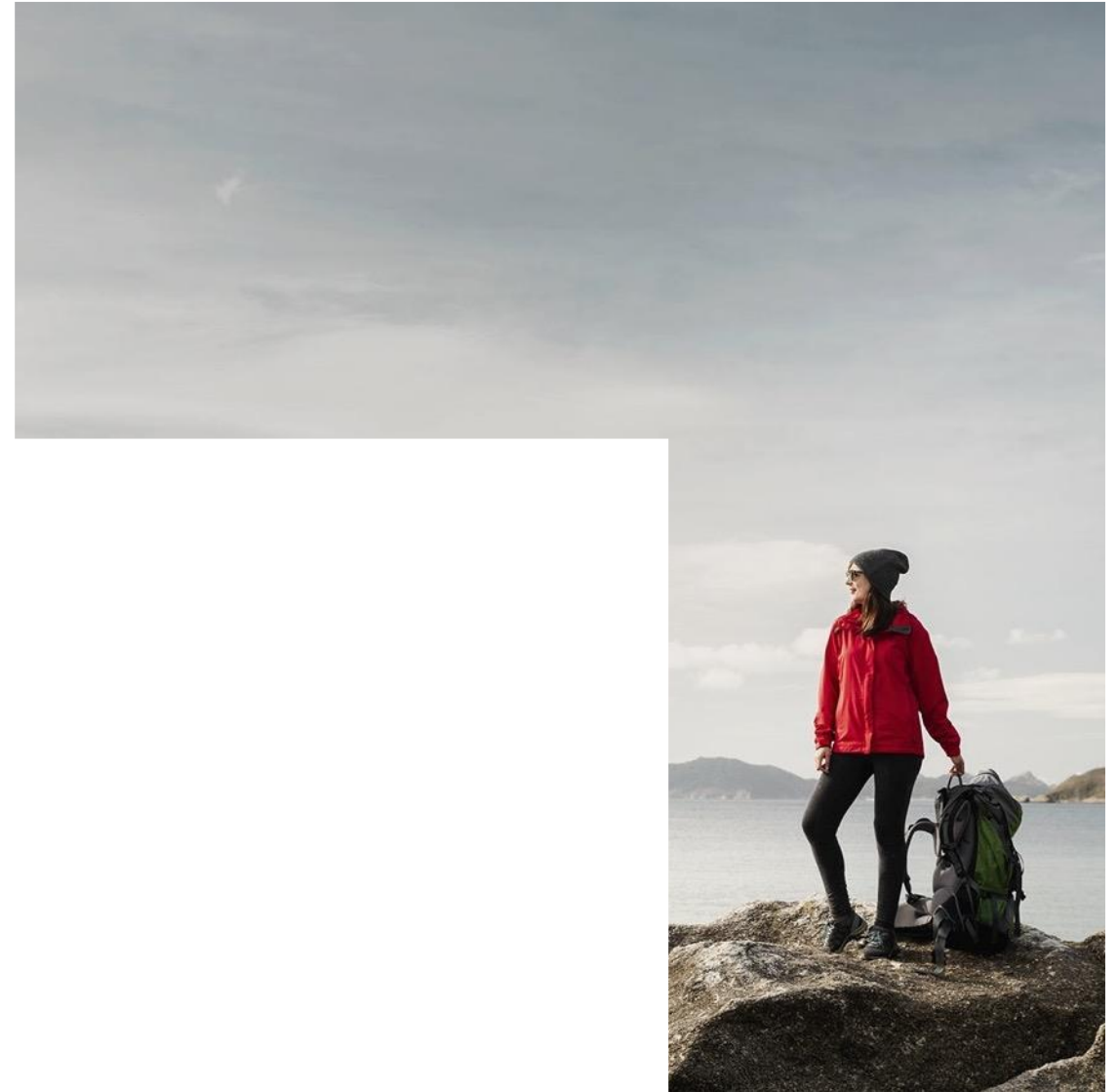
```
df = df.withColumn("cumulative_sales", F.sum("sales").over(wind))
```

PySpark Window functions operate on a group of rows and return a single value for every input row. PySpark SQL supports three kinds of window functions: **Ranking** (row_number, rank),

Aggregations (min, max, sum, avg) and **Analytics** (lag, lead)

Let's practice

2.5 Window Functions



USER DEFINED FUNCTION (UDF)

```
from pyspark.sql.functions as F
```

```
def upperCase(str):
```

```
    return str.upper()
```

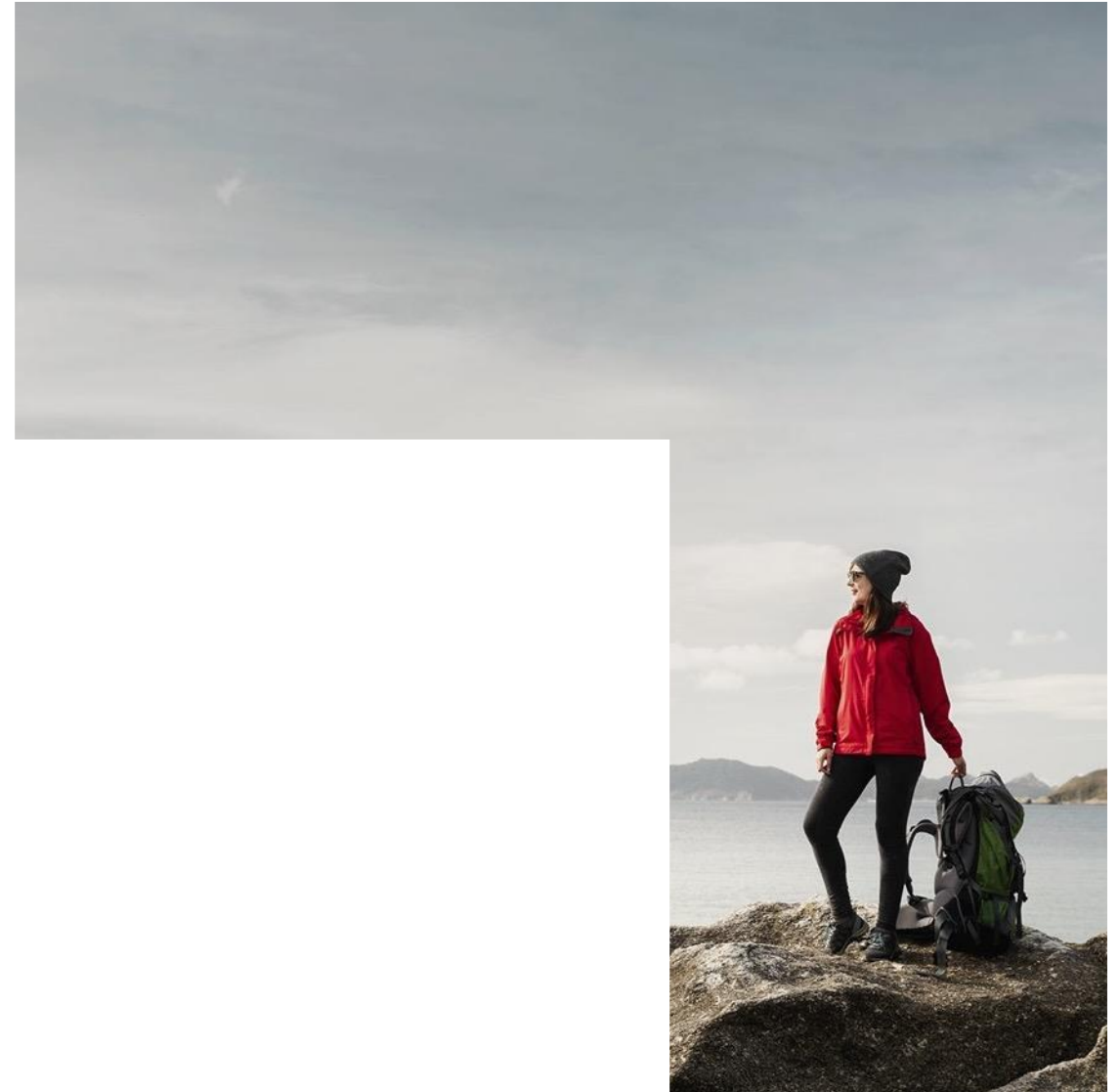
```
upperCaseUDF = F.udf(lambda z:upperCase(z), StringType())
```

```
df = df.withColumn("Created Name", upperCaseUDF(col("Name")))
```

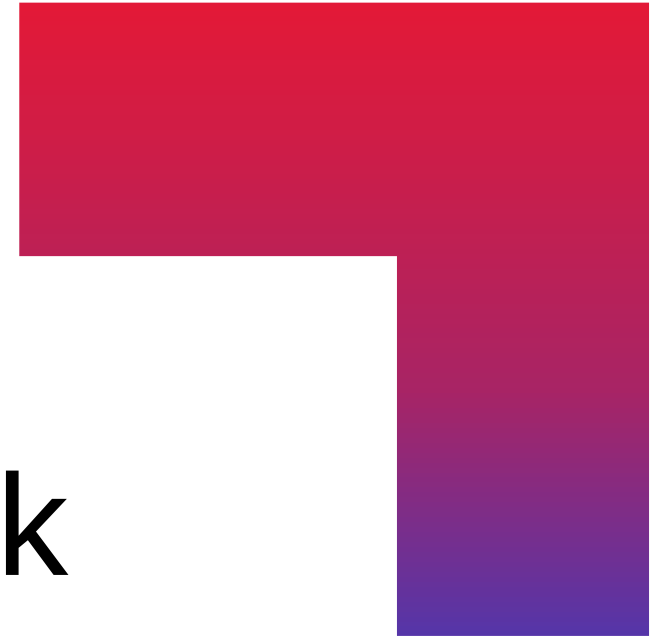
PySpark UDF's are similar to UDF on traditional databases. In PySpark, you create a function in a Python syntax and wrap it with PySpark SQL `udf()`, all you need is a **python function** and the **return type** of your function.

Let's practice

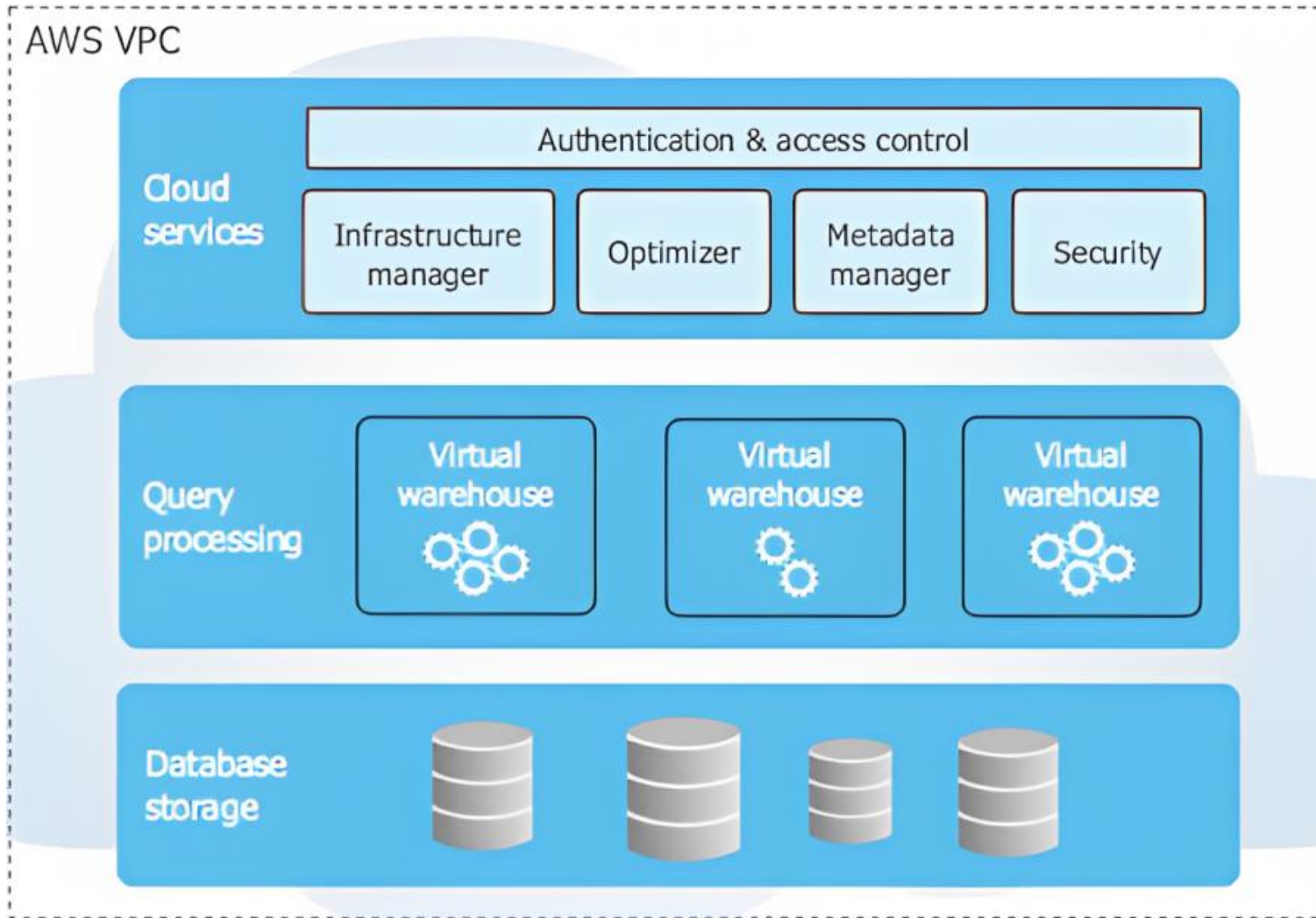
2.6 UDFs



Snowflake and Snowpark



SNOWFLAKE ARCHITECTURE



1.Database Storage

File system in Snowflake is backed by AWS S3 in Snowflake's account, all data is encrypted, compressed, and distributed to optimize performance

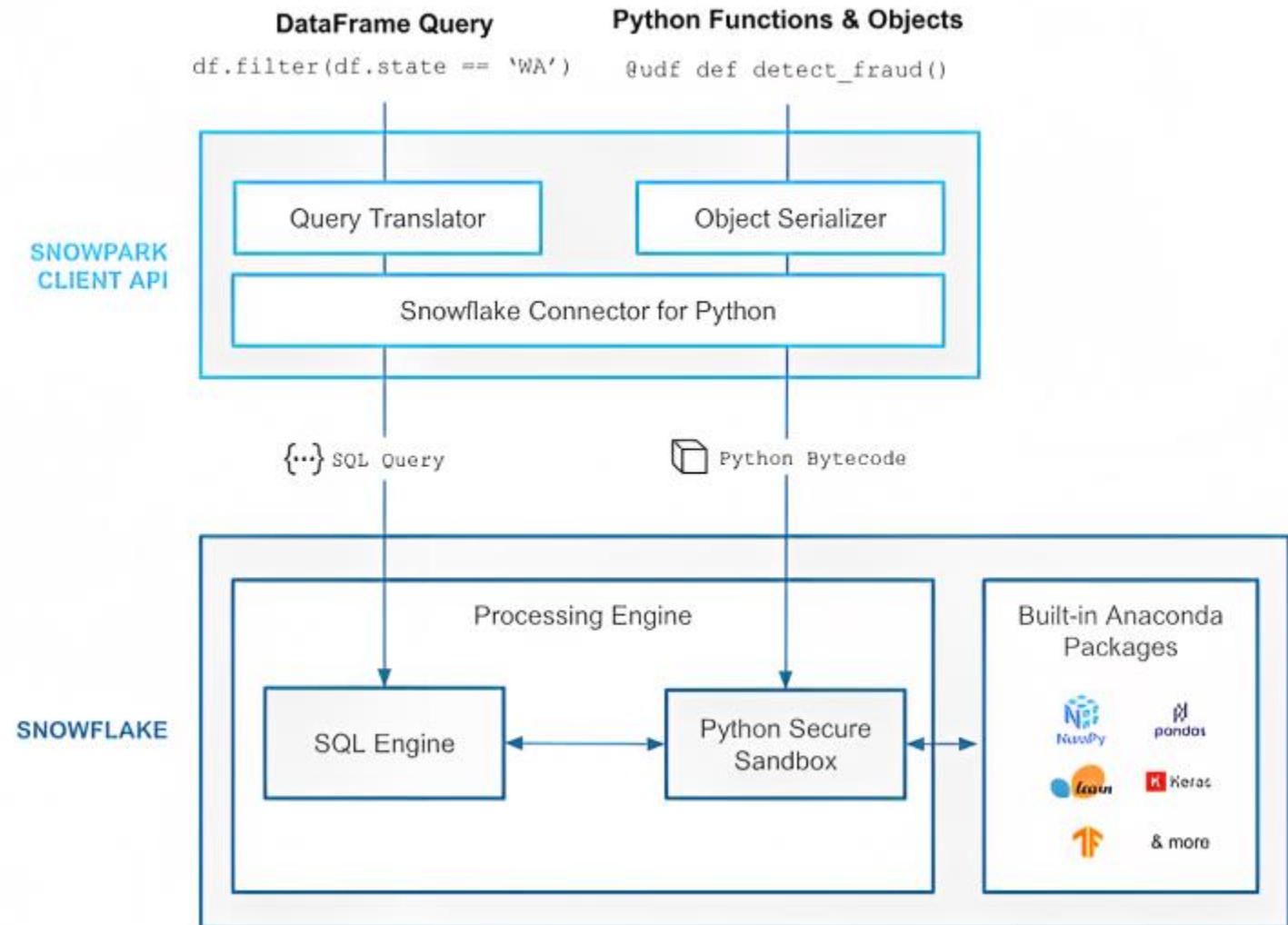
2.Query Processing

Snowflake provides the ability to create "Virtual Warehouses". Virtual Warehouses can be used to load data or run queries and can do both tasks concurrently.

3.Cloud Services

Coordinates and handles all other services in Snowflake including sessions, authentication, SQL compilation, encryption, etc.

SNOWPARK FOR PYTHON



Initialising Spark Session

Spark

```
from pyspark.sql import SparkSession

spark = SparkSession \
    .builder \
    .appName("Python Spark SQL basic example") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()
```

`pip install snowflake-snowpark-python`

Snowpark

```
from snowflake.snowpark.session import Session

CONNECTION_PARAMETERS = {
    'account': snowflake_account,
    'user': snowflake_user,
    'password': user_password,
    'schema': snowflake_schema,
    'database': snowflake_database,
    'warehouse': snowflake_warehouse,
    'role': snowflake_user_role,
}

session = Session.builder.configs(CONNECTION_PARAMETERS).create()
```

SLIGHT DIFFERENCES IN UDFs

```
# "PySpark Code Sample"
<!-- PySpark Code Sample -->

from pyspark.sql.types import StructType, StructField, StringType, IntegerType
spark.udf.register("toUpper",
    lambda s: s.upper() if s is not None else 'NA', returnType=StringType())

# Using UDF in SQL Query
spark.sql("select EmployeeName, toUpper(EmployeeName) as EmpNameCaps \
    from emp where EmployeeName is not null").show()
```

```
# "Snowpark Python Code Sample"
<!-- Snowpark Python Code Sample -->

from snowflake.snowpark.types import StringType, StructField, StructType, IntegerType, DoubleType, FloatType, TimestampType
session.udf.register(
    lambda s: s.upper() if s is not None else 'NA', return_type=StringType(),
    input_types=[StringType()],
    name="toUpper", replace=True)

# Using UDF in SQL Query
session.sql("select *, toUpper(EMPLOYEE_NAME) as UpperEmpName from emp ").toPandas()
```



Limitations and Solutions



OBJECTIVES

- Understand the limitations of Databricks and potential solutions to counter these

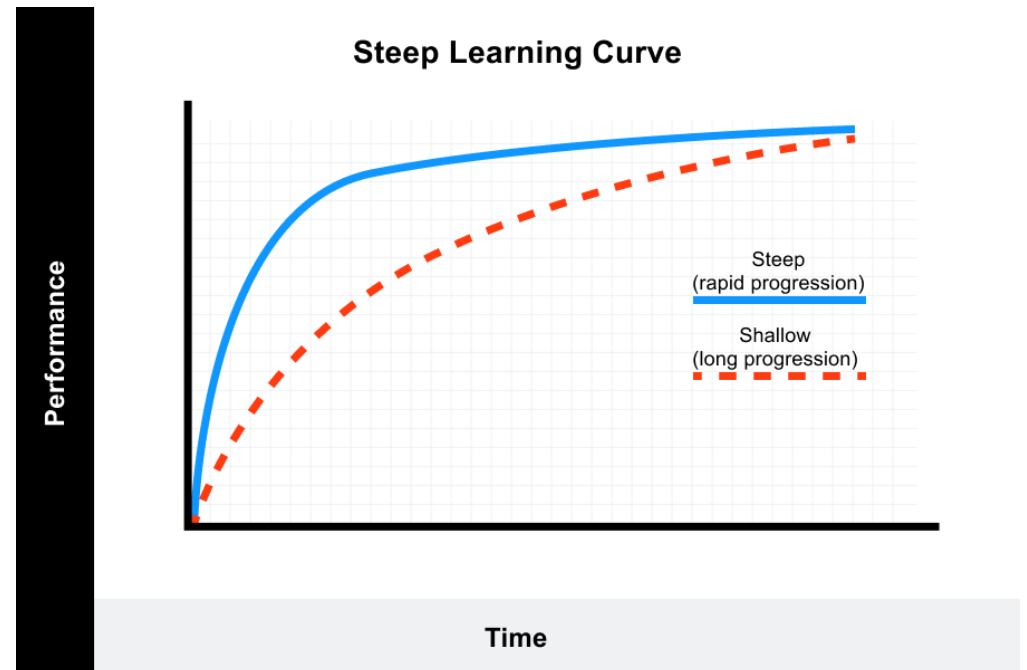
PROBLEM : STEEP LEARNING CURVE

- Spark is complex ! To work with Big Data or to improve performance, requires specific knowledge of how to deal with spill, shuffles, skew etc. This knowledge often takes months or years of practice to obtain.

```
6 // Enable AQE features to automatically coalesce partitions
7 spark.conf.set("spark.sql.adaptive.enabled", true)
8 spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", true)
9
10 spark
11   .read.format("delta").load(sourceDir)
12   .groupBy($"city_id").count
13   .write.format("noop").mode("overwrite").save()
```

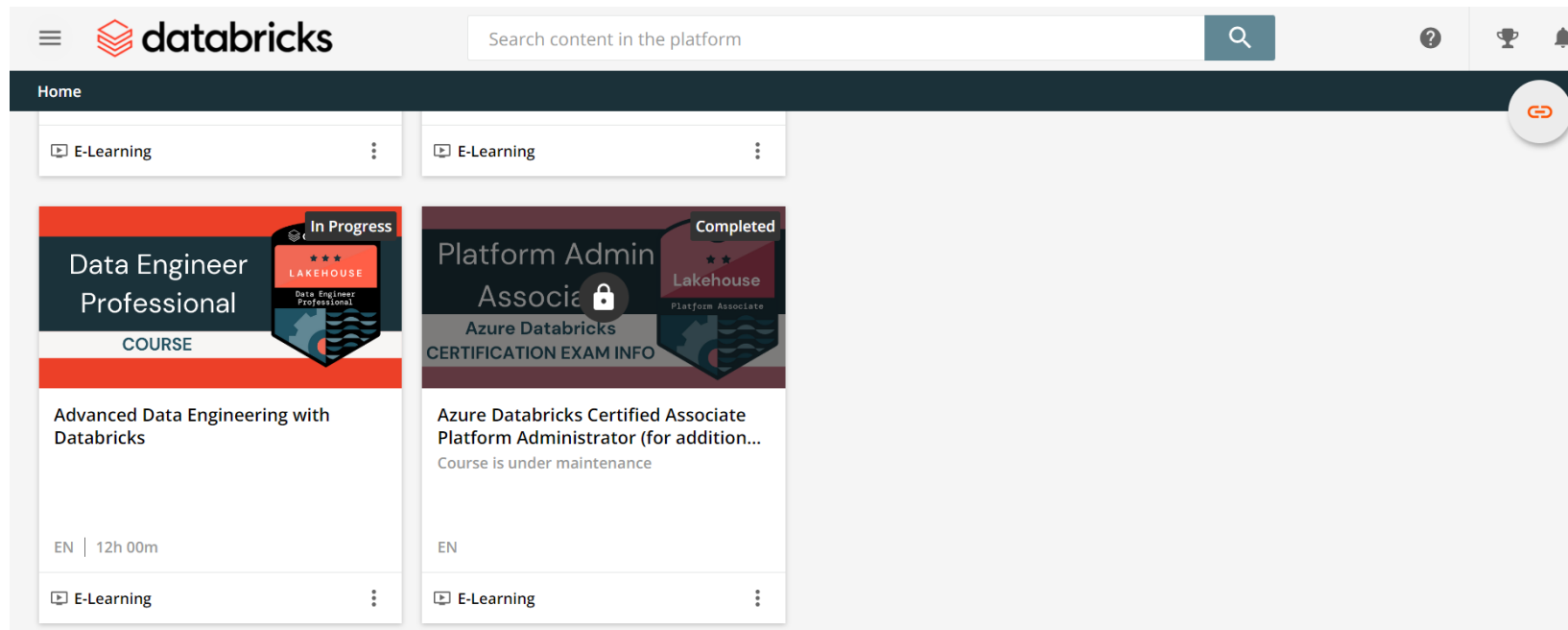
▼ (3) Spark Jobs

- ▼ Job 5 [View](#) (Stages: 1/1, 1 skipped)
 - Stage 11: 0/1 [i](#) skipped
 - Stage 12: 1/1 [i](#)
- ▼ Job 6 [View](#) (Stages: 1/1)
 - Stage 13: 825/825 [i](#)
- ▼ Job 7 [View](#) (Stages: 1/1, 1 skipped)
 - Stage 14: 0/825 [i](#) skipped
 - Stage 15: 17/17 [i](#)



SOLUTION : USE DATABRICKS ACADEMY

- <https://databricks.com/learn/training/login>
- Dedicated portal for learning for customers, partners and Microsoft employees.



NO NATIVE SUPPORT FOR GEO OBJECTS

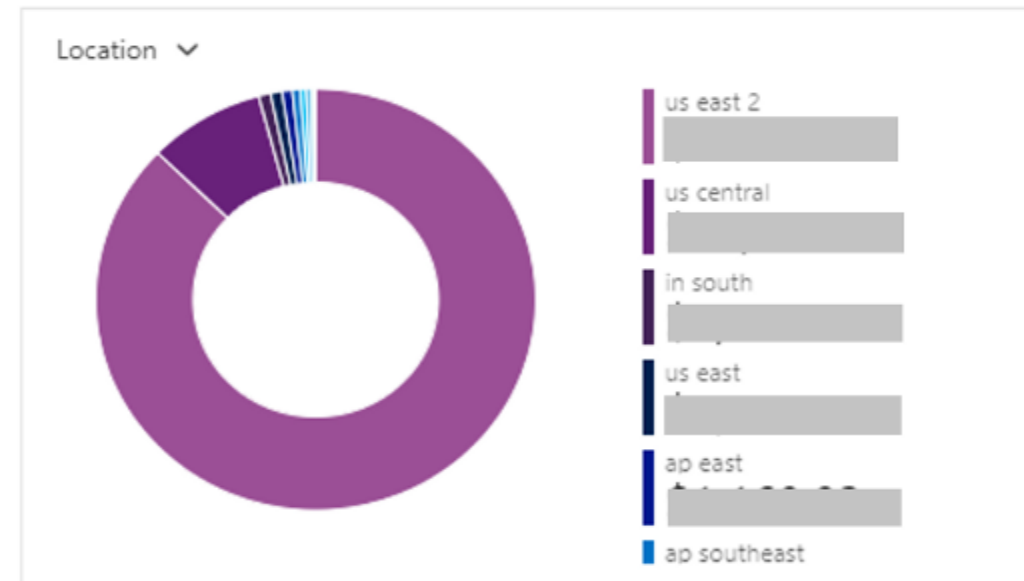
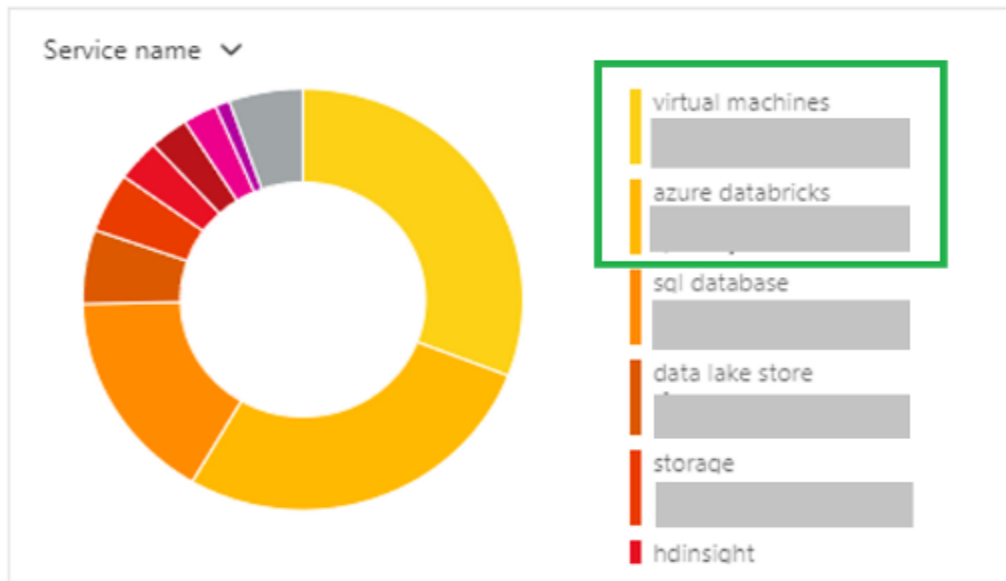
- Databricks has no native support for geographical objects like polygons. Requires additional library support and requires to pass through pandas, for using libraries like GeoPandas or GeoSpark.

```
import pandas as pd
from geopandas import GeoDataFrame # Loading boundaries Data
from shapely.geometry import Point, Polygon, shape # creating geospatial data
from shapely import wkb, wkt # creating and parsing geospatial data
from ast import literal_eval as make_tuple # used to decode data from java
```

```
# Create SparkSession and attach Sparkcontext to it
spark = SparkSession.builder.appName("pyspark-geopandas").getOrCreate()
sc = spark.sparkContext
```

COST

- Databricks is expensive !!! There is a cost for Databricks Units and for running virtual machines for every hour the cluster is on.
- Tip: Stop your clusters when you are sure you don't need them anymore. Or set a shorter time for auto-shutdown.



COST

- A typical premium workspace running a general-purpose cluster with 1 driver on a good virtual machine for data analytics workloads can cost upto 1000 dollars quite easily.

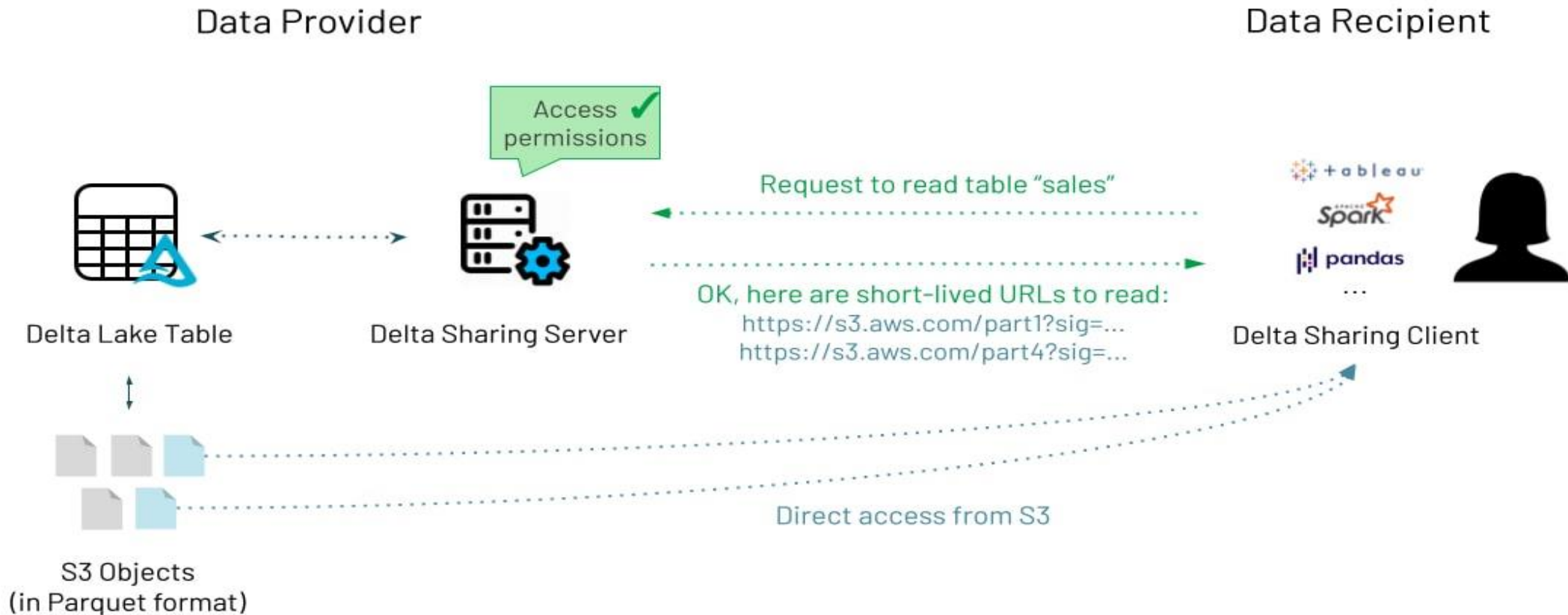
Pricing Tier	Workload Type	Numbers	DBU#	DBU Price/hour*	Linux VM Cost/hour	Hours to Run / Day	Days to Run	DBU Cost	VM Cost	Total Cost / month
Standard	Data Engineering	1	1.5	\$0.150	\$0.458	24	30	\$162.00	\$329.76	\$491.76
Premium	Data Engineering	1	1.5	\$0.300	\$0.458	24	30	\$324.00	\$329.76	\$653.76
Standard	Data Analytics	1	1.5	\$0.400	\$0.458	24	30	\$432.00	\$329.76	\$761.76
Premium	Data Analytics	1	1.5	\$0.550	\$0.458	24	30	\$594.00	\$329.76	\$923.76
Standard	Data Engineering Light	1	1.5	\$0.070	\$0.458	24	30	\$75.60	\$329.76	\$405.36
Premium	Data Engineering Light	1	1.5	\$0.220	\$0.458	24	30	\$237.60	\$329.76	\$567.36

Note: Hypothetical example , has no reference to any client

SHARING LIMITATIONS

- Cannot share Delta tables to people outside the workspace.
- Makes it difficult to share tables to key business users directly.
- Process is currently on for changing this, but non-partners of Databricks cannot connect.

DELTA SHARING: A POTENTIAL SOLUTION IN THE FUTURE



Source : <https://databricks.com/fr/blog/2021/05/26/introducing-delta-sharing-an-open-protocol-for-secure-data-sharing.html>

CHAPTER OBJECTIVE REVIEW

- Understand the limitations of Databricks

Certifications & Badges



USING DATABRICKS ACADEMY

- Go to: <https://www.databricks.com/learn/training/login>

And login on the Customer or Partner portal

Which of the following best describes you?

		
Customers and prospects	Partners	Microsoft
Existing customers of Databricks or those who want to learn about Databricks	Employees at system integrators and ISVs (independent software vendors)	Employees of Microsoft

USING DATABRICKS ACADEMY

- Choose from one of the 4 Major learning pathways



Data science and
engineering



Databricks Administrators



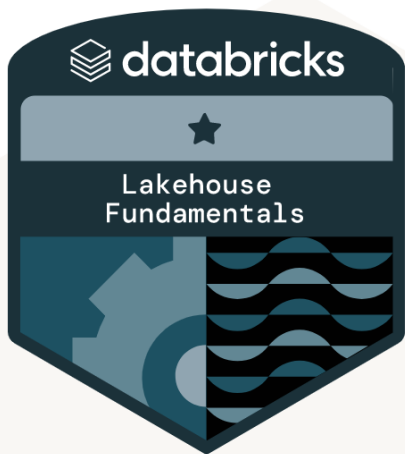
SQL analytics



Machine learning

DATABRICKS ACADEMY

- First take the Data Lakehouse Fundamentals (free of charge)
- Pre-requisite for all other learning paths and higher certifications



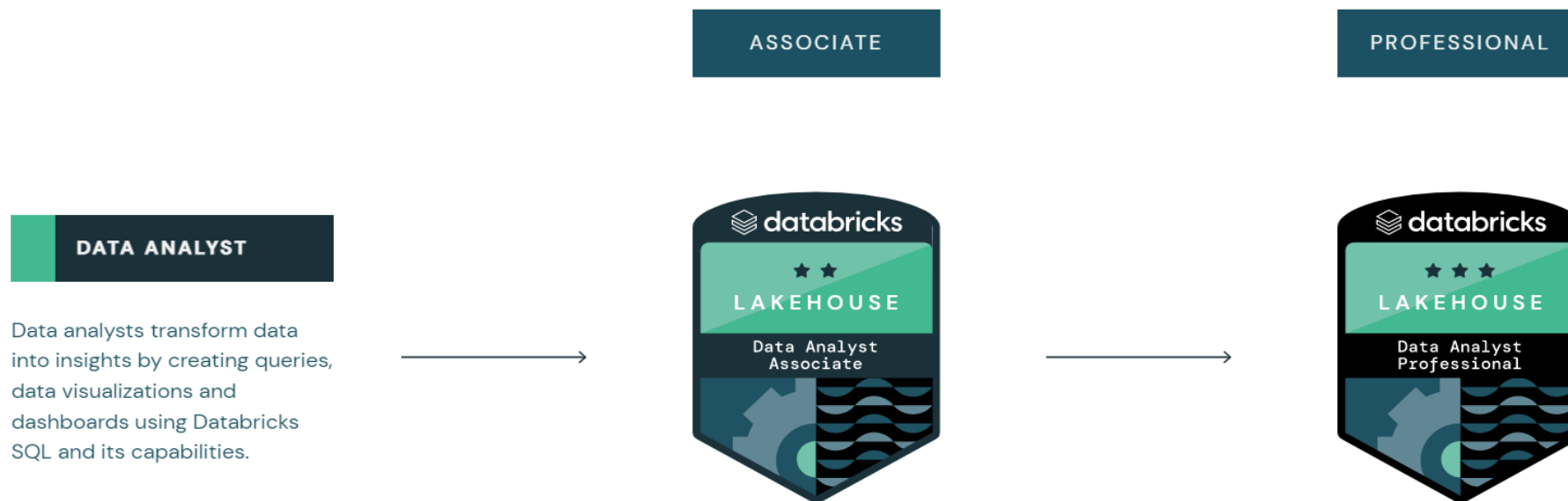
Uplevel your Lakehouse skills

The Databricks Lakehouse Platform has created a paradigm shift by unifying data and AI on a single, open platform. Learn the fundamentals of Lakehouse architecture through four fast-paced videos and earn an accreditation.

Take Lakehouse Fundamentals for free

<https://databricks.com/learn/training/home>

DATA ANALYST PATHWAY



There are 45 multiple-choice questions on the certification exam in 90 minutes.

- Databricks SQL – 22% (10/45)
- Data Management – 20% (9/45)
- SQL – 29% (13/45)
- Data Visualization and Dashboards – 18% (8/45)
- Analytics Applications – 11% (5/45)

DATA ENGINEER PATHWAY

DATA ENGINEER

Data engineers design, develop, test and maintain batch and streaming data pipelines using the Databricks Lakehouse Platform and its capabilities.



There are 45 multiple-choice questions on the certification exam in 90 minutes.

- Databricks Lakehouse Platform – 24% (11/45)
- ELT with Spark SQL and Python – 29% (13/45)
- Incremental Data Processing – 22% (10/45)
- Production Pipelines – 16% (7/45)
- Data Governance – 9% (4/45)

MACHINE LEARNING PATHWAY

ML DATA SCIENTIST

Machine learning practitioners develop, deploy, test and maintain machine learning models and pipelines using Databricks Machine Learning and its capabilities.



There are 45 multiple-choice questions on the certification exam in 90 minutes.

- Databricks Machine Learning – 29% (13/45)
- ML Workflows – 29% (13/45)
- Spark ML – 33% (15/45)
- Scaling ML Models – 9% (4/45)

SPARK DEVELOPER PATHWAY



Candidates have 120 minutes to answer 60 questions about Python and SQL.

- Apache Spark Architecture Concepts – 17% (10/60)
- Apache Spark Architecture Applications – 11% (7/60)
- Apache Spark DataFrame API Applications – 72% (43/60)

Questions:

Time: 15 minutes



CGI

Question 1: READING FILES

Which of the following code blocks reads a parquet at the file path filePath into a DataFrame?

- A. `spark.read().parquet(filePath)`
- B. `spark.read().path(filePath, source = "parquet")`
- C. `spark.read.path(filePath, source = "parquet")`
- D. `spark.read.parquet(filePath)`

Question 2: READING FILES

Which of the following code blocks reads a parquet at the file path filePath into a DataFrame?

- A. `spark.read().schema(schema).format(json).load(filePath)`
- B. `spark.read.schema(schema).format("json").load(filePath)`
- C. `spark.read().schema(schema).format("json").load(filePath)`
- D. `spark.read.schema("schema").format("json").load(filePath)`

Question 3: WRITING FILES

Which of the following code blocks writes DataFrame storesDF to file path filePath as parquet?

- A. `storesDF.write.parquet(filePath)`
- B. `storesDF.write.option("parquet").path(filePath)`
- C. `storesDF.write.path(filePath)`
- D. `storesDF.write().parquet(filePath)`

Question 4: WRITING FILES

```
storesDF.write.repartition("division").parquet(filePath)
```

Find the error in this write statement

- A. The argument should be wrapped in the `col()` function to return a Column object.
- B. There is no `parquet()` operation for `DataFrameWriter` — the `save()` operation should be used instead.
- C. There is no `repartition()` operation for `DataFrameWriter` — the `partitionBy()` operation should be used instead.
- D. `DataFrame.write` is an operation — it should be followed by parentheses to return a `DataFrameWriter`.

Question 5: Pyspark

Given a dataframe df, select the code that returns its number of rows:

A. `df.collect()`

B. `df.count()`

C. `df.show()`

D. `df.numRows()`

Question 6: Pyspark

I want to filter storesDF with all values in the “sqft” column less than 25000

Code block: `storesDF.filter(sqft < 25000)`

Identify the correct improvement to make the code work.

- A. It should be with col function: like `storesDF.filter(col("sqft") <= 25000)`.
- B. The column name sqft needs to be quoted like `storesDF.filter("sqft" <= 25000)`.
- C. The sign in the logical condition inside filter() needs to be changed from <= to >.
- D. The sign in the logical condition inside filter() needs to be changed from <= to >=.

Question 7: Pyspark

*I want to join df1 and df2 on ID with a left join. Which syntax is **correct**?*

- A. `df1.join(df2, on = "ID" , how = "left")`
- B. `df1.join(df2, df1.ID == df2.ID, how = "left")`
- C. `join(df1, df2, how = "left", on = "ID")`

Question 8: Pyspark

Which command will do an ordering of the column created_date in ascending order with null values last

- A. `df.orderBy(col("created_date").asc())`
- B. `df.order(asc_nulls_last("created_date"))`
- C. `df.orderBy(col("created_date").asc_nulls_last())`
- D. `df.orderBy(col("created_date"), ascending=True)`
- E. `df.order(col("created_date").asc())`

Question 9: Pyspark

Which of the following code blocks returns a DataFrame containing only column storeId and column division from DataFrame storesDF?

- A. `storesDF.select(col("storeId", "division"))`
- B. `storesDF.select(storeId).select(division)`
- C. `storesDF.select("storeId").select("division")`
- D. `storesDF.select("storeId", "division")`

Question 10: Spark SQL

What is the syntax to create a table ? My notebook is a python notebook on Databricks.

- A. %python Create Table X
- B. %sql Create Table X if not exists
- C. %python Use Table X
- D. %sql Drop Table X if not exists

Question 11: Spark SQL

Which of the following operations can execute a SQL query on a table?

- A. `spark.query()`
- B. `DataFrame.sql()`
- C. `spark.sql()`
- D. `DataFrame.createOrReplaceTempView()`

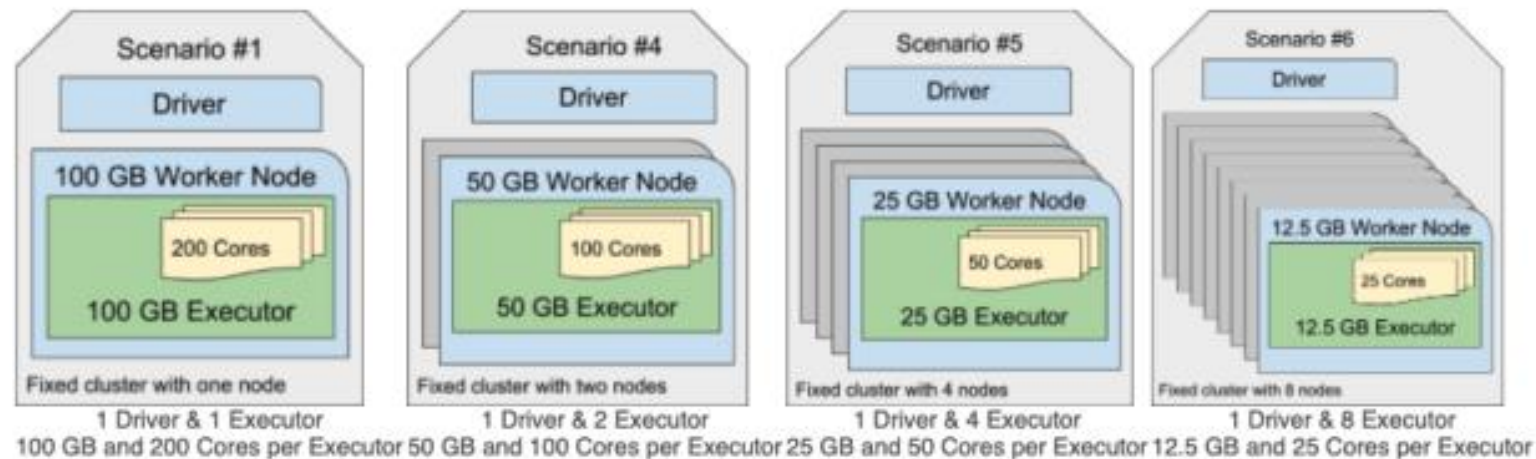
Question 12: Spark Architecture

Which of the following DataFrame commands will NOT generate a shuffle of data from each executor across the cluster?

- A. `df.reduce()`
- B. `df.collect()`
- C. `df.orderBy()`
- D. `df.sort()`
- E. `df.distinct()`
- F. `df.join()`

Question 12: Spark Architecture

Which of the following cluster configurations will ensure the completion of a Spark application in light of a worker node failure?



Note: each configuration has roughly the same compute power using 100GB of RAM and 200 cores.

- A. Scenario #1
- B. They should all ensure completion because worker nodes are fault-tolerant.
- C. Scenario #4
- D. Scenario #5
- E. Scenario #6

Question 13: Spark Architecture

Which of the following data structures are Spark DataFrames built on top of?

- A. RDDs
- B. Arrays
- C. Vectors
- D. SQL Tables

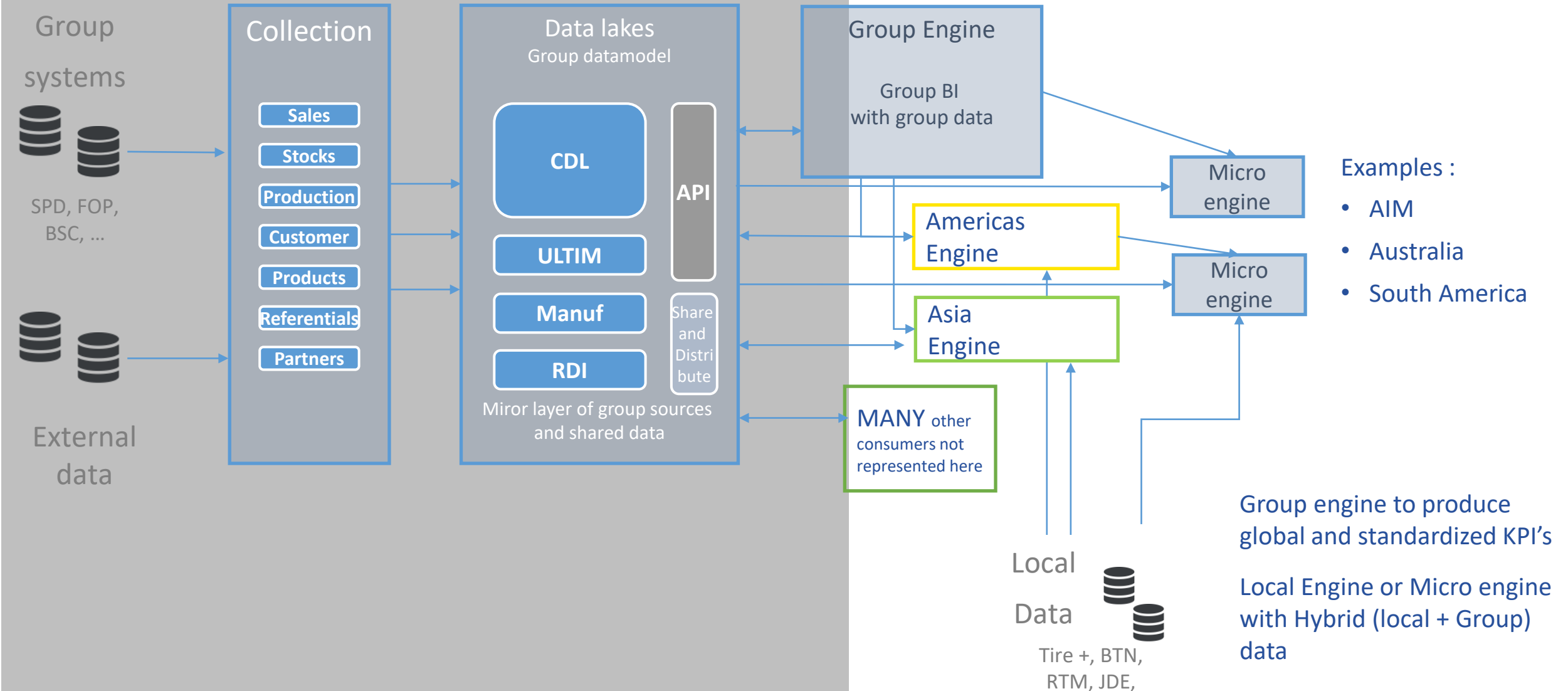
Question 14: Spark Architecture

Which of the following describes a shuffle?

- A. A shuffle is the process by which data is compared across partitions.
- B. A shuffle is the process by which data is compared across executors.
- C. A shuffle is the process by which partitions are allocated to tasks.
- D. A shuffle is the process by which tasks are ordered for execution.

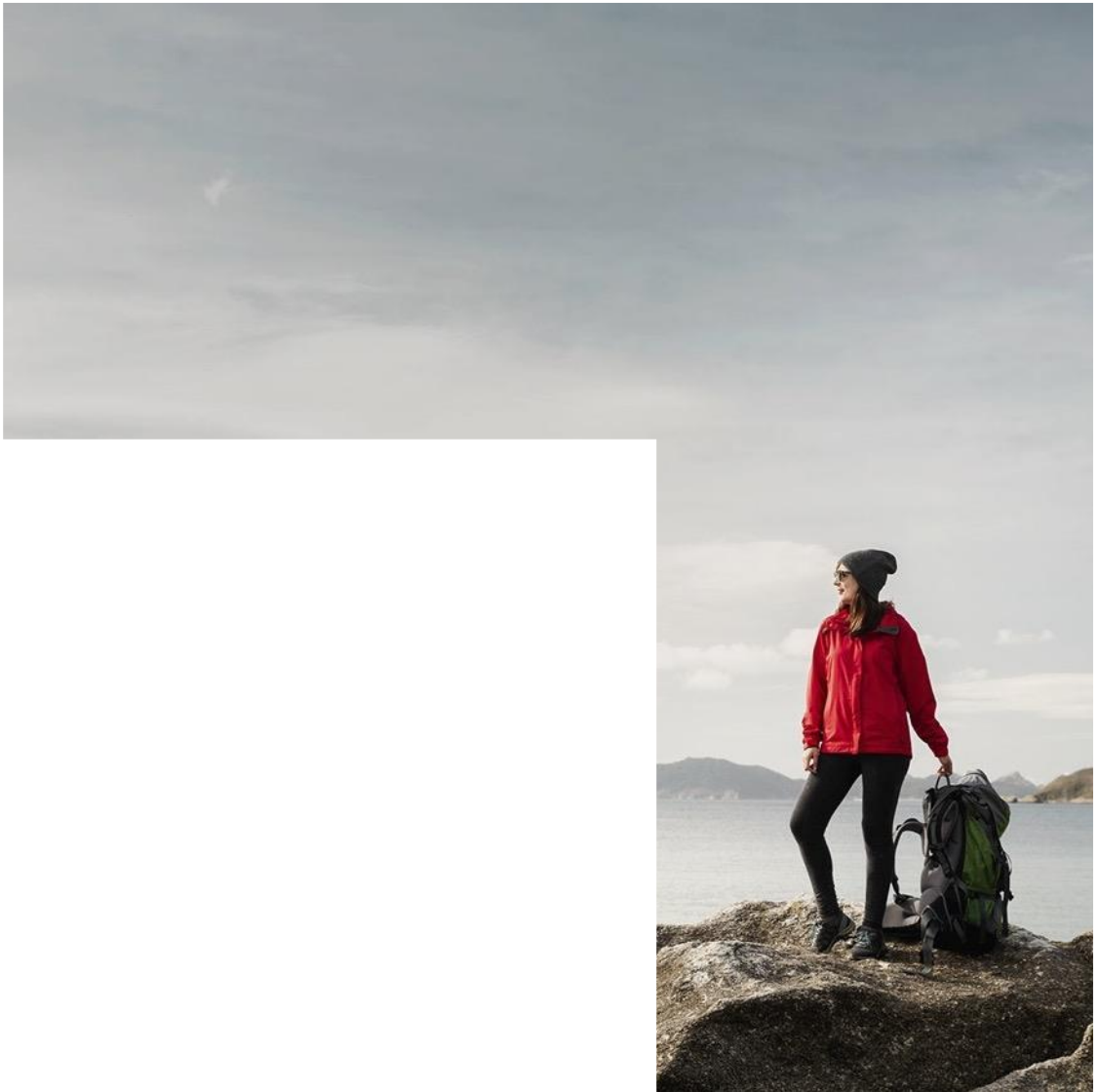
Federated data lakes enable data engines to combine group and local data in an hybrid mode

....flexibility at the edge



Syntax Comparison and Review

Let's do a review of all we learnt about the
syntax between Pyspark and Spark SQL



ADD A NEW COLUMN

Pyspark

```
df.withColumn("NewId",  
col("id")+1)
```

Spark SQL

```
df = spark.sql("select id,  
id+1 as NewId from table")
```

ROUNDING

Pyspark

```
df.withColumn("QUANTITY",  
  round(col("Quantity"),  
  2)
```

Spark SQL

```
df = spark.sql("select  
  round(Quantity, 2) as  
  QUANTITY from table ")
```

CAST TIMESTAMP TO DATE

Pyspark

```
df.withColumn("NewDate",  
date_format(to_date(col(  
"Timestamp"), "yyyy-MM-  
dd")))
```

Spark SQL

```
df = spark.sql("select  
to_date(Timestamp, "yyyy-  
MM-dd") as NewDate from  
table1
```

GROUP BY & AGGREGATE

Pyspark

```
df.groupBy("CAD_C").agg  
(func.sum("QTY").alias(  
"QUANTITY"))
```

Spark SQL

```
df = spark.sql("select  
cad_c,  
sum(QTY) as QUANTITY  
from table  
group by cad_c")
```

GROUP BY & COUNT

Pyspark

```
df.groupBy("id").agg  
(func.count("id"))
```

Spark SQL

```
df = spark.sql("select id,  
count(*) from table group  
by id")
```

WHEN & OTHERWISE

Pyspark

```
df.withColumn("IO_ROUTING",  
when(col("SRC_STATUS")== "2", lit  
("PIT"))  
.when(col("SRC_STATUS")== "3",  
lit("GIT")).otherwise(lit("NA")  
))
```

Spark SQL

```
df = spark.sql("select  
Case when IO_ROUTING = 2,  
Then PIT  
Case when IO_ROUTING = 3,  
Then GIT  
Else NA  
End as IO_ROUTING  
from table")
```

RENAME COLUMN

Pyspark

```
df.withColumnRenamed("A",  
"STATUS")
```

Spark SQL

```
df = spark.sql("select A AS STATUS  
from table")
```


ORDER BY

Pyspark

```
df.select("date",  
"quantity").orderBy(desc("date"  
))
```

Spark SQL

```
df = spark.sql("select date,  
quantity from table order by date  
desc")
```

WHERE

Pyspark

```
df.where(df.ID == "11234")
```

Spark SQL

```
df = spark.sql("select * from table  
where ID = \"11234\"")
```

UNION

Pyspark

```
df1.union(df2)
```

Spark SQL

```
df = spark.sql("select * from table1  
unionAll select * from table2 ")
```

UNION

Pyspark

Same name
Same number of columns
Same order of columns

Spark SQL

Same number of columns
Same order of columns